

Content Authenticity and Provenance for Audiovisual Collections

An Environmental Review

David Rice

August 2026

Contents

1	Content Authenticity and Provenance for Audiovisual Collections: An Environmental Review	3
1.1	Authorship	3
1.2	Overview	3
1.3	Background and Motivation	4
1.4	A Framework for Assessing CAP Maturity	4
1.5	C2PA and the Archival Context	6
1.6	Report Scope and Limitations	7
2	CAP Registry Reference	7
2.1	Common Fields (all three registries)	8
2.2	Tools Registry	9
2.3	Formats Registry	11
2.4	Mechanisms Registry	13
3	Tools Registry	15
3.1	Siegfried	16
3.2	FIDO	17
3.3	MediaInfo	17
3.4	ExifTool	18
3.5	JHOVE	19
3.6	FFprobe	20
3.7	md5deep / hashdeep	20
3.8	Fixity Pro	21
3.9	BagIt-Python	22
3.10	BWF MetaEdit	22
3.11	FFmpeg	23
3.12	HandBrake	24
3.13	c2patool	25
3.14	c2pa-rs	26
3.15	c2pa-python	26
3.16	Content Credentials Verify	27
3.17	Archivematica	28

3.18	SynthID	29
3.19	PRONOM	29
3.20	yt-dlp	30
3.21	rsync	31
4	Formats Registry	32
4.1	Broadcast WAVE Format (BWF)	33
4.2	FLAC (Free Lossless Audio Codec)	34
4.3	JPEG	36
4.4	PNG (Portable Network Graphics)	38
4.5	TIFF (Tagged Image File Format)	40
4.6	MP4 / ISO Base Media File Format	43
4.7	Matroska (MKV)	44
4.8	C2PA Manifest (Content Credentials)	46
4.9	XMP (Extensible Metadata Platform)	48
4.10	EXIF (Exchangeable Image File Format)	49
4.11	IPTC Photo Metadata Standard 2025.1	50
4.12	PREMIS (Preservation Metadata)	51
4.13	BagIt (RFC 8493)	53
4.14	METS (Metadata Encoding and Transmission Standard)	54
5	Mechanisms Registry	55
5.1	FLAC Specific Integrity Features	56
5.2	BWF Specific Features (BEXT Chunk)	58
5.3	JPEG Specific Features (APP Segments)	60
5.4	PNG Specific Features (Chunks)	62
5.5	TIFF Specific Features (IFD Tags)	64
5.6	MP4 / ISOBMFF Specific Features (Atoms)	66
5.7	Matroska Specific Features (EBML Tags, CRC-32, and Attachments)	68
5.8	Vorbis Comment	71
5.9	ID3 General Encapsulated Object (GEOB) — C2PA	73
5.10	BWF BEXT Chunk	75
5.11	BWF LIST-INFO Chunk	77
5.12	BWF XML Chunks (aXML / iXML)	79
5.13	BWF Audio-Data MD5 Checksum	80
5.14	BWF C2PA Chunk	82
5.15	XMP Embedded Metadata	85
5.16	EXIF Embedded Metadata	87
5.17	IPTC Photo Metadata	89
5.18	JUMBF C2PA Manifest Store (Embedded)	91
5.19	C2PA Sidecar Manifest	94
5.20	PREMIS Sidecar / Embedded Event Record	97
5.21	BagIt Package Fixity	99
6	Featured Tools: Content Authenticity and Provenance in Practice	102
6.1	FFmpeg	102
6.2	BWF MetaEdit	114
6.3	vrecord	118

6.4	MediaInfo	126
6.5	embARC	128
7	Appendix: embARC and Content Authenticity and Provenance (CAP)	130
7.1	An Evaluation of embARC and CAP metadata	130
7.2	DPX Header Fields That Carry Provenance	131
7.3	C2PA and the DPX Format Support Problem	133
7.4	4. Options Considered	134
7.5	5. Recommended Approach: Sequence-Level Sidecar (Option 2)	135
7.6	C2PA Actions in Scope for embARC	137
7.7	embARC as Claim Generator vs. Manifest Consumer	138
7.8	Implementation Notes and Release Considerations	139
7.9	A Draft Proposal of embARC Command Line Options for C2PA	139
8	Appendix: BWF MetaEdit and Content Authenticity and Provenance (CAP)	141
8.1	An Evaluation of BWF MetaEdit and CAP Metadata	141
8.2	BWF’s Embedded Metadata and Provenance Fields	141
8.3	C2PA and the WAV Format	142
8.4	Options Considered	142
8.5	Recommended Approach	144
8.6	Relationship Between BWF MetaEdit’s MD5 and C2PA Hard Binding	144
8.7	Proposed Command Line Options for BWF MetaEdit C2PA Integration	145

1 Content Authenticity and Provenance for Audiovisual Collections: An Environmental Review

1.1 Authorship

Author: David Rice

Prepared for: Library of Congress

Date: August 2026

1.2 Overview

This report provides an environmental overview of Content Authenticity and Provenance (CAP) metadata for digital audiovisual collections, covering both digitized and born-digital materials. It examines the current state of CAP-related tools, formats, and mechanisms available to the audiovisual preservation community, with particular attention to free and open source software (FOSS) and to the C2PA (Coalition for Content Provenance and Authenticity) specification as the emerging technical standard for signed provenance manifests.

The report is structured in three parts:

1. **Registries:** A systematic registry of tools, file formats, and provenance mechanisms, each summarized against a set of CAP-relevant terms. The registry provides a reference for under-

standing what exists in the ecosystem and how each component relates to content authenticity and provenance.

2. **Featured Tools:** A closer examination of selected FOSS tools that are central to audiovisual preservation workflows (FFmpeg, BWF MetaEdit, vrecord, MediaInfo, and embARC). For each tool, this section describes its current role in provenance documentation, worked examples showing how its outputs can be mapped to C2PA manifest structures, and considerations for integration into a CAP workflow. This section also maps tool capabilities to the emerging TCR4CAP (Tiered Community Recommendations for Content Authenticity and Provenance) criteria being developed by the FADGI Audio-Visual Working Group.
3. **Appendices:** Technical appendices detailing a proof-of-concept implementation plan for integrating C2PA manifest generation into embARC and BWF MetaEdit, the FADGI metadata auditing and correction tools for DPX, MXF, and WAV files. The appendices address the specific architectural challenges of embedding or sidecarring C2PA manifests in DPX image sequences, including hashing strategies that preserve each tool’s design priority of not modifying audiovisual data.

1.3 Background and Motivation

With the rapid proliferation of generative AI tools, reliable information about the origins of images and audio has become increasingly critical. Generative AI images, along with other forms of media manipulation, have an alarming capacity to mislead, confuse, or distort perceptions of current moments and historic realities. A blending of artificial records with authentic ones adds fear, uncertainty, and doubt, and can place persuasion and truth into a confusing conflict.

Archives generally serve as institutions of trust by storing records, sustaining them as authentically as possible against manipulation and decay, and then facilitating access accurately for future users. As a result, archivists have long considered the concepts of Content Authenticity and Provenance in the design of their workflows. These concepts can be seen clearly through the principal standards on archival workflow design such as OAIS (the Open Archival Information System reference model) and the NDSA Levels of Digital Preservation, and in metadata standards themselves such as PREMIS.

For archives, the abundance of media and information artificially produced by generative AI has accelerated the urgency of CAP and expanded the expectations placed upon it. The C2PA specification, developed by a coalition of companies including Adobe, Google, Microsoft, Sony, and others, provides a technical framework for cryptographically signed provenance manifests that can be embedded in or attached to digital files. While C2PA was designed primarily for editorial photography and journalism workflows, its architecture is general enough to apply to audiovisual preservation, and this report examines that applicability in detail.

1.4 A Framework for Assessing CAP Maturity

To organize the environmental review, this report assesses tools, formats, and mechanisms against four qualities that together describe a spectrum of CAP maturity. These qualities are not a tiered standard (such as the case with FADGI Working Group’s forthcoming TCR4CAP framework), but rather an analytical model for evaluating where the field currently stands and where further effort and collaboration could have the most impact.

1.4.1 Awareness

At the most fundamental level, archives should be aware of what information is available that could support the perceived authenticity or provenance of their records. Such information may be generated through intentional practices such as registration forms, may happen to exist within a digital file, or be present in logs related to the records.

Examples include metadata embedded within a digital record that provides context to the record's creation, such as the name and version of the creation tool, the creation date, or the name of the user that created it. The purpose of such data is to give context to the creation process behind the media and provide some answers to who created it and how, and what steps or tools were involved.

There are levels within this category. At a basic level, an archive may be aware that a QuickTime file contains the name and version of the tool that created it and the date of the creation event, all self-contained within the media file. At a more significant level, the archive may have that awareness plus store a log of the tool that created the file, which can provide fuller context of the characteristics and variables of the creation event. Many of the tools reviewed in this report (vrecord, FFmpeg's FFREPORT, BWF MetaEdit's export functions, embARC's audit reports) generate this kind of contextual logging and the featured tools section of this report demonstrates how these logs can be retained as part of the preservation package.

1.4.2 Structure

Moving beyond awareness, CAP information should be structured in a way that is consistently parseable, comparable across files and formats, and preservable over time. Unstructured free-text notes embedded in a file's comment field are a form of awareness but not of structure; they cannot be reliably extracted, validated, or compared.

Structured provenance takes several forms in the current ecosystem. The EBU BEXT chunk in Broadcast WAVE files places provenance information in defined fields with controlled vocabularies: Originator, OriginatorReference, OriginationDate, CodingHistory. The DPX header fields similarly encode provenance in fixed byte positions to describe the provenance of digitized moving image frames. PREMIS provides a structured XML schema for recording events, agents, and objects. The C2PA specification provides a highly formal structure: a CBOR-encoded manifest store containing assertions and signed according to a specified cryptographic chain.

The registries in this report assess whether and how each tool and format presents structured provenance information. The featured tools section of this report demonstrates how the semi-structured outputs of tools like FFmpeg (FFREPORT logs, fprobe JSON) can be mapped into the formal structure of C2PA action assertions.

1.4.3 Relational

Provenance information gains meaning when it is related across files, transformations, and time. A single file's creation metadata is useful but a chain that links a preservation copy to its access derivatives, and all of the above to the digitization event that produced them, is even more useful.

The C2PA ingredients model is designed to create relational provenance cryptographically. Each manifest can declare its input assets as ingredients, and if those inputs carry their own manifests,

the chain is verifiable back to the point of origin. PREMIS events create relational provenance by linking event records to object records via identifiers. BWF MetaEdit’s OriginatorReference and UMID fields create relational provenance by giving each file a persistent identifier that can be referenced from other systems.

This report examines whether and how each tool and format supports relational provenance. The worked examples in the featured tools section illustrate both the potential and the limits of the C2PA ingredients model, particularly at the analog-to-digital transition where vrecord captures videotape signals that have no prior digital manifest to reference as an ingredient.

1.4.4 Verifiable

At the highest level of maturity, provenance claims should be independently verifiable. A file that asserts “this content was captured on July 8, 2026, by a particular named technician, using a DeckLink SDI 4K card” is making a claim. A file with that same assertion, cryptographically bound to a content hash that can be independently re-evaluated and compared against the signed manifest, is making a verifiable claim. Verifiability requires both fixity (the ability to detect whether content has changed) and authenticity (the ability to confirm that a provenance record was produced by a stated agent and has not been altered since).

Several tools in this report provide elements of verifiability without C2PA. BWF MetaEdit’s audio-data MD5 chunk provides content fixity that is independent of metadata changes. vrecord’s framemd5 output provides per-frame fixity generated at the moment of capture. embARC’s guarantee of not touching image data provides a basis for asserting that content has not been modified during metadata correction. C2PA builds on these foundations by wrapping fixity assertions and action claims in a cryptographically signed manifest that is tamper-evident. Any modification to either the content or the manifest after signing would break the signature.

This report examines the current state of verifiability across the tools and formats in the audiovisual preservation ecosystem, and the appendices detail how a C2PA integration in embARC could extend verifiability to DPX sequence workflows at scale.

1.5 C2PA and the Archival Context

The C2PA specification defines a manifest store format (based on JUMBF, the JPEG Universal Metadata Box Format), a set of standard assertion types (including actions, ingredients, and hash assertions), and a cryptographic signing model. Manifests can be embedded in supported file formats (including JPEG, PNG, ISO-BMFF/MP4, and RIFF/WAV) or carried as sidecar files for formats that do not support embedding.

For audiovisual archives, several aspects of C2PA require careful consideration:

- **Supported formats:** Many preservation-grade formats used in archives (Matroska, DPX, MXF) are not currently C2PA-supported for embedding. This report examines sidecar strategies for these formats and the specific challenges of DPX sequences, which can contain hundreds of thousands of individual files representing a single work.
- **The analog-to-digital transition:** C2PA’s ingredients model was designed for digital-to-digital provenance chains. When a file is created by digitizing an analog source (videotape,

film, audio tape), the ingredient has no prior digital manifest. This report considers how the ingredients model can be honestly applied at this transition, declaring the analog source with descriptive metadata but acknowledging that the cryptographic chain begins at digitization where hashing becomes feasible.

- **Scale and I/O:** Feature-length content digitized as DPX sequences can produce 100,000 or more individual frames. Hashing every file’s image data for a C2PA manifest is a significant input/output operation. The appendices examine a hashing strategy that hashes only the image data region of each DPX file (excluding the mutable header), making the expensive processing cost a one-time operation per sequence rather than a recurring cost for each metadata correction.
- **Signing infrastructure:** C2PA requires certificates and private keys for signing. Archives and archival processing tools should consider establishing and managing signing credentials, a potentially significant requirement within most current preservation workflows. This report treats the signing step as a post-processing operation separate from the tools that generate provenance data (FFmpeg, BWF MetaEdit, vrecord, embARC), using the c2pa Rust SDK or c2patool; however, this report also examines scenarios where select tools could be expanded to incorporate signatures.

1.6 Report Scope and Limitations

This report focuses on free and open source tools, as these tools are often more amenable to community-driven enhancement, skill-sharing, and sustainable scaled deployment. Commercial tools that participate in the C2PA ecosystem (Adobe Photoshop, Microsoft Designer, and others) are outside the scope of this review but are acknowledged as the primary drivers of C2PA adoption in the broader market.

The report reflects the state of the field as of mid-2026. C2PA support in free and open source audiovisual tools is in very early stages. No mainstream preservation tool currently generates or reads C2PA manifests natively, and the experimental forks that add C2PA to FFmpeg are in proof-of-concept implementations. The TCR4CAP framework, referenced throughout this report, was under development by the FADGI Audio-Visual Working Group at the time of writing. Efforts were made to keep the development of the two reports in sync. It is acknowledged that the C2PA Technical Specifications are also a work in progress with many changes ongoing. The recommendations and implementation plans in this report should be revisited as these related developments mature.

The examples in the featured tools section derive C2PA manifests from real tool logs (such as FFmpeg and vrecord logs). These manifests are offered as examples of potential but currently are not generated by any existing tool and should be treated as draft ideas for C2PA implementations. The JSON representations and logs within this report are examples and in some cases are simplified or truncated for readability.

2 CAP Registry Reference

This section defines the fields and controlled vocabularies used within the entries of all three CAP registries of this environmental review: **Tools**, **Formats**, and **Mechanisms**. The registries are

internally managed in XML with the intention of presenting its information in transformed reports and tables.

2.1 Common Fields (all three registries)

2.1.1 Name

The human-readable display name of the tool, format, or mechanism.

2.1.2 FDD References

Links to Library of Congress Format Description Documents (FDDs) that describe the format or mechanism. Each reference carries a **name** attribute of the name of the FDD, as well as an **ID** attribute, a **status** attribute (see the definition list below), and a **url** attribute that links to the FDD:

2.1.2.1 Status Attribute values

Value	Meaning
full	The FDD fully describes this format or mechanism
partial	The FDD exists and is relevant but does not fully cover the focused topic
preliminary	The FDD is in preliminary/draft status at the Library of Congress
none	No applicable FDD exists

2.1.3 URL

One or more reference URLs. Each URL may include an optional **type** attribute to contextualize the link:

type value	Meaning
homepage	Official project or product homepage
repo	Source code repository (e.g. GitHub)
spec	Normative specification document
guidelines	Recommendations and/or rules on implementations
<i>(none)</i>	General reference with no specific type assigned

2.1.4 Description

Descriptions are meant to be concise and factual and give an overview on what the entry represents.

2.1.5 TCR4CAP Comments

This is a structured set of observations against the six TCR4CAP assessment criteria. These are intended as descriptive and observational notes, and not as scores or editorialized reviews. All six criteria are present for each entry and it is noted when the criteria is out of scope. A short summary of each of the criteria is listed here below; see FADGI’s “Framework for Tiered Community Recommendations for Content Authenticity and Provenance” (TCR4CAP) documentation for more context of any of these criteria.

Criteria	Scope
Awareness	Whether the tool/format/mechanism has any CAP awareness
Tamper Evidence	Whether modification of content or metadata can be detected
Binding	Whether metadata is linked to the media content
AI Attribution	Whether AI generation or training provenance can be recorded
Substantiation	Whether provenance claims can be verified against external anchors
Interoperability	License, standards basis, and breadth of implementation

2.2 Tools Registry

Tools are software applications, libraries, SDKs, online services, or reference registries relevant to content authenticity and provenance workflows.

2.2.1 Category (type)

This is a broad functional category of the tool.

Value	Meaning
Identification Tools	File format identification (PRONOM/FDD signatures)
Characterization Tools	Technical metadata extraction and format validation
Fixity Tools	Checksum computation, audit, and package integrity
Metadata Editors	Tools that read and write embedded metadata
Transcoding Tools	Audio/video format conversion and processing
C2PA Tools	Tools with native C2PA manifest support
Digital Preservation Systems	End-to-end preservation workflow systems
Watermarking Tools	Content-level imperceptible watermarking
Metadata Standards and Registries	Authoritative format or metadata registries
Capture and Transmission Tools	Tools that capture, download, or transfer media files (e.g. yt-dlp, rsync)

2.2.2 Version

The current (as of the writing of this document) release version of the tool in ISO 8601 format (YYYY-MM-DD).

2.2.3 License

The software license under which the tool is distributed. Examples: **Apache-2.0**, **MIT/Apache-2.0**, **GPL-2.0**, **LGPL-2.1**, **CC0-1.0**, **Public Domain**, **AGPL-3.0**, **Proprietary**

2.2.4 Provenance Scope

The level at which the tool operates with respect to provenance, assessed across three dimensions: scope, durability, and depth.

2.2.4.1 Scope

Value	Meaning
file-level	Operates on individual files
package-level	Operates on packages or collections of files (e.g. BagIt bags)
content-level	Operates on the content signal itself (e.g. watermarking)
n/a	Not applicable

2.2.4.2 Durability The extent to which the tool preserves provenance across transformations it performs on the file. Does the tool preserve existing provenance data through its operations? Does it record or append its own actions to the chain of custody?

Value	Meaning
preserves	Preserves existing provenance data through transformations
preserves-with-action	Preserves existing provenance and appends its own actions to the chain of custody
strips	Removes or overwrites existing provenance data during operations
n/a	Not applicable (tool does not transform content)

2.2.4.3 Depth The extent of provenance detail the tool captures or processes.

Value	Meaning
full	Captures, processes, or preserves complete provenance detail (e.g. full C2PA manifest traversal)
summary	Captures or processes key provenance fields (e.g. tool name, creation date)
none	No provenance detail captured or processed
n/a	Not applicable

2.2.5 C2PA Support

The degree to which the tool supports the C2PA specification, expressed as a functional capability level.

Value	Meaning
create-sign	Can create, embed, and sign C2PA manifests (signing requires an external certificate from a CA on the C2PA Trust List); can also verify
verify	Can cryptographically verify C2PA manifest signatures and hard binding, and display manifest content; cannot create or sign
detect-parse	Can detect and structurally parse C2PA manifests but cannot verify signatures, create, or sign
none-official	No official C2PA support; experimental forks may exist
none	No C2PA support

2.3 Formats Registry

Formats are file formats or metadata standards relevant to content authenticity and provenance. This includes container formats (BWF, MP4), image formats (JPEG, PNG), and metadata formats (XMP, PREMIS, BagIt).

2.3.1 Type

The broad media or metadata category of the format.

Value	Meaning
audio	Audio container or codec format
moving image	Video container format
image	Still image format
metadata	Metadata standard or packaging format

2.3.2 Structure

The internal structural model of the format.

Value	Meaning
binary-chunk	Binary format organised as named chunks (e.g. RIFF/WAVE)
binary-block	Binary format organised as typed blocks (e.g. FLAC)
binary-segment	Binary format organised as typed segments (e.g. JPEG)
binary-atom	Binary format organised as nested atoms/boxes (e.g. MP4)
binary-ebml	Binary format using EBML encoding (e.g. Matroska)
binary	Generic binary format (e.g. TIFF IFD)
cbor-jumbf	CBOR-encoded data in a JUMBF container (C2PA manifest)
xml-rdf	RDF/XML serialisation (XMP)

Value	Meaning
xml-xmp	XMP-carried vocabulary (IPTC Photo Metadata)
xml	Generic XML (PREMIS, METS)
text-manifest	Plain-text manifest files (BagIt)

2.3.3 Readability (type attribute)

The extent to which the format specification is openly available.

Value	Meaning
open	Specification is publicly available at no cost no licensing restrictions on implementations.
published-standard	The specification is published openly, but may have licensing or patent impacts.

2.3.4 Verifiability (type attribute)

The highest level of integrity or authenticity verification the format supports.

Value	Meaning
signed	Cryptographic signature with hard binding to content
integrity-only	Fixity values or checksums present; but not necessary signed or provenance chain
none	No integrity or authenticity mechanism

2.3.5 Persistence

A description of the format's suitability and institutional support for long-term preservation. This field is descriptive (no controlled vocabulary) and should address: - Whether the format is widely adopted as an archival primary format - The level of long-term institutional support (standards bodies, community organizations, national archives) - Any relevant recommendations from FADGI or other professional guidelines - Known limitations on archival use (e.g., lossy compression, proprietary dependencies)

2.3.6 C2PA Support Modes

A format may support C2PA delivery via one or more modes. Each mode has a **type** attribute:

type value	Meaning
embedded-app11	Embedded in JPEG APP11 (0xFFE0) segment
embedded-cabx-chunk	Embedded in PNG caBX chunk
embedded-riff-chunk	Embedded in a custom C2PA RIFF chunk (BWF/WAV)

type value	Meaning
<code>embedded-uuid-atom</code>	Embedded in MP4/ISOBMFF uuid atom
<code>embedded-via-id3</code>	Embedded via ID3 GEOB frame (FLAC)
<code>embedded</code>	Embedded via format-specific container (generic, used in C2PA manifest entry)
<code>sidecar</code>	Delivered as a standalone <code>.c2pa</code> sidecar file
<code>none</code>	No C2PA support defined

2.3.7 Mechanisms

A list of cross-references to entries in the Mechanisms registry that describe how metadata is stored in or alongside this format. Each link resolves to the named mechanism’s detail page.

2.4 Mechanisms Registry

Mechanisms are the specific technical methods by which metadata is stored in, attached to, or associated with a media asset. Since many formats rely on similar methods to store this data, mechanisms are described separately from formats with a relationship. Note also how the C2PA technical specifications group supported formats together based on shared mechanisms.

2.4.1 Type

The broad category of the mechanism.

Value	Meaning
<code>format-specific</code>	A chunk, block, or segment defined by the host format specification
<code>embedded-metadata</code>	A general-purpose embedded metadata standard (XMP, EXIF, IPTC)
<code>embedded-c2pa</code>	A C2PA manifest store embedded in the host file
<code>integrity</code>	A checksum or hash stored within the file
<code>sidecar-c2pa</code>	A C2PA manifest store delivered as a separate sidecar file
<code>sidecar-metadata</code>	A metadata document delivered as a separate sidecar file
<code>package-integrity</code>	Checksum manifests at the package level (BagIt)

2.4.2 Editability — File-level (type attribute)

The extent to which the container that holds this mechanism can be written to.

Value	Meaning
<code>embedded-writable</code>	The chunk/block/segment is embedded and can be written or updated by tools
<code>sidecar-writable</code>	The mechanism lives in a sidecar file that can be written independently
<code>package-writable</code>	The mechanism lives in package-level manifest files

2.4.3 Editability — Content-level (type attribute)

How the metadata content itself behaves when updated.

Value	Meaning
overwrite	Values are replaced in place on write; no history is preserved
append	New entries are conventionally appended; prior entries remain
append-only	The store is structurally append-only; prior entries are immutable
fixed	The value is computed at creation and cannot be changed without recomputing

2.4.4 Verifiability — Media Integrity (type attribute)

The extent to which the mechanism provides any integrity check over the primary media data (audio samples, image pixels, video frames).

Value	Meaning
signed	Includes a signature covering the media data (C2PA hard binding)
checksum	A hash or CRC covers the media data
external-only	Integrity is recorded externally (e.g. PREMIS fixity, BagIt manifest)
none	No media-data integrity mechanism

2.4.5 Verifiability — Metadata Integrity (type attribute)

The extent to which the mechanism provides any integrity check over the metadata itself.

Value	Meaning
signed	Includes a signature covers the metadata (C2PA manifest signing)
checksum	A hash or CRC covers the metadata chunk
none	No metadata integrity mechanism; fields can be silently overwritten

2.4.6 Verifiability — Chain of Custody (type attribute)

The extent to which the mechanism supports a verifiable provenance chain linking the asset to an external identity or signing authority.

Value	Meaning
signed	Includes a signed provenance chain (such as a C2PA manifest store)
external-only	Provenance chain exists but is held externally (PREMIS events, BagIt)
none	No provenance chain mechanism

2.4.7 Metadata Fields

Where defined, a table of the noted named fields carried by the mechanism that are relevant to CAP. Many fields are defined vaguely and could be or could not be used in a manner relevant to CAP, so this list is a selection of notable fields and not intended to be exhaustive. Each field has:

- **Field** — the field name as defined in the specification
- **CAP Concept** — the provenance concept the field addresses (see vocabulary below)
- **Description** — what the field records

2.4.7.1 CAP Concept vocabulary

Value	Meaning
actor-identity	Identity of a person, organisation, or software agent
asset-identity	A unique identifier for the asset itself
creation-datetime	Date and/or time of creation, capture, or origination
technical-provenance	Technical characteristics of the capture device or process
transformation-history	A record of processing steps applied to the asset
provenance-chain	A link in a verifiable chain of custody
media-integrity	A checksum or hash over the primary media data
metadata-integrity	A checksum or hash over the metadata itself
ai-attribution	AI generation, training, or model identity information
rights	Copyright, licensing, or usage rights statements

2.4.8 Used by Formats

A list of cross-references back to the Formats registry entries that reference this mechanism. Each link resolves to the named format's detail page.

3 Tools Registry

ID	Name
siegfried	Siegfried
fido	FIDO
mediainfo	MediaInfo
exiftool	ExifTool
jhove	JHOVE
ffprobe	FFprobe
md5deep	md5deep / hashdeep
fixity-pro	Fixity Pro
bagit-python	BagIt-Python
bwmfmetaedit	BWF MetaEdit
ffmpeg	FFmpeg
handbrake	HandBrake

ID	Name
c2patool	c2patool
c2pa-rs	c2pa-rs
c2pa-python	c2pa-python
verify-contentcredentials	Content Credentials Verify
archivematica	Archivematica
synthid	SynthID
pronom	PRONOM
yt-dlp	yt-dlp
rsync	rsync

3.1 Siegfried

ID: siegfried

Category: Identification Tools

Version: 1.11.4 (2026-01-23)

License: Apache-2.0

Provenance Scope: file-level

Provenance Durability: n/a

Provenance Depth: none

C2PA Support: none

URL: homepage: <https://www.itforarchivists.com/siegfried>

URL: repo: <https://github.com/richardlehane/siegfried>

Signature-based file format identification tool using PRONOM, LOC FDD, and Wikidata signatures. Outputs format identification results as JSON, CSV, or YAML. Widely used in digital preservation ingest workflows.

3.1.1 TCR4CAP Comments

Awareness: Siegfried identifies file formats against PRONOM and FDD signatures. Siegfried has no C2PA or provenance-specific awareness beyond format identification.

Tamper Evidence: Siegfried performs format identification only and does not assess or verify metadata integrity or tamper-evidence.

Binding: No binding mechanism. Format identification is based on file signatures.

AI Attribution: AI attribution is out of scope.

Substantiation: Outputs are structured and reproducible. Format identification results are aligned with PRONOM and FDD.

Interoperability: Apache-2.0. Widely implemented in digipres workflows. Outputs common, open formats: JSON, CSV, YAML.

3.2 FIDO

ID: fido

Category: Identification Tools

Version: 1.6.1 (2022-12-22)

License: Apache-2.0

Provenance Scope: file-level

Provenance Durability: n/a

Provenance Depth: none

C2PA Support: none

URL: <https://github.com/openpreserve/fido>

Format Identification for Digital Objects. Python-based PRONOM signature tool from Open Preservation Foundation. Outputs PUID identifiers for identified formats.

3.2.1 TCR4CAP Comments

Awareness: FIDO identifies file formats against PRONOM signatures. No C2PA or provenance-specific awareness beyond format identification.

Tamper Evidence: FIDO performs format identification only and does not assess or verify meta-data integrity or tamper-evidence.

Binding: No binding mechanism. Format identification is based on file signatures.

AI Attribution: AI attribution is out of scope.

Substantiation: Outputs are structured and format identification results are aligned with PRONOM.

Interoperability: Apache-2.0. Python-based. Widely implemented in digipres workflows.

3.3 MediaInfo

ID: mediainfo

Category: Characterization Tools

Version: 26.05 (2026-05-12)

License: BSD-2-Clause

Provenance Scope: file-level

Provenance Durability: n/a

Provenance Depth: summary

C2PA Support: detect-parse

URL: <https://mediaarea.net/en/MediaInfo>

Technical metadata extraction for audio and video files. As of December 2025 can detect and parse C2PA metadata in JPEG, PNG, and WAV. Outputs technical metadata as XML, JSON, HTML, or plain text. Widely used in broadcast, archives, and repository workflows.

3.3.1 TCR4CAP Comments

Awareness: MediaInfo extracts and reports technical metadata including C2PA detection and parsing (as of December 2025). Widely used in broadcast and archive workflows.

Tamper Evidence: MediaInfo can detect and report C2PA manifest presence. MediaInfo does not perform cryptographic verification of C2PA signatures.

Binding: MediaInfo reports C2PA manifest presence and characteristics, but does not verify the cryptographic binding between manifest and asset content.

AI Attribution: MediaInfo can report C2PA AI attribution assertions when present and does not generate or verify them.

Substantiation: Outputs are structured and technical metadata reports are reproducible. Widely used as an authoritative tool in preservation workflows for identifying the technical characteristics of audiovisual files.

Interoperability: BSD-2-Clause. Widely implemented across broadcast, archive, and repository tools. Outputs XML, JSON, HTML, plain text.

3.4 ExifTool

ID: exiftool

Category: Characterization Tools

Version: 13.59 (2026-05-27)

License: GPL/Artistic

Provenance Scope: file-level

Provenance Durability: n/a

Provenance Depth: full

C2PA Support: detect-parse

URL: <https://exiftool.org>

Reads and writes metadata in image, audio, and video files. Supports EXIF, IPTC, XMP, and many other mechanisms. C2PA/JUMBF metadata can be read and structurally parsed from JPEG, PNG, TIFF, DNG, MP4, MOV, HEIF, AVIF, WAV, AVI, WebP, PDF, SVG, GIF, and ID3v2. Exiftool can parse the JUMBF hierarchy and express individual CBOR assertion fields as structured tags. C2PA JUMBF data can be deleted from writable formats. ExifTool does not write or cryptographically verify C2PA manifests.

3.4.1 TCR4CAP Comments

Awareness: ExifTool reads and writes EXIF, IPTC, XMP, and many other mechanisms, and provides an extensive open-source structural parsing of C2PA/JUMBF content. Widely used across creative, archive, and repository workflows.

Tamper Evidence: ExifTool can structurally parse C2PA manifests but does not perform cryptographic verification. ExifTool can silently overwrite non-C2PA metadata without detection.

Binding: ExifTool parses and exposes C2PA hard-binding fields but does not verify the cryptographic binding between manifest and asset content.

AI Attribution: ExifTool can read and structurally parse C2PA AI attribution assertions, IPTC 2025.1 AI fields, and XMP AI attribution fields. ExifTool does not generate or verify C2PA AI attribution assertions.

Substantiation: Outputs are structured and metadata reports are reproducible. Widely used as an authoritative metadata extraction source.

Interoperability: Artistic/GPL. Widely implemented across creative, archive, and repository tools. Supports a multitude of metadata formats.

3.5 JHOVE

ID: jhove

Category: Characterization Tools

Version: 1.34.0 (2025-07-02)

License: LGPL-2.1

Provenance Scope: file-level

Provenance Durability: n/a

Provenance Depth: none

C2PA Support: none

URL: homepage: <https://jhove.openpreservation.org>

URL: repo: <https://github.com/openpreserve/jhove>

JSTOR/Harvard Object Validation Environment. Format validation and characterization for digital objects. Supports AIFF, ASCII, GIF, HTML, JPEG, JPEG 2000, PDF, TIFF, UTF-8, WAVE, and XML.

3.5.1 TCR4CAP Comments

Awareness: JHOVE performs format validation and characterization. JHOVE has no C2PA or provenance-specific awareness beyond format validation.

Tamper Evidence: JHOVE performs format validation only and does not assess or verify metadata integrity or tamper-evidence.

Binding: No binding mechanism. Format validation is based on format structure.

AI Attribution: AI attribution is out of scope.

Substantiation: Outputs are structured and format validation results are reproducible. Widely used in digipres ingest workflows.

Interoperability: LGPL-2.1. Widely used in digipres ingest workflows. Outputs XML.

3.6 FFprobe

ID: ffprobe

Category: Characterization Tools

Version: 8.1.1 (2026-05-04)

License: LGPL-2.1/GPL-2.0

Provenance Scope: file-level

Provenance Durability: n/a

Provenance Depth: summary

C2PA Support: none

URL: <https://ffmpeg.org/ffprobe.html>

Multimedia stream analyzer bundled with FFmpeg. Extracts technical metadata from audio/video containers and streams. Outputs JSON, XML, CSV, or plain text.

3.6.1 TCR4CAP Comments

Awareness: FFprobe extracts technical metadata from audio, video, image, and text files. FFprobe has no C2PA or provenance-specific awareness.

Tamper Evidence: FFprobe reports on technical metadata and does not assess or verify metadata integrity or tamper-evidence.

Binding: No binding mechanism. Technical metadata reporting is based on containers and encodings.

AI Attribution: FFprobe can report metadata tags that may include AI attribution fields by convention. No structured AI attribution vocabulary is defined.

Substantiation: Outputs are structured and technical metadata reports are reproducible. Widely used in broadcast and archive workflows.

Interoperability: LGPL-2.1/GPL-2.0. Widely implemented across broadcast, archive, and repository tools. Outputs JSON, XML, plain text.

3.7 md5deep / hashdeep

ID: md5deep

Category: Fixity Tools

Version: 4.4 (2014-01-29)

License: Public Domain

Provenance Scope: file-level

Provenance Durability: preserves-with-action

Provenance Depth: summary

C2PA Support: none

URL: <https://github.com/jessek/hashdeep>

Cross-platform checksum computation and audit tool. Supports MD5, SHA-1, SHA-256, Tiger, Whirlpool. hashdeep mode supports recursive directory checksumming and audit against a known-good manifest. Also supports Digital Forensics XML for reporting on checksums and file attributes.

3.7.1 TCR4CAP Comments

Awareness: md5deep/hashdeep computes and audits file checksums. No C2PA or provenance-specific awareness beyond fixity checking.

Tamper Evidence: Checksum computation and audit against a known-good manifest detects modification of any file. No signing mechanism.

Binding: Checksum entries bind each digest to a specific file path. No cryptographic binding between the manifest and an external identity or signing authority.

AI Attribution: AI attribution is out of scope.

Substantiation: Outputs are structured and fixity records are reproducible. Widely used in digipres transfer workflows.

Interoperability: Public Domain. Widely implemented across digipres, library, and archival transfer tools.

3.8 Fixity Pro

ID: fixity-pro

Category: Fixity Tools

Version: 1.14

Provenance Scope: file-level

Provenance Durability: preserves-with-action

Provenance Depth: summary

C2PA Support: none

URL: homepage: <https://fixitypro.com>

Standalone fixity checking application. Originally created by AVPS in 2013; transferred to the Open Preservation Foundation (OPF) in September 2025. Monitors file integrity over time with scheduled checks and email reporting. Supports MD5 and SHA-256.

3.8.1 TCR4CAP Comments

Awareness: Fixity Pro monitors file integrity over time. No C2PA or provenance-specific awareness beyond fixity checking.

Tamper Evidence: Scheduled fixity checks detect modification of monitored files over time. No signing mechanism.

Binding: Checksum entries bind each digest to a specific file path. No cryptographic binding between the manifest and an external identity or signing authority.

AI Attribution: AI attribution is out of scope.

Substantiation: Outputs are structured fixity reports with scheduled monitoring history. Supports email reporting for institutional workflows.

Interoperability: Maintained by OPF. Used in institutional preservation workflows.

3.9 BagIt-Python

ID: bagit-python

Category: Fixity Tools

Version: 1.9.0 (2025-06-13)

License: CC0-1.0

Provenance Scope: package-level

Provenance Durability: preserves-with-action

Provenance Depth: summary

C2PA Support: none

URL: <https://github.com/LibraryOfCongress/bagit-python>

Library of Congress BagIt implementation in Python. Creates and validates BagIt packages (RFC 8493) with payload manifests. Supports MD5, SHA-1, SHA-256, SHA-512.

3.9.1 TCR4CAP Comments

Awareness: BagIt-Python creates and validates BagIt packages. No C2PA or provenance-specific awareness beyond package fixity.

Tamper Evidence: Payload and tag manifest checksums detect modification of any file within the bag. No signing mechanism at the BagIt level.

Binding: Manifest entries bind each checksum to a specific file path within the bag. Tag manifests cover bag metadata files.

AI Attribution: AI attribution is out of scope.

Substantiation: Creates and validates structured, reproducible BagIt packages. Widely used in digipres transfer workflows.

Interoperability: CC0-1.0. Library of Congress reference implementation. Widely used across digipres, library, and archival transfer tools.

3.10 BWF MetaEdit

ID: bwfmetaedit

Category: Metadata Editors

Version: 26.01 (2026-02-02)

License: Public Domain / CC0

Provenance Scope: file-level
Provenance Durability: preserves-with-action
Provenance Depth: summary
C2PA Support: none
URL: homepage: <https://mediaarea.net/BWFMetaEdit>
URL: repo: <https://github.com/MediaArea/BWFMetaEdit>

Embed, validate, and export metadata in Broadcast WAVE Format files. Supports FADGI, EBU, and BEXT chunk metadata including CodingHistory, originator, origination date/time, and UMID. GUI and CLI.

3.10.1 TCR4CAP Comments

Awareness: BWF MetaEdit is the reference tool for BEXT chunk metadata in BWF files. Widely used in broadcast and archival audio workflows.

Tamper Evidence: BWF MetaEdit embeds and validates BEXT chunk metadata. BWF MetaEdit does not provide cryptographic tamper-evidence for the metadata it writes.

Binding: BWF MetaEdit embeds metadata directly in the BWF file structure. There is no cryptographic binding between the metadata chunks and audio content, but BWF MetaEdit can write and verify an MD5 chunk that hashes the audio content.

AI Attribution: BWF MetaEdit can embed CodingHistory, other BEXT fields, XMP, and other metadata that could document AI processing. No structured AI attribution vocabulary or workflow is defined.

Substantiation: Outputs are structured metadata in XML and CSV conforming to FADGI guidelines. Widely used as an authoritative metadata embedding tool in archival audio workflows.

Interoperability: Public Domain / CC0. Widely implemented across broadcast, archival, and audio tools. GUI and CLI.

3.11 FFmpeg

ID: ffmpeg
Category: Transcoding Tools
Version: 8.1.1 (2026-05-04)
License: LGPL-2.1/GPL-2.0
Provenance Scope: file-level
Provenance Durability: strips
Provenance Depth: none
C2PA Support: none-official
URL: <https://ffmpeg.org>

Open-source multimedia framework for transcoding, muxing, and processing audio and video. As of June 2026 there is no official C2PA support in the mainline codebase; however, a few ffmpeg

forks (c2pa_libffmpeg_eqty) add experimental C2PA manifest embedding in MP4. FFmpeg offers substantial control over preserving metadata while transcoding.

3.11.1 TCR4CAP Comments

Awareness: FFmpeg is a widely used open-source multimedia framework. No official C2PA support in the mainline codebase as of June 2026.

Tamper Evidence: FFmpeg strips most metadata by default. No cryptographic tamper-evidence for metadata it writes or preserves.

Binding: No cryptographic binding between metadata and media content. Metadata preservation depends on explicit command-line options.

AI Attribution: No structured AI attribution vocabulary. AI documentation requires explicit metadata injection via command-line options.

Substantiation: FFmpeg can preserve or inject metadata fields. No formal mechanism for publishing transformation history or CAP policies.

Interoperability: LGPL-2.1/GPL-2.0. Widely implemented across broadcast, archive, and repository tools. Supports virtually all audio/video formats.

3.12 HandBrake

ID: handbrake

Category: Transcoding Tools

Version: 1.11.2 (2026-06-07)

License: GPL-2.0

Provenance Scope: file-level

Provenance Durability: strips

Provenance Depth: none

C2PA Support: none

URL: homepage: <https://handbrake.fr>

URL: repo: <https://github.com/HandBrake/HandBrake>

Open-source video transcoder with GUI and CLI. Removes most metadata by default during transcoding.

3.12.1 TCR4CAP Comments

Awareness: HandBrake is a video transcoder. HandBrake has no C2PA or provenance-specific awareness.

Tamper Evidence: HandBrake removes most metadata by default when transcoding. No cryptographic tamper-evidence features.

Binding: No cryptographic binding between metadata and media content.

AI Attribution: AI attribution is out of scope.

Substantiation: No formal mechanism for publishing transformation history or CAP policies.

Interoperability: GPL-2.0. Widely used for video transcoding. GUI and CLI.

3.13 c2patool

ID: c2patool

Category: C2PA Tools

Version: 0.26.62 (2026-06-03)

License: MIT/Apache-2.0

Provenance Scope: file-level

Provenance Durability: preserves-with-action

Provenance Depth: full

C2PA Support: create-sign

URL: homepage: <https://opensource.contentauthenticity.org/docs/c2patool/c2patool-index/>

URL: repo: <https://github.com/contentauth/c2pa-rs>

The official CLI tool from the Content Authenticity Initiative for reading and writing C2PA manifests. Supports JPEG, PNG, MP4, WAV, and many other formats. Reads and displays manifest content, creates and signs new manifests, verifies manifest signatures and hard binding. Built on the c2pa-rs Rust SDK. Source for c2patool was moved into the contentauth/c2pa-rs repository in December 2024.

3.13.1 TCR4CAP Comments

Awareness: c2patool is the official CAI CLI tool in alignment with the development of the C2PA specification. Widely used as the reference implementation for C2PA manifest creation and verification.

Tamper Evidence: c2patool verifies C2PA manifest signatures and hard binding and detects any modification to manifest or asset content.

Binding: Creates and verifies C2PA hard binding (content hash) between manifest and asset content.

AI Attribution: Provides full support for C2PA AI attribution assertions.

Substantiation: c2patool creates and verifies complete C2PA manifest stores with full transformation history. Supports external trust anchors and timestamps.

Interoperability: MIT/Apache-2.0. Official CAI reference implementation. Supports all C2PA-defined embedding paths.

3.14 c2pa-rs

ID: c2pa-rs

Category: C2PA Tools

Version: 0.26.62 (2026-06-03)

License: MIT/Apache-2.0

Provenance Scope: file-level

Provenance Durability: preserves-with-action

Provenance Depth: full

C2PA Support: create-sign

URL: <https://github.com/contentauth/c2pa-rs>

Official Rust SDK for the C2PA specification from the Content Authenticity Initiative. Core library underlying c2patool, c2pa-python, and many other implementations. Provides full C2PA manifest creation, signing, embedding, and verification.

3.14.1 TCR4CAP Comments

Awareness: c2pa-rs is the official CAI Rust SDK with full C2PA specification support. c2pa-rs is the core implementation underlying the C2PA ecosystem.

Tamper Evidence: c2pa-rs provides full C2PA manifest signature verification and hard binding verification.

Binding: Creates and verifies C2PA hard binding (content hash) between manifest and asset content.

AI Attribution: Provides full support for C2PA AI attribution assertions.

Substantiation: c2pa-rs creates and verifies complete C2PA manifest stores with full transformation history. Supports external trust anchors and timestamps.

Interoperability: MIT/Apache-2.0. Official CAI reference library. Widely used as the foundation for C2PA implementations across languages and platforms.

3.15 c2pa-python

ID: c2pa-python

Category: C2PA Tools

Version: 0.32.12 (2026-06-03)

License: MIT/Apache-2.0

Provenance Scope: file-level

Provenance Durability: preserves-with-action

Provenance Depth: full

C2PA Support: create-sign

URL: <https://github.com/contentauth/c2pa-python>

The official Python binding for the c2pa-rs library from the Content Authenticity Initiative. Enables C2PA manifest creation, signing, embedding, and verification in Python workflows. Supports all formats and assertions supported by c2pa-rs.

3.15.1 TCR4CAP Comments

Awareness: The official CAI Python bindings with full C2PA specification support via c2pa-rs. Enables C2PA integration in Python-based preservation and DAM workflows.

Tamper Evidence: Provides full C2PA manifest signature verification and hard binding verification via c2pa-rs.

Binding: Creates and verifies C2PA hard binding (content hash) between manifest and asset content via c2pa-rs.

AI Attribution: Provides full support for C2PA AI attribution assertions via c2pa-rs.

Substantiation: Creates and verifies complete C2PA manifest stores with full transformation history via c2pa-rs. Supports external trust anchors and timestamps.

Interoperability: MIT/Apache-2.0. Official CAI Python bindings. Enables C2PA integration in Python-based workflows.

3.16 Content Credentials Verify

ID: verify-contentcredentials

Category: C2PA Tools

Version: online (unknown)

License: Proprietary

Provenance Scope: file-level

Provenance Durability: n/a

Provenance Depth: full

C2PA Support: verify

URL: <https://contentcredentials.org/verify>

Online C2PA manifest verification tool from the Content Authenticity Initiative. Upload a file to inspect its Content Credentials. Displays manifest assertions, signing certificate, timestamp, and hard binding status.

3.16.1 TCR4CAP Comments

Awareness: The official CAI online verification tool. Widely used for public verification of Content Credentials.

Tamper Evidence: Content Credentials Verify verifies C2PA manifest signatures and reports hard binding status and detects any modification to manifest or asset content.

Binding: Verifies C2PA hard binding status. The online tool requires file upload.

AI Attribution: Displays C2PA AI attribution assertions when present.

Substantiation: Displays complete manifest content including assertions, signing certificate, timestamp, and provenance chain.

Interoperability: Proprietary, online tool. Requires file upload.

3.17 Archivematica

ID: archivematica

Category: Digital Preservation Systems

Version: 1.18.0 (2025-09-26)

License: AGPL-3.0

Provenance Scope: package-level

Provenance Durability: preserves-with-action

Provenance Depth: full

C2PA Support: none

URL: homepage: <https://www.archivematica.org>

URL: repo: <https://github.com/artefactual/archivematica>

Open-source digital preservation system implementing the OAIS reference model. Generates PREMIS events, METS/PREMIS packages, and BagIt transfers. Integrates Siegfried, JHOVE, MediaInfo, and ExifTool for format identification and characterization. No native C2PA support as of June 2026.

3.17.1 TCR4CAP Comments

Awareness: Archivematica implements OAIS and generates PREMIS events and METS packages. Widely used in institutional preservation workflows. No native C2PA support.

Tamper Evidence: Archivematica generates BagIt packages with payload manifests and PREMIS fixity events. Tamper-evidence depends on the BagIt and PREMIS layers.

Binding: PREMIS fixity values and BagIt manifests bind checksums to specific file states. No cryptographic binding between metadata and asset content at the C2PA level.

AI Attribution: No native AI attribution support. PREMIS eventDetail and eventAgent fields could document AI processing actions by convention.

Substantiation: Archivematica generates structured PREMIS event records and METS packages documenting the full preservation workflow. Widely supported by repository systems.

Interoperability: AGPL-3.0. Widely implemented in institutional preservation workflows. Integrates with DSpace, Fedora, and other repository systems.

3.18 SynthID

ID: synthid

Category: Watermarking Tools

Version: unknown (unknown)

License: Proprietary

Provenance Scope: content-level

Provenance Durability: preserves

Provenance Depth: summary

C2PA Support: none

URL: <https://deepmind.google/models/synthid/>

SynthID is Google DeepMind’s imperceptible watermarking tool for AI-generated images, audio, text, and video. Embeds watermarks directly into content pixels/samples at generation time. Detection requires access to the SynthID detection API. No C2PA support. Operates independently of the C2PA manifest store.

3.18.1 TCR4CAP Comments

Awareness: SynthID is a published, deployed watermarking system from Google DeepMind. Widely discussed in AI provenance contexts.

Tamper Evidence: Imperceptible watermarks are embedded in content pixels/samples and are resistant to common image processing operations. Detection requires access to the SynthID API.

Binding: The watermark is embedded directly in the content signal. Binding is content-level and survives format conversion and metadata stripping.

AI Attribution: SynthID is purpose-built for AI-generated content attribution and embeds AI generation provenance directly in the content signal.

Substantiation: Watermark detection confirms AI generation provenance but does not provide a structured, auditable transformation history or external trust anchor.

Interoperability: Proprietary. Detection requires access to the SynthID API. No interoperability with C2PA or other open provenance standards.

3.19 PRONOM

ID: pronom

Category: Metadata Standards and Registries

Version: v122 (2026-01-19)

License: Open Government Licence v3.0

Provenance Scope: n/a

Provenance Durability: n/a

Provenance Depth: n/a

C2PA Support: none

URL: <https://www.nationalarchives.gov.uk/PRONOM/>

National Archives UK file format registry. Provides PUID (PRONOM Unique Identifier) identifiers used by Siegfried, FIDO, DROID, and Archivematica for format identification. Maintained by The National Archives with community contributions.

3.19.1 TCR4CAP Comments

Awareness: PRONOM is the authoritative file format registry for digital preservation. Widely used as the identification authority in institutional workflows.

Tamper Evidence: Tamper-evidence is out of scope.

Binding: Binding is out of scope.

AI Attribution: AI attribution is out of scope.

Substantiation: PRONOM provides authoritative, citable format identifiers (PUIDs) used across digipres tools and policies.

Interoperability: Open Government Licence. Widely implemented across digipres tools and systems. PUIDs are a de facto standard for format identification.

3.20 yt-dlp

ID: yt-dlp

Category: Capture and Transmission Tools

Version: 2026.07.04

License: Unlicense

Provenance Scope: file-level

Provenance Durability: strips

Provenance Depth: none

C2PA Support: none

URL: repo: <https://github.com/yt-dlp/yt-dlp>

yt-dlp is a command-line audio/video downloader that supports thousands of websites. It downloads media from web platforms and can embed metadata via FFmpeg post-processing. yt-dlp has no C2PA awareness and does not preserve C2PA manifests during download or re-encoding. When re-encoding is performed (e.g. format conversion or metadata embedding), embedded C2PA data generally does not survive (as per ffmpeg remuxing defaults). Direct downloads without re-encoding may preserve embedded C2PA if the source format and container are retained byte-for-byte.

3.20.1 TCR4CAP Comments

Awareness: yt-dlp has no awareness of C2PA or Content Credentials. It downloads media files and can embed general metadata (title, uploader, upload date, description) via FFmpeg, but does not detect, parse, or preserve C2PA manifests. Source-platform metadata such as YouTube AI disclosure labels is not captured as structured CAP data.

Tamper Evidence: No tamper-evidence capabilities. yt-dlp does not generate or verify fixity values. Downloads are byte-for-byte when no re-encoding occurs, but re-encoding via FFmpeg strips embedded C2PA and modifies the file, breaking any existing tamper-evidence chain.

Binding: No binding mechanism. Metadata embedded by yt-dlp (via `--embed-metadata`) is written into standard metadata containers (e.g. MP4 atoms, Matroska tags) with no cryptographic binding to content. Any pre-existing C2PA hard binding is lost if re-encoding occurs.

AI Attribution: No AI attribution support. yt-dlp does not detect or preserve AI provenance metadata from source platforms. YouTube AI disclosure labels and C2PA AI attribution assertions are not captured.

Substantiation: No provenance chain preservation. yt-dlp does not record or append its own download actions to a chain of custody. The `--embed-info-json` option can write a sidecar JSON file with download metadata (URL, extractor, timestamp), but this is informal and not structured CAP data. The relationship between the info.json sidecar and the media file is not maintained by any binding mechanism.

Interoperability: Open source (Unlicense). Cross-platform. Relies on FFmpeg for post-processing, inheriting FFmpeg's format support limitations regarding C2PA preservation.

3.21 rsync

ID: rsync

Category: Capture and Transmission Tools

Version: 3.4.4

License: GPL-3.0

Provenance Scope: file-level

Provenance Durability: preserves

Provenance Depth: summary

C2PA Support: none

URL: homepage: <https://rsync.samba.org/>

URL: repo: <https://github.com/RsyncProject/rsync>

rsync is an open source file synchronization and transfer tool that copies files between systems using a delta-transfer algorithm. rsync transfers files byte-for-byte, so existing, embedded CAP metadata within file structures is preserved. However, rsync has no CAP awareness and does not verify or validate metadata after transfer. Extended attributes are preserved only with the explicit `-X/--xattrs` flag; the default `-a` archive mode does not include xattrs.

3.21.1 TCR4CAP Comments

Awareness: rsync has no awareness of CAP metadata. It treats files as opaque byte streams. Embedded CAP data survives transfer because rsync preserves file content exactly, but rsync cannot detect, report, or validate the presence of CAP metadata. Institutions must use separate tools to verify CAP data integrity before and after transfer.

Tamper Evidence: rsync uses checksums internally for both its delta-transfer algorithm and its automatic post-transfer whole-file verification, but does not output or persist these checksums as a fixity record. The `-checksum (-c)` flag changes rsync’s file comparison method from size-and-time to a 128-bit checksum, but this is for internal transfer decisions and is not reported to the user. File-level fixity must be computed separately (e.g. with `bagit`, `md5sum`, `sha256sum`, etc) before and after transfer to create a persistent tamper-evidence record.

Binding: No binding mechanism. rsync does not create or verify any binding between metadata and content. Embedded C2PA hard binding is preserved structurally (since the file is transferred byte-for-byte), but rsync cannot report whether the binding is intact. Sidecar relationships (e.g. `.c2pa`, `.xmp`, or PREMIS XML files paired with media files) are not tracked or maintained by rsync; both files must be transferred explicitly and their association managed by the institution.

AI Attribution: No AI attribution support. rsync does not interact with or preserve AI provenance metadata beyond preserving the file bytes that contain it.

Substantiation: rsync does not natively record or append transfer actions to a structured chain of custody. However, the `-log-file` option can log transfer actions (source, destination, filenames, timestamps, itemized changes) to a file, and `-log-file-format` allows customization of the log output. This log can serve as an informal provenance record documenting where files came from and when they were transferred, though it is not structured CAP data. Institutions should integrate rsync logs into formal provenance systems (e.g. PREMIS events) for transfer documentation.

Interoperability: Open source (GPL-3.0). Universally available on Linux, macOS, and Unix-like systems. Widely used in digital preservation workflows for file transfer and synchronization. The `-aAX` flag combination preserves permissions, ownership, timestamps, ACLs, and extended attributes, but awareness of these flags and their CAP implications is the institution’s responsibility.

4 Formats Registry

ID	Name
<code>bwf</code>	Broadcast WAVE Format (BWF)
<code>flac</code>	FLAC (Free Lossless Audio Codec)
<code>jpeg</code>	JPEG
<code>png</code>	PNG (Portable Network Graphics)
<code>tiff</code>	TIFF (Tagged Image File Format)
<code>mp4-isobmff</code>	MP4 / ISO Base Media File Format
<code>mkv</code>	Matroska (MKV)
<code>c2pa-manifest</code>	C2PA Manifest (Content Credentials)
<code>xmp</code>	XMP (Extensible Metadata Platform)
<code>exif</code>	EXIF (Exchangeable Image File Format)
<code>iptc-photo</code>	IPTC Photo Metadata Standard 2025.1
<code>premis</code>	PREMIS (Preservation Metadata)
<code>bagit</code>	BagIt (RFC 8493)
<code>mets</code>	METS (Metadata Encoding and Transmission Standard)

4.1 Broadcast WAVE Format (BWF)

ID: bwf

Type: audio

Structure: binary-chunk

Property	Type	Notes
Readability	open	EBU Tech 3285 and FADGI specifications are publicly available; no licensing restrictions on implementation or use.
Verifiability	integrity-only	No native cryptographic signing. BEXT chunk carries CodingHistory, originator, and origination date/time as advisory provenance fields. BWF may store XMP data. C2PA stored in a custom C2PA RIFF chunk provides cryptographic signing when present.
Persistence		Widely adopted archival audio format with long-term institutional support from EBU, FADGI, and national archives.

FDD References:

- [FDD000356](#) (full: Broadcast WAVE Audio File Format, Version 1)
- [FDD000357](#) (full: Broadcast WAVE Audio File Format, Version 2)

URL: guidelines: <https://www.digitizationguidelines.gov/guidelines/digitize-embedding.html>

C2PA Support Modes:

- **embedded-riff-chunk:** C2PA manifest store may be embedded in a custom C2PA RIFF chunk within the BWF file; hard binding to audio content via C2PA cryptographic signing chain.
- **sidecar:** C2PA manifest store delivered as a standalone .c2pa sidecar file alongside the BWF asset; uses C2PA soft binding. Relationship must be maintained by the packaging or delivery layer.

Mechanisms:

Mechanism	Type	File-level	Content-level	Metadata Integrity
bext-chunk				

Mechanism	Type	File-level	Content-level	Metadata Integrity
XMP Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none
JUMBF C2PA Manifest Store (Embedded)	embedded-c2pa	embedded-writable	append-only	signed
PREMIS Sidecar / Embedded Event Record	sidecar-metadata	sidecar-writable	append	none
C2PA Sidecar Manifest	sidecar-c2pa	sidecar-writable	append-only	signed

EBU/FADGI standard audio format based on RIFF/WAVE. BEXT chunk carries CodingHistory, originator, origination date/time, and UMID. C2PA stored in a custom C2PA RIFF chunk. Widely used as the archival master or primary format for audio in broadcast, library, and archival workflows. Supported by BWF MetaEdit for metadata embedding and validation.

4.1.1 TCR4CAP Comments

Awareness: BWF is a well-established archival audio format. BEXT chunk provenance fields are widely understood in broadcast and archival workflows. C2PA embedding path is defined but not yet widely implemented in BWF-specific tooling.

Tamper Evidence: BEXT chunk fields are informative and not integrity-protected. They can be silently overwritten, edited, added, or removed. C2PA RIFF chunk provides cryptographic tamper-evidence when present.

Binding: BEXT chunk is structurally embedded. C2PA RIFF chunk provides cryptographic hard binding to audio content when present. An md5 chunk may store the md5 hash of the audio content.

AI Attribution: No purpose-built AI attribution fields in the BWF/BEXT specification. AI documentation requires C2PA assertions or repurposing of CodingHistory.

Substantiation: BEXT CodingHistory provides a readable transformation record. C2PA manifest store could preserve a full transformation history when present.

Interoperability: Open specification (EBU Tech 3285, FADGI). Widely implemented across broadcast, archival, and audio tools.

4.2 FLAC (Free Lossless Audio Codec)

ID: flac

Type: audio

Structure: binary-block

Property	Type	Notes
Readability	open	Full open specification originally published by Xiph.org and later standardized as IETF RFC 9639; no licensing restrictions on implementation or use. Natively stores an MD5 signature of unencoded audio in the STREAMINFO block and CRC-8/CRC-16 checksums per frame (RFC 9639). Provides format-level integrity verification of audio data only; checksums do not cover metadata blocks. No signature or provenance chain in the base format. Open, lossless, widely implemented format. Long-term institutional support via IETF standardization (RFC 9639).
Verifiability	integrity-only	
Persistence		

FDD References:

- [FDD000657](#) (full: FLAC Family)
- [FDD000198](#) (full: FLAC (Free Lossless Audio Codec), Version 1.1.2)

C2PA Support Modes:

- **embedded-via-id3:** C2PA manifest store embedded using the ID3 General Encapsulated Object frame (GEOB); hard binding to audio content via C2PA cryptographic signing chain.
- **sidecar:** C2PA manifest store delivered as a standalone .c2pa sidecar file alongside the FLAC asset; uses C2PA soft binding. Relationship must be maintained by the packaging or delivery layer.

Mechanisms:

Mechanism	Type	File-level	Content-level	Metadata Integrity
Vorbis Comment	embedded-metadata	embedded-writable	overwrite	none
ID3 General Encapsulated Object (GEOB) — C2PA	embedded-c2pa	embedded-writable	append-only	signed
XMP Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none

Mechanism	Type	File-level	Content-level	Metadata Integrity
PREMIS Sidecar / Embedded Event Record	sidecar-metadata	sidecar-writable	append	none
C2PA Sidecar Manifest	sidecar-c2pa	sidecar-writable	append-only	signed

A container for the FLAC lossless audio codec. C2PA embedding is supported via ID3 using the General Encapsulated Object frame (GEOB). Natively stores an MD5 signature of unencoded audio in the STREAMINFO block and CRC-8/CRC-16 checksums per frame (RFC 9639), providing format-level integrity verification of audio data. These checksums do not cover metadata blocks.

4.2.1 TCR4CAP Comments

Awareness: C2PA-specific awareness in FLAC workflows is not yet common. The GEOB embedding path is defined but not widely recognized in tooling or policy.

Tamper Evidence: Native MD5 (STREAMINFO) and per-frame CRCs (RFC 9639) provide structured format-level integrity of audio data. These do not cover metadata blocks; tamper-evidence of CAP data depends on the mechanism used.

Binding: The format defines discrete metadata blocks and a specified C2PA embedding location (ID3 GEOB). Binding strength depends on the mechanism used.

AI Attribution: No purpose-built AI attribution fields in the FLAC or Vorbis Comment specification. Any AI documentation requires repurposing general fields or use of C2PA via GEOB.

Substantiation: FLAC metadata is open and readable. No formal mechanism for publishing transformation history or CAP policies at point of access at the format level.

Interoperability: Open specification (Xiph.org, RFC 9639). Widely implemented across tools and platforms. ID3 broadly supported.

4.3 JPEG

ID: jpeg

Type: image

Structure: binary-segment

Property	Type	Notes
Readability	published-standard	ISO/IEC 10918 and ITU-T T.81; open standard with no licensing restrictions on implementation or use.

Property	Type	Notes
Verifiability	signed	C2PA stored in APP11 (0xFFEB) chunk provides cryptographic signing and hard binding to image content. EXIF, IPTC, and XMP metadata in APP1/APP13 are informative and not integrity-protected. JPEG Trust (ISO/IEC 21617) provides an additional trust framework.
Persistence		Lossy compression limits use as an archival master or primary format; widely used for distribution and delivery. C2PA manifest survives most JPEG-preserving operations.

FDD References:

- [FDD000017](#) (full: JPEG Image Encoding Family)
- [FDD000018](#) (full: JFIF, JPEG File Interchange Format, Version 1.02)
- [FDD000147](#) (full: JPEG Encoded File with Exif Metadata)
- [FDD000538](#) (preliminary: JPEG XL File Format)
- [FDD000653](#) (partial: JPEG Universal Metadata Box Format (JUMBF))
- [FDD000655](#) (preliminary: JPEG XT (JPEG eXTension))

URL: homepage: <https://jpeg.org/>

C2PA Support Modes:

- **embedded-app11:** C2PA manifest store embedded in APP11 (0xFFEB) chunk. Multiple sequential APP11 segments used for large manifests. Hard binding to image content via content hash. The C2PA Technical Specifications cite JPEG XT, ISO/IEC 18477-3 (which defines the JPEG file format) as the basis of the definition of the APP11 structure that stores the C2PA manifest store. If a manifest is embedded in the center of a JPEG-1 file in an APP11 segment, the claim creator may exclude the APP11 segment(s) from the hash calculation, as defined in C2PA specification 2.4.
- **sidecar:** C2PA manifest store delivered as a standalone .c2pa sidecar file; uses C2PA soft binding.

Mechanisms:

Mechanism	Type	File-level	Content-level	Metadata Integrity
EXIF Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none
XMP Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none

Mechanism	Type	File-level	Content-level	Metadata Integrity
IPTC Photo Metadata	embedded-metadata	embedded-writable	overwrite	none
JUMBF C2PA Manifest Store (Embedded)	embedded-c2pa	embedded-writable	append-only	signed
PREMIS Sidecar / Embedded Event Record	sidecar-metadata	sidecar-writable	append	none
C2PA Sidecar Manifest	sidecar-c2pa	sidecar-writable	append-only	signed

JPEG (ISO/IEC 10918) is a dominant photographic image format with a segment-based structure (SOI, APPn, DQT, DHT, SOS, EOI). Supports EXIF (APP1), IPTC (APP13), and XMP (APP1) metadata. JPEG Trust (ISO/IEC 21617) provides an additional trust framework. Widely adopted by camera manufacturers (Leica M11-P, Nikon Z6 III) for C2PA signing at capture.

4.3.1 TCR4CAP Comments

Awareness: JPEG is a widely deployed image format and has relatively extensive support with metadata processing tools. C2PA embedding in APP11 is a defined, published standard with broad industry adoption.

Tamper Evidence: C2PA APP11 chunk provides cryptographic tamper-evidence via signed manifests and hard binding to image content.

Binding: C2PA hard binding via content hash provides cryptographic linkage between manifest and image content.

AI Attribution: C2PA AI attribution assertions (c2pa.ai.generatedWith, c2pa.training-mining) and IPTC 2025.1 AI fields are both supported in JPEG.

Substantiation: C2PA manifest store preserves full transformation history. External trust anchors supported. Public verification via contentcredentials.org.

Interoperability: Universally implemented across cameras, phones, image editing tools, and repository systems.

4.4 PNG (Portable Network Graphics)

ID: png

Type: image

Structure: binary-chunk

Property	Type	Notes
Readability	open	ISO/IEC 15948 and W3C PNG specification; open standard with no licensing restrictions.
Verifiability	signed	C2PA stored in custom caBX chunk provides cryptographic signing and hard binding to image content. XMP and EXIF metadata in iTXt/eXif chunks are informative and not integrity-protected. PNG CRC checksums provide per-chunk structural integrity only.
Persistence		Lossless compression; widely used as an archival and distribution format for images requiring transparency or lossless fidelity. Recommended by FADGI for Documents (Unbound): Modern Textual Records: https://www.digitizationguidelines.gov/guidelines/

FDD References:

- [FDD000153](#) (full: Portable Network Graphics)

C2PA Support Modes:

- **embedded-cabx-chunk:** C2PA manifest store embedded in custom caBX chunk. Hard binding to image content via content hash.
- **sidecar:** C2PA manifest store delivered as a standalone .c2pa sidecar file; uses C2PA soft binding.

Mechanisms:

Mechanism	Type	File-level	Content-level	Metadata Integrity
EXIF Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none
XMP Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none
IPTC Photo Metadata	embedded-metadata	embedded-writable	overwrite	none
JUMBF C2PA Manifest Store (Embedded)	embedded-c2pa	embedded-writable	append-only	signed

Mechanism	Type	File-level	Content-level	Metadata Integrity
PREMIS Sidecar / Embedded Event Record	sidecar-metadata	sidecar-writable	append	none
C2PA Sidecar Manifest	sidecar-c2pa	sidecar-writable	append-only	signed

PNG (ISO/IEC 15948) is a lossless image format with a chunk-based structure. C2PA stored in custom caBX chunk. Also supports XMP (iTXt), EXIF (eXIf), and text metadata (tEXt/zTXt) chunks. Strong archival format; widely used for images requiring transparency or lossless fidelity.

4.4.1 TCR4CAP Comments

Awareness: PNG is a widely deployed image format. C2PA embedding in caBX chunk is a defined, published standard.

Tamper Evidence: C2PA caBX chunk provides cryptographic tamper-evidence via signed manifests and hard binding to image content.

Binding: C2PA hard binding via content hash provides cryptographic linkage between manifest and image content.

AI Attribution: C2PA AI attribution assertions and IPTC 2025.1 AI fields are both supported in PNG.

Substantiation: C2PA manifest store preserves full transformation history. External trust anchors supported.

Interoperability: Widely implemented across image editing tools, browsers, and repository systems.

4.5 TIFF (Tagged Image File Format)

ID: tiff

Type: image

Structure: binary

Property	Type	Notes
Readability	open	Open standard with no licensing restrictions on implementation or use.

Property	Type	Notes
Verifiability	signed	C2PA manifests can be natively embedded in TIFF via a JUMBF box stored in the IFD structure, as defined in C2PA specification 2.4, providing cryptographic signing and hard binding to the content. A sidecar .c2pa file is also supported as an alternative delivery path. Native TIFF metadata (EXIF, IPTC, XMP) is informative and not integrity-protected outside of C2PA.
Persistence		Widely used archival image format; long-term institutional support from libraries, archives, and digitization programs. Recommended by FADGI for Documents (Unbound): Modern Textual Records: https://www.digitizationguidelines.gov/guidelines

FDD References:

- [FDD000022](#) (full: TIFF, Revision 6.0)
- [FDD000145](#) (full: TIFF Uncompressed File with Exif Metadata)

C2PA Support Modes:

- **embedded-jumbf-ifd:** C2PA manifest store embedded via a JUMBF box stored in the TIFF IFD structure, as defined in C2PA specification 2.4. Hard binding to image content via C2PA cryptographic signing chain. Use of a general box binding is strongly recommended per C2PA guidance.
- **sidecar:** C2PA manifest store delivered as a standalone .c2pa sidecar file alongside the TIFF asset; uses C2PA soft binding. Relationship must be maintained by the packaging or delivery layer.

Mechanisms:

Mechanism	Type	File-level	Content-level	Metadata Integrity
EXIF Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none
XMP Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none

Mechanism	Type	File-level	Content-level	Metadata Integrity
IPTC Photo Metadata	embedded-metadata	embedded-writable	overwrite	none
JUMBF C2PA Manifest Store (Embedded)	embedded-c2pa	embedded-writable	append-only	signed
C2PA Sidecar Manifest	sidecar-c2pa	sidecar-writable	append-only	signed
PREMIS Sidecar / Embedded Event Record	sidecar-metadata	sidecar-writable	append	none

TIFF is a widely used archival image format with a tag-based IFD structure. Supports EXIF (SubIFD), IPTC (tag 33723), and XMP (tag 700) metadata. C2PA manifests can be natively embedded via a JUMBF box in the IFD structure per C2PA specification 2.4; a sidecar .c2pa file is also supported. Common digitization master or primary format in library and archival workflows.

4.5.1 TCR4CAP Comments

Awareness: TIFF is a well-established archival format recommended by FADGI for digitization of cultural heritage materials. EXIF, IPTC, and XMP provenance fields are widely understood. C2PA embedding via JUMBF box in the IFD structure is defined in C2PA specification 2.4 but not yet widely implemented in TIFF-specific tooling.

Tamper Evidence: EXIF, IPTC, and XMP metadata are informative and not integrity-protected. C2PA JUMBF embedding provides cryptographic tamper-evidence via signed manifests and hard binding to image content when present.

Binding: C2PA hard binding via content hash provides cryptographic linkage between manifest and image content when embedded. Sidecar binding is soft and depends on file management conventions.

AI Attribution: C2PA AI attribution assertions (c2pa.ai.generatedWith, c2pa.training-mining) and IPTC 2025.1 AI fields (via XMP) are both supported in TIFF.

Substantiation: EXIF, IPTC, and XMP provide a readable provenance record. C2PA manifest store preserves full transformation history when present. PREMIS sidecar provides institutional transformation history.

Interoperability: Universally implemented across image editing tools, scanners, and repository systems.

4.6 MP4 / ISO Base Media File Format

ID: mp4-isobmff

Type: moving image

Structure: binary-atom

Property	Type	Notes
Readability	published-standard	ISO/IEC 14496-12 (ISOBMFF); open standard. MP4 file format (ISO/IEC 14496-14) and related derivatives (MOV, MJ2, 3GP) are based on the same box/atom structure.
Verifiability	signed	C2PA stored in uuid atom provides cryptographic signing and hard binding to video content. XMP and other metadata atoms are informative and not integrity-protected.
Persistence		Widely used video container format; long-term institutional support from ISO, MPEG, and industry. Widely used for archival and distribution of video content.

FDD References:

- [FDD000079](#) (full: ISO Base Media File Format)
- [FDD000155](#) (full: MPEG-4 File Format, Version 2)
- [FDD000525](#) (preliminary: High Efficiency Image File (HEIF) Format, MPEG-H Part 12)

C2PA Support Modes:

- **embedded-uuid-atom:** C2PA manifest store embedded in uuid atom (UUID: d8fec3d6-1b0e-483c-9297-5844b8f49b68). Hard binding to video content via content hash. Covers MP4, MOV, MJ2, 3GP, and other ISO-BMFF derivatives.
- **sidecar:** C2PA manifest store delivered as a standalone .c2pa sidecar file; uses C2PA soft binding.

Mechanisms:

Mechanism	Type	File-level	Content-level	Metadata Integrity
XMP Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none

Mechanism	Type	File-level	Content-level	Metadata Integrity
JUMBF C2PA Manifest Store (Embedded)	embedded-c2pa	embedded-writable	append-only	signed
PREMIS Sidecar / Embedded Event Record	sidecar-metadata	sidecar-writable	append	none
C2PA Sidecar Manifest	sidecar-c2pa	sidecar-writable	append-only	signed

MP4 / ISO Base Media File Format (ISO/IEC 14496-12) is the primary video container format for C2PA implementations. C2PA stored in uuid atom. Covers MP4, MOV, MJ2, 3GP, and other ISO-BMFF derivatives. Supports XMP, IPTC, and other metadata in dedicated atoms.

4.6.1 TCR4CAP Comments

Awareness: MP4/ISOBMFF is the widely used video container format. C2PA embedding in uuid atom is a defined, published standard with growing industry adoption.

Tamper Evidence: C2PA uuid atom provides cryptographic tamper-evidence via signed manifests and hard binding to video content.

Binding: C2PA hard binding via content hash provides cryptographic linkage between manifest and video content.

AI Attribution: C2PA AI attribution assertions (c2pa.ai.generatedWith, c2pa.training-mining) are fully supported in MP4/ISOBMFF.

Substantiation: C2PA manifest store preserves full transformation history. External trust anchors supported.

Interoperability: Universally implemented across video tools, cameras, and repository systems.

4.7 Matroska (MKV)

ID: mkv

Type: moving image

Structure: binary-ebml

Property	Type	Notes
Readability	open	Matroska specification published by the Matroska.org community; IETF RFC 9559 (EBML) provides the underlying container standard.

Property	Type	Notes
Verifiability	none	No native cryptographic signing (though an early EBML draft included an Element signing feature which was dropped before standardization). Tag and attachment elements can store UTF-8 and binary metadata. Any Matroska Element may store a hash of the following content of the Element. This feature can verify integrity of the Element against the stored hash (if it exists), but there is no signature mechanism. No C2PA embedding path is defined in the C2PA Technical Specification as of 2026-06.
Persistence		Open container format. Widely used in archival video workflows and video distribution. Long-term support from the Matroska community and IETF Cellar working group.

FDD References:

- [FDD000342](#) (full: Matroska Multimedia Container)
- [FDD000516](#) (preliminary: Extensible Binary Meta Language)

C2PA Support Modes:

- **sidecar:** C2PA manifest store delivered as a standalone .c2pa sidecar file alongside the MKV asset; uses C2PA soft binding. No native MKV embedding path is defined in the C2PA Technical Specification as of 2026-05.

Mechanisms:

Mechanism	Type	File-level	Content-level	Metadata Integrity
XMP Embedded Metadata	embedded-metadata	embedded-writable	overwrite	none
C2PA Sidecar Manifest	sidecar-c2pa	sidecar-writable	append-only	signed
PREMIS Sidecar / Embedded Event Record	sidecar-metadata	sidecar-writable	append	none

Matroska (MKV) is an open container format based on EBML (RFC 9559). Supports tags and attachments for metadata. No native C2PA embedding path defined as of 2026-05. Used in some archival video workflows and widely in consumer distribution.

4.7.1 TCR4CAP Comments

Awareness: Matroska is a well-known container format. No native C2PA embedding path is defined; C2PA awareness in MKV workflows is limited.

Tamper Evidence: No native cryptographic signing. Tag and attachment metadata are advisory and not integrity-protected. C2PA sidecar provides tamper-evidence but can be separated from the asset.

Binding: No native C2PA embedding path. Sidecar binding is soft and depends on file management conventions.

AI Attribution: No purpose-built AI attribution fields in the Matroska specification. AI documentation requires C2PA sidecar or repurposing of tag elements.

Substantiation: Tag elements provide a readable metadata record. No formal mechanism for publishing transformation history at the format level.

Interoperability: Open specification. Widely implemented across video tools and platforms.

4.8 C2PA Manifest (Content Credentials)

ID: c2pa-manifest

Type: metadata

Structure: cbor-jumbf

Property	Type	Notes
Readability	published-standard	C2PA Technical Specification published by the Coalition for Content Provenance and Authenticity; open specification with reference implementation (c2pa-rs) under open-source license.

Property	Type	Notes
Verifiability	signed	Each manifest is cryptographically signed using CMS/COSE. Hard binding links the manifest to the asset content via content hash. The manifest store is append-only; prior manifests are preserved as ingredient references. Any modification to a manifest or to the asset content invalidates the signature or binding.
Persistence		Append-only manifest store preserves the full provenance chain across the asset lifecycle. Signed manifests are immutable. External trust anchors (TSA, credential issuers) provide long-term verifiability.

C2PA Support Modes:

- **embedded:** Embedded in host files via format-specific JUMBF containers (APP11 in JPEG, caBX in PNG, uuid atom in MP4/ISOBMFF, C2PA RIFF chunk in WAV/BWF).
- **sidecar:** Stored as a standalone .c2pa sidecar file alongside the asset.

Core C2PA metadata format. CBOR-encoded assertions in a JUMBF container. Cryptographically signed using CMS/COSE. Embedded in media files via format-specific containers or stored as a standalone .c2pa sidecar file. Supports hard binding (content hash) and soft binding (perceptual identifier). Defines purpose-built assertions for AI generative and training provenance (c2pa.ai.generatedWith, c2pa.training-mining). The manifest store is append-only; prior manifests are preserved as ingredient references.

4.8.1 TCR4CAP Comments

Awareness: C2PA Technical Specification is published by the Coalition for Content Provenance and Authenticity. Adopted by Adobe, Microsoft, Google, camera manufacturers, and news organizations.

Tamper Evidence: Each manifest is cryptographically signed; any modification invalidates the signature. Hard binding means any modification to the asset content invalidates the active manifest.

Binding: Hard binding provides cryptographic linkage between manifest and asset content via content hash. Soft binding provides a defined alternative for non-embeddable assets.

AI Attribution: C2PA defines purpose-built assertions for AI generative and training provenance: c2pa.ai.generatedWith, c2pa.training-mining, and related fields for model identity and generation

parameters.

Substantiation: Manifest store preserves full transformation history. External trust anchors (credential issuers, TSA) are supported. Public verification is possible via the C2PA trust list.

Interoperability: Open published specification. Reference implementation (c2patool, c2pa-rs) available under open-source license. Adopted across image, video, and audio tools and camera manufacturers.

4.9 XMP (Extensible Metadata Platform)

ID: xmp

Type: metadata

Structure: xml-rdf

Property	Type	Notes
Readability	published-standard	ISO 16684; open standard with no licensing restrictions on implementation or use.
Verifiability	none	XMP packets have no integrity protection; properties can be silently overwritten or the entire packet stripped without detection. xmpMM:History entries are advisory and unverified.
Persistence		XMP packets can be stripped by processing tools; persistence depends on tool behavior and workflow policy. Sidecar .xmp files can be managed independently.

C2PA Support Modes:

- **none:** XMP has no native C2PA support. IPTC 2025.1 AI fields carried in XMP namespaces provide structured AI attribution but without cryptographic protection.

XMP (ISO 16684) is an RDF/XML metadata format embedded in host files or stored as standalone .xmp sidecar files. Supports multiple namespaces including Dublin Core (dc:), XMP Basic (xmp:), XMP Media Management (xmpMM:), and IPTC (Iptc4xmpCore/Ext:). The xmpMM:History array provides a conventional provenance record. XMP packets have no integrity protection.

4.9.1 TCR4CAP Comments

Awareness: XMP is defined in ISO 16684. Implemented across all major creative, archival, and repository tools. xmpMM:History and xmpMM:DerivedFrom are recognized provenance fields.

Tamper Evidence: XMP packets have no integrity protection. Properties can be silently overwritten or the entire packet stripped.

Binding: XMP is structurally embedded in host files but there is no cryptographic binding between XMP content and the primary media data.

AI Attribution: IPTC 2025.1 AI fields (AiPrompt, AiSystemUsed, AiSystemVersion) are carried in XMP namespaces. No purpose-built AI attribution fields are defined in the core XMP specification.

Substantiation: xmpMM:History and xmpMM:DerivedFrom provide a readable provenance record. XMP sidecar files can be published independently.

Interoperability: ISO 16684. Implemented across all major creative, archival, and repository tools.

4.10 EXIF (Exchangeable Image File Format)

ID: exif

Type: metadata

Structure: binary

Property	Type	Notes
Readability	open	CIPA DC-008; open standard with no licensing restrictions on implementation or use.
Verifiability	none	EXIF data has no integrity protection; tags can be silently overwritten or stripped without detection. No provenance chain mechanism in the EXIF specification.
Persistence		EXIF data can be stripped by processing tools; persistence depends on tool behavior and workflow policy.

C2PA Support Modes:

- **none:** EXIF has no native C2PA support. EXIF capture device and datetime fields are referenced by C2PA assertions but are not themselves integrity-protected.

EXIF (CIPA DC-008) is a binary metadata standard using an IFD tag structure. Records capture

device identity, camera settings, GPS coordinates, and datetime. Embedded in JPEG APP1, TIFF SubIFD, PNG eXif chunk, and HEIF. EXIF data has no integrity protection and can be silently overwritten or stripped.

4.10.1 TCR4CAP Comments

Awareness: EXIF is defined by CIPA DC-008. Universally implemented across cameras, phones, and image processing tools. Capture device identity and datetime fields are widely used as provenance indicators.

Tamper Evidence: EXIF data has no integrity protection. Tags can be silently overwritten or stripped.

Binding: EXIF is structurally embedded in host files but there is no cryptographic binding between EXIF tag values and the image content.

AI Attribution: No AI attribution fields are defined in the EXIF specification. Software (0x0131) can record processing tool identity but has no structured AI attribution vocabulary.

Substantiation: Capture datetime, GPS, and device fields provide a factual provenance record. No mechanism links EXIF fields to external policies or transformation histories.

Interoperability: CIPA DC-008. Universally implemented across cameras, phones, and image processing tools.

4.11 IPTC Photo Metadata Standard 2025.1

ID: iptc-photo

Type: metadata

Structure: xml-xmp

Property	Type	Notes
Readability	open	IPTC Photo Metadata Standard published by IPTC; open standard with no licensing restrictions on implementation or use. IPTC fields carried in XMP have no integrity protection; fields can be silently overwritten or stripped without detection. IPTC fields carried in XMP can be stripped by processing tools; persistence depends on tool behavior and workflow policy.
Verifiability	none	
Persistence		

C2PA Support Modes:

- **none:** IPTC Photo Metadata has no native C2PA support. IPTC 2025.1 AI fields provide structured AI attribution in XMP namespaces but without cryptographic protection.

The IPTC Photo Metadata Standard defines a vocabulary of fields carried in XMP namespaces (Iptc4xmpCore, Iptc4xmpExt). Version 2025.1 adds dedicated AI fields: AiPrompt (11.2), AiSystemUsed (11.4), AiSystemVersion (11.5), and DataMiningPermission (11.11). Widely used in photojournalism, stock photography, and digital asset management.

4.11.1 TCR4CAP Comments

Awareness: IPTC Photo Metadata Standard is published by IPTC. Version 2025.1 AI fields are defined in the specification. Widely implemented in photo editing, DAM, and publishing tools.

Tamper Evidence: IPTC fields carried in XMP have no integrity protection. Fields can be silently overwritten or stripped.

Binding: IPTC fields are structurally embedded via XMP in host files but there is no cryptographic binding between field values and the image content.

AI Attribution: Version 2025.1 defines AiPrompt, AiSystemUsed, AiSystemVersion, and DataMiningPermission as purpose-built, standardized AI attribution fields within the IPTC vocabulary.

Substantiation: IPTC fields provide a structured, readable provenance record. The standard is publicly documented.

Interoperability: Implemented in photo editing, DAM, and publishing tools including Adobe Photoshop, Lightroom, and Capture One.

4.12 PREMIS (Preservation Metadata)

ID: premis

Type: metadata

Structure: xml

Property	Type	Notes
Readability	open	PREMIS Data Dictionary published by the Library of Congress; open standard with no licensing restrictions on implementation or use.

Property	Type	Notes
Verifiability	integrity-only	PREMIS XML has no native integrity protection; tamper-evidence depends on the repository or packaging layer (e.g. BagIt fixity, audit logs). Fixity values in objectCharacteristics/fixity link the PREMIS record to a specific file state.
Persistence		PREMIS is designed for long-term institutional preservation. Widely implemented in repository systems with long-term support from the Library of Congress.

C2PA Support Modes:

- **none:** PREMIS has no native C2PA support. PREMIS events map conceptually to C2PA actions (c2pa.created, c2pa.converted, c2pa.edited); PREMIS agent maps to C2PA softwareAgent or person assertions.

PREMIS (Library of Congress) is an XML schema for recording preservation metadata. Models Objects, Events, Agents, and Rights. PREMIS events map to C2PA actions; PREMIS agent maps to C2PA softwareAgent or person assertions. Generated by preservation systems including Archivematica, DSpace, and Fedora. PREMIS XML has no native integrity protection; tamper-evidence depends on the repository or packaging layer.

4.12.1 TCR4CAP Comments

Awareness: PREMIS Data Dictionary is published by the Library of Congress. Implemented in Archivematica, DSpace, Fedora, and other repository systems.

Tamper Evidence: PREMIS XML has no integrity protection. Tamper-evidence depends on the repository or packaging layer.

Binding: Fixity values in objectCharacteristics/fixity link the PREMIS record to a specific file state. Binding is not cryptographically enforced within the PREMIS document itself.

AI Attribution: No purpose-built AI attribution fields in the PREMIS Data Dictionary. Agent/agentType and eventType vocabularies could accommodate AI agents and events by convention.

Substantiation: PREMIS event records provide a structured, policy-driven transformation history. The PREMIS Data Dictionary is an open standard. Event records are auditable by repository administrators.

Interoperability: Published by the Library of Congress as an open standard. Implemented across major digital preservation systems and repository platforms.

4.13 BagIt (RFC 8493)

ID: bagit

Type: metadata

Structure: text-manifest

Property	Type	Notes
Readability	open	IETF RFC 8493; open standard with no licensing restrictions on implementation or use.
Verifiability	integrity-only	Manifest files (<i>manifest-.txt</i>) record checksums for all payload files; tag manifests (<i>tagmanifest-.txt</i>) record checksums for all tag files. Any modification to a payload or tag file produces a checksum mismatch detectable by bag validation. No signing mechanism; the manifest files themselves could be replaced along with modified payload files without detection at the BagIt level.
Persistence		BagIt is widely adopted in digital preservation, library, and archival transfer workflows. Long-term support from the Library of Congress and IETF standardization.

C2PA Support Modes:

- **none:** BagIt has no native C2PA support. C2PA sidecar files can be included as payload files within a bag; bag-info.txt custom fields could carry AI processing notes by convention.

BagIt (IETF RFC 8493) is a hierarchical file packaging format for reliable digital transfer and storage. A bag contains a payload directory, manifest files recording checksums for all payload files, tag files carrying descriptive metadata, and tag manifest files recording checksums for tag files. Operates at the package level; does not interact with file-level metadata mechanisms. Supported algorithms include MD5, SHA-1, SHA-256, and SHA-512.

4.13.1 TCR4CAP Comments

Awareness: BagIt is defined in IETF RFC 8493. Widely adopted in digital preservation, library, and archival transfer workflows.

Tamper Evidence: Manifest checksums detect modification of any payload or tag file. No signing mechanism at the BagIt level.

Binding: Manifest entries bind each checksum to a specific file path within the bag. No cryptographic binding between the bag as a whole and an external identity or signing authority.

AI Attribution: No AI attribution fields in the BagIt specification. bag-info.txt custom fields could carry AI processing notes by convention.

Substantiation: Manifest files provide a verifiable, reproducible integrity record for all payload and tag files. bag-info.txt carries descriptive provenance fields.

Interoperability: IETF RFC 8493. Implemented across digital preservation, library, and archival transfer tools and systems.

4.14 METS (Metadata Encoding and Transmission Standard)

ID: mets

Type: metadata

Structure: xml

Property	Type	Notes
Readability	open	METS schema published by the Library of Congress; open standard with no licensing restrictions on implementation or use.
Verifiability	integrity-only	METS XML has no native integrity protection; tamper-evidence depends on the repository or packaging layer. METS can carry fixity values for described files in the fileSec/fileGrp/file/FLocat elements.
Persistence		METS is designed for long-term institutional preservation. Widely implemented in repository systems with long-term support from the Library of Congress.

C2PA Support Modes:

- **none:** METS has no native C2PA support. METS can reference C2PA sidecar files as file objects within the fileSec; PREMIS events carried in the amdSec can document C2PA-related processing actions.

METS (Library of Congress) is an XML standard for encoding descriptive, administrative, and structural metadata for digital objects. Sections include dmdSec (descriptive), amdSec (administrative, including PREMIS), fileSec (file inventory), and structMap (structural). Used by Archivematica, DSpace, and many repository systems. METS XML has no native integrity protection; tamper-evidence depends on the repository or packaging layer.

4.14.1 TCR4CAP Comments

Awareness: METS schema is published by the Library of Congress. Implemented in Archivematica, DSpace, and other repository systems.

Tamper Evidence: METS XML has no integrity protection. Tamper-evidence depends on the repository or packaging layer.

Binding: METS fileSec references described files by path and can carry fixity values. Binding is not cryptographically enforced within the METS document itself.

AI Attribution: No purpose-built AI attribution fields in the METS schema. PREMIS events in the amdSec could document AI processing actions by convention.

Substantiation: METS provides a structured, comprehensive metadata record for digital objects. Widely supported by repository systems.

Interoperability: Published by the Library of Congress as an open standard. Implemented across major digital preservation systems and repository platforms.

5 Mechanisms Registry

ID	Name
flac-features	FLAC Specific Integrity Features
bwf-features	BWF Specific Features (BEXT Chunk)
jpeg-features	JPEG Specific Features (APP Segments)
png-features	PNG Specific Features (Chunks)
tiff-features	TIFF Specific Features (IFD Tags)
mp4-features	MP4 / ISOBMFF Specific Features (Atoms)
mkv-features	Matroska Specific Features (EBML Tags, CRC-32, and Attachments)
vorbis-comment	Vorbis Comment
id3-geob-c2pa	ID3 General Encapsulated Object (GEOB) — C2PA
bwf-bext	BWF BEXT Chunk

ID	Name
bwf-info	BWF LIST-INFO Chunk
bwf-xml	BWF XML Chunks (aXML / iXML)
bwf-md5	BWF Audio-Data MD5 Checksum
bwf-c2pa	BWF C2PA Chunk
xmp-embedded	XMP Embedded Metadata
exif-embedded	EXIF Embedded Metadata
iptc-photo	IPTC Photo Metadata
jumbf-c2pa	JUMBF C2PA Manifest Store (Embedded)
sidecar-c2pa	C2PA Sidecar Manifest
sidecar-premis	PREMIS Sidecar / Embedded Event Record
bagit-features	BagIt Package Fixity

5.1 FLAC Specific Integrity Features

ID: flac-features

Type: format-specific

5.1.1 Editability

Level	Type	Notes
File-level	embedded-writable	The STREAMINFO block is embedded at a fixed offset in the file and can be overwritten in place without re-encoding the audio stream. Its field values are derived from the audio stream at encode time; overwriting them without re-encoding produces a file whose metadata no longer accurately describes the audio frames.
Content-level	fixed	The MD5 signature of unencoded audio and per-frame CRC values are computed at encode time and are fixed for the lifetime of the file unless the audio stream is re-encoded.

5.1.2 Verifiability

Dimension	Type	Notes
Media integrity	checksum	MD5 signature of unencoded PCM audio stored in STREAMINFO; CRC-8 per frame header and CRC-16 per frame footer. Recomputable by any conformant FLAC decoder (e.g. flac -test, ffmpeg).
Metadata integrity	none	No integrity mechanism covers FLAC metadata blocks (Vorbis Comment, ID3, GEOB, PICTURE). Metadata can be altered without affecting the audio-data checksums.
Chain of custody	none	No provenance chain or signing mechanism in the base FLAC format specification.

FDD References:

- [FDD000198](#) (full:)

5.1.3 Metadata Fields

Field	CAP Concept	Description
MD5 signature	media-integrity	Stored in STREAMINFO block; MD5 of unencoded raw PCM audio data.
CRC-8	media-integrity	Per audio frame header; covers frame header bytes up to but not including the CRC byte itself.
CRC-16	media-integrity	Per audio frame footer; covers encoded frame data.

FLAC encodes integrity data into the bitstream as defined in RFC 9639. The STREAMINFO metadata block stores an MD5 signature of the unencoded (raw PCM) audio data. Each audio frame header carries a CRC-8 and each frame footer carries a CRC-16 of the encoded frame. These values are computed at the time of encoding and cover audio data only and do not cover any metadata blocks.

5.1.4 TCR4CAP Comments

Awareness: STREAMINFO MD5 and per-frame CRCs are defined in RFC 9639 and are present in all conformant FLAC files. These fields are not mapped to CAP vocabularies in any current CAP framework or policy document.

Tamper Evidence: Any modification to the audio bitstream would produce a different MD5 or CRC value during verification, detectable by recomputing against the stored values. Metadata blocks are not covered.

Binding: The MD5 is computed over the unencoded audio content at the time of encoding and stored in the same file. No mechanism links the MD5 to an external identity or signing authority.

AI Attribution: No fields in the FLAC format specification address AI generation, training, or processing provenance.

Substantiation: No fields in the FLAC format specification link to external policies, transformation histories, or rights statements.

Interoperability: Defined in RFC 9639 (IETF). Implemented by all conformant FLAC encoders and decoders. Verifiable by `flac -test`, `ffmpeg`, and any FLAC-capable tool.

5.2 BWF Specific Features (BEXT Chunk)

ID: `bwf-features`

Type: format-specific

5.2.1 Editability

Level	Type	Notes
File-level	embedded-writable	The BEXT chunk can be written or updated without re-encoding the audio data. CodingHistory is conventionally appended on each processing step; other BEXT fields are overwritten on write.
Content-level	append	

5.2.2 Verifiability

Dimension	Type	Notes
Media integrity	none	No included audio-data integrity mechanism in the BWF/BEXT specification itself. Audio-data MD5 is a tool-level addition (see bwf-md5 mechanism).
Metadata integrity	none	BEXT fields have no integrity protection; any field can be overwritten without detection.
Chain of custody	none	No signing or provenance chain mechanism in the BWF/BEXT specification. CodingHistory is a conventional append-only log with no enforcement.

FDD References:

- [FDD000356](#) (full:)
- [FDD000357](#) (full:)

5.2.3 Metadata Fields

Field	CAP Concept	Description
Originator	actor-identity	Name of the originating organization; 32 bytes.
OriginatorReference	asset-identity	Unique identifier assigned by the originator; 32 bytes.
OriginationDate	creation-datetime	Date of origination in YYYY-MM-DD format.
OriginationTime	creation-datetime	Time of origination in HH-MM-SS format.
CodingHistory	transformation-history	Free-text log of processing steps; conventionally appended on each encode or transfer.

The Broadcast Extension (BEXT) chunk is defined in EBU Tech 3285 and carries originator identity, originator reference, origination date and time, time reference (sample offset), UMID, loudness values, and a CodingHistory field documenting the processing chain. BEXT is the primary provenance carrier in BWF files. Fields can be silently overwritten; changes are not detectable without an external fixity record or audio-data MD5 comparison.

5.2.4 TCR4CAP Comments

Awareness: BEXT fields are defined in EBU Tech 3285 and are present in all conformant BWF files. Originator, OriginatorReference, OriginationDate/Time, and CodingHistory are recognized provenance fields in broadcast and archival audio workflows. FADGI and EBU publish guidelines for consistent field use.

Tamper Evidence: BEXT fields have no integrity protection. Any field can be overwritten without detection. CodingHistory is appended by convention; no mechanism enforces append-only behavior or detects deletion of history entries.

Binding: BEXT is embedded in the RIFF container alongside the audio data. No cryptographic mechanism links BEXT field values to the audio bitstream.

AI Attribution: No fields in the BEXT specification address AI generation, training, or processing provenance. CodingHistory is a free-text field that could carry AI processing notes by convention.

Substantiation: CodingHistory documents processing steps in a structured free-text format. UMID provides a globally unique material identifier. No mechanism links BEXT fields to external policies or trust anchors.

Interoperability: Defined in EBU Tech 3285. Implemented across broadcast and archival audio tools. FADGI and EBU publish open guidelines for field use.

5.3 JPEG Specific Features (APP Segments)

ID: jpeg-features

Type: format-specific

5.3.1 Editability

Level	Type	Notes
File-level	embedded-writable	APP segments can be written or replaced without re-encoding the image data.
Content-level	overwrite	Individual APP segment contents are replaced on write; C2PA APP11 is append-only at the manifest store level.

5.3.2 Verifiability

Dimension	Type	Notes
Media integrity	none	No image-data integrity mechanism in the base JPEG/JFIF specification.
Metadata integrity	none	APP segments have no integrity protection in the base specification; segments can be added, removed, or replaced without detection.
Chain of custody	signed	C2PA manifest store in APP11 provides cryptographic signing and hard binding to image content when implemented.

FDD References:

- [FDD000017](#) (full:)
- [FDD000018](#) (full:)
- [FDD000147](#) (full:)

5.3.3 Metadata Fields

Field	CAP Concept	Description
APP1 (0xFFE1) - EXIF	technical-provenance	Carries EXIF IFD structure with capture device, settings, GPS, and datetime fields.
APP1 (0xFFE1) - XMP	provenance-chain	Carries XMP RDF/XML packet including xmpMM:History, xmpMM:DerivedFrom, and IPTC AI fields.
APP13 (0xFFED) - IPTC-IIM	actor-identity	Carries IPTC Information Interchange Model fields including creator and rights.
APP11 (0xFFEB) - C2PA	provenance-chain	Carries C2PA manifest store in JUMBF container; multiple sequential APP11 segments used for large manifests.

JPEG defines a segmented binary structure with APP marker segments for metadata. APP1 carries EXIF and XMP; APP13 carries IPTC-IIM; APP11 carries C2PA manifest stores in a JUMBF container. These are intrinsic features of the JPEG/JFIF specification.

5.3.4 TCR4CAP Comments

Awareness: APP segment structure is defined in ISO/IEC 10918. APP11 C2PA embedding is defined in the C2PA Technical Specification Appendix A. APP1 EXIF, XMP, and APP13 IPTC-IIM are widely implemented across image tools.

Tamper Evidence: APP segments can be added, removed, or replaced without detection in the base format. C2PA APP11 provides cryptographic tamper-evidence when present; any modification to the signed content invalidates the manifest signature.

Binding: C2PA APP11 provides hard binding: the manifest contains a hash of the image content, cryptographically linking the manifest to the specific image bitstream at signing time. EXIF and XMP are structurally embedded but not cryptographically bound.

AI Attribution: C2PA AI attribution assertions (c2pa.ai.generatedWith, training data references) are carried in APP11 when present. IPTC 2025.1 AI fields are carried in XMP via APP1.

Substantiation: C2PA manifest store in APP11 preserves transformation history with external trust anchor support. XMP xmpMM:History and IPTC fields provide additional descriptive provenance.

Interoperability: JPEG/JFIF is defined in ISO/IEC 10918 and widely implemented. APP11 C2PA is supported by c2patool and c2pa-rs reference implementations.

5.4 PNG Specific Features (Chunks)

ID: png-features

Type: format-specific

5.4.1 Editability

Level	Type	Notes
File-level	embedded-writable	PNG chunks can be added, replaced, or removed without re-encoding the image data.
Content-level	overwrite	Metadata chunks are replaced on write; caBX C2PA chunk is append-only at the manifest store level.

5.4.2 Verifiability

Dimension	Type	Notes
Media integrity	checksum	CRC-32 per chunk covers each chunk's type and data fields. Detects accidental corruption of individual chunks including image data chunks.
Metadata integrity	checksum	CRC-32 covers metadata chunks (tEXt, iTXt, eXIf, caBX) as well as image data chunks. Does not detect intentional replacement of a chunk with a recalculated CRC.
Chain of custody	signed	C2PA manifest store in caBX chunk provides cryptographic signing and hard binding to image content when implemented.

FDD References:

- [FDD000153](#) (full:)

5.4.3 Metadata Fields

Field	CAP Concept	Description
caBX chunk	provenance-chain	Carries C2PA manifest store in JUMBF container.
eXIf chunk	technical-provenance	Carries EXIF IFD structure with capture device, settings, and datetime fields.
iTXt chunk (XMP)	provenance-chain	Carries XMP RDF/XML packet including xmpMM:History, xmpMM:DerivedFrom, and IPTC AI fields.
CRC-32 (per chunk)	media-integrity	32-bit cyclic redundancy check covering each chunk's type and data.

PNG defines a chunk-based binary structure. The custom caBX chunk carries C2PA manifest stores. The iTXt chunk carries XMP metadata. EXIF data is carried in an eXIf chunk. PNG includes a CRC-32 per chunk for structural integrity. These are intrinsic features of the PNG specification (ISO/IEC 15948).

5.4.4 TCR4CAP Comments

Awareness: PNG chunk structure is defined in ISO/IEC 15948. caBX C2PA embedding is defined in the C2PA Technical Specification. Per-chunk CRC-32 is implemented by all conformant PNG encoders and decoders.

Tamper Evidence: Per-chunk CRC-32 detects accidental corruption of any chunk. C2PA caBX provides cryptographic tamper-evidence when present. CRC-32 does not detect intentional replacement of a chunk with a recalculated CRC.

Binding: C2PA caBX provides hard binding: the manifest contains a hash of the image content. Per-chunk CRC-32 binds each chunk's integrity check to its content but is not a cryptographic signature.

AI Attribution: C2PA AI attribution assertions are carried in caBX when present. IPTC 2025.1 AI fields are carried in XMP via iTXt.

Substantiation: C2PA manifest store in caBX preserves transformation history. XMP in iTXt provides additional descriptive provenance.

Interoperability: PNG is defined in ISO/IEC 15948 and widely implemented. caBX C2PA is supported by c2patool and c2pa-rs reference implementations.

5.5 TIFF Specific Features (IFD Tags)

ID: tiff-features

Type: format-specific

5.5.1 Editability

Level	Type	Notes
File-level	embedded-writable	IFD tags can be written or updated without re-encoding the image data. Tag values are replaced on write.
Content-level	overwrite	

5.5.2 Verifiability

Dimension	Type	Notes
Media integrity	none	No image-data integrity mechanism in the TIFF 6.0 specification.

Dimension	Type	Notes
Metadata integrity	none	IFD tags have no integrity protection; any tag can be overwritten without detection.
Chain of custody	none	No signing or provenance chain mechanism in the TIFF 6.0 specification. No C2PA embedding defined.

FDD References:

- [FDD000022](#) (full:)
- [FDD000145](#) (full:)

5.5.3 Metadata Fields

Field	CAP Concept	Description
Tag 306 (DateTime)	creation-datetime	Date and time of image creation in YYYY:MM:DD HH:MM:SS format.
Tag 271 (Make) / Tag 272 (Model)	actor-identity	Camera manufacturer and model name.
Tag 700 (XMP)	provenance-chain	XMP RDF/XML packet; carries xmpMM:History, xmpMM:DerivedFrom, IPTC AI fields, and other provenance vocabularies.
Tag 33723 (IPTC-IIM)	actor-identity	IPTC Information Interchange Model binary data block.
SubIFD (EXIF)	technical-provenance	EXIF IFD structure with capture device, settings, and GPS fields.

TIFF uses an Image File Directory (IFD) tag structure to carry metadata. EXIF metadata is carried in a SubIFD; XMP is carried in tag 700; IPTC is carried in tag 33723. No C2PA embedding is defined in the current TIFF specification; C2PA is delivered via sidecar for TIFF assets.

5.5.4 TCR4CAP Comments

Awareness: TIFF IFD tag structure is defined in TIFF 6.0. Tag 700 XMP and tag 33723 IPTC are widely implemented in archival imaging tools. No C2PA embedding path exists in the current specification.

Tamper Evidence: IFD tags have no integrity protection and can be silently overwritten or stripped without detection.

Binding: Metadata tags are structurally embedded in the IFD alongside the image data but there is no cryptographic binding between tag values and the image bitstream.

AI Attribution: IPTC 2025.1 AI fields can be carried via XMP in tag 700. No purpose-built AI attribution fields exist in the TIFF 6.0 specification.

Substantiation: Tag 700 XMP and tag 33723 IPTC provide descriptive provenance fields. No mechanism links IFD tags to external policies or trust anchors.

Interoperability: TIFF 6.0 is an open specification widely implemented across archival imaging tools and repository systems.

5.6 MP4 / ISOBMFF Specific Features (Atoms)

ID: mp4-features

Type: format-specific

5.6.1 Editability

Level	Type	Notes
File-level	embedded-writable	Metadata atoms can be written or updated without re-encoding the media data. Metadata atom values are replaced on write; C2PA uuid atom is append-only at the manifest store level.
Content-level	overwrite	

5.6.2 Verifiability

Dimension	Type	Notes
Media integrity	none	No media-data integrity mechanism in the ISO 14496-12 base specification.
Metadata integrity	none	Metadata atoms have no integrity protection in the base specification.
Chain of custody	signed	C2PA manifest store in uuid atom provides cryptographic signing and hard binding to media content when implemented.

FDD References:

- [FDD000155](#) (full:)
- [FDD000525](#) (preliminary:)

5.6.3 Metadata Fields

Field	CAP Concept	Description
uuid atom (C2PA UUID)	provenance-chain	Carries C2PA manifest store in JUMBF container; UUID value defined in C2PA Technical Specification.
uuid atom (XMP UUID)	provenance-chain	Carries XMP RDF/XML packet including xmpMM:History, xmpMM:DerivedFrom, and IPTC AI fields.
udta / moov metadata atoms	creation-datetime	Carries creation date, encoder identity, and copyright fields.

MP4 and ISO Base Media File Format use a hierarchical atom (box) structure. C2PA manifest stores are embedded in a uuid atom with a defined C2PA UUID. Descriptive metadata is carried in udta and moov atoms. XMP can be carried in a uuid atom with the XMP UUID. Covers MP4, MOV, MJ2, 3GP, and other ISOBMFF derivatives.

5.6.4 TCR4CAP Comments

Awareness: ISOBMFF atom structure is defined in ISO 14496-12. C2PA uuid atom embedding is defined in the C2PA Technical Specification Appendix A. uuid atom XMP embedding is defined in the XMP specification.

Tamper Evidence: Metadata atoms have no integrity protection in the base specification. C2PA uuid atom provides cryptographic tamper-evidence when present; modification of the media data invalidates the C2PA hard binding.

Binding: C2PA uuid atom provides hard binding: the manifest contains a hash of the media content. udta and moov metadata atoms are structurally embedded but not cryptographically bound.

AI Attribution: C2PA AI attribution assertions are carried in the uuid atom when present.

Substantiation: C2PA manifest store in uuid atom preserves transformation history with external trust anchor support.

Interoperability: ISO 14496-12 is an ISO standard. MP4 is widely implemented across broadcast, streaming, and archival video tools. C2PA uuid atom is supported by c2patool and c2pa-rs.

5.7 Matroska Specific Features (EBML Tags, CRC-32, and Attachments)

ID: mkv-features

Type: format-specific

5.7.1 Editability

Level	Type	Notes
File-level	embedded-writable	EBML tag elements and attachment elements can be written or updated without re-encoding the media data. The Tags top-level element is structurally independent of the Cluster elements that carry media data.
Content-level	overwrite	Tag values are replaced on write. Updating a tag rewrites the Tags element in place or appends a new Tags element; if present, the CRC-32 of the Tags element should be recomputed on any edit.

5.7.2 Verifiability

Dimension	Type	Notes
Media integrity	checksum	CRC-32 elements are defined in RFC 8794 (EBML) and applied per top-level element in Matroska. RFC 9559 §6.2 states that all Top-Level Elements (SeekHead, Info, Tracks, Chapters, Cluster, Cues, Attachments, Tags) SHOULD include a CRC-32 as their first child element. The CRC-32 covers all element data of the parent element excluding the CRC-32 element itself, using IEEE-CRC-32 (little-endian). FFmpeg writes CRC-32 in all Level 1 elements by default (write_crc32=1). This provides per-element integrity verification but does not constitute a whole-file hash or a cryptographic signature.
Metadata integrity	checksum	The Tags top-level element SHOULD carry its own CRC-32 child, covering the tag bytes. This allows detection of accidental corruption of the Tags element. It does not prevent intentional modification; any writer can recompute a valid CRC-32 after altering tag values.
Chain of custody	none	No signing or provenance chain mechanism is defined in the current Matroska specification. No C2PA embedding path is defined in RFC 9559.

FDD References:

- [FDD000342](#) (full:)

5.7.3 Metadata Fields

Field	CAP Concept	Description
TAG / SimpleTag (TagName=DATE_RECORDED)	creation-datetime	The time that the recording began.
TAG / SimpleTag (TagName=DATE_ENCODED)	creation-datetime	The time that encoding of this item was completed.
TAG / SimpleTag (TagName=DATE_TAGGED)	creation-datetime	The time that the tags were written for this item.
TAG / SimpleTag (TagName=DATE_DIGITIZED)	creation-datetime	The time that the item was transferred to a digital medium.
TAG / SimpleTag (TagName=ENCODER)	actor-identity	The software or hardware used to encode this item.
TAG / SimpleTag (TagName=ENCODER_SETTINGS)	actor-identity	A list of the settings used for encoding this item.
TAG / SimpleTag (TagName=ENCODED_BY)	actor-identity	The person or organization that encoded the item.
TAG / SimpleTag (TagName=ORIGINAL)	provenance-chain	Nested container tag describing the original work this item is based on. Any tag may be nested under ORIGINAL to describe the source work.
TAG / SimpleTag (TagName=ORIGINAL_MEDIA_TYPE)	provenance-chain	Describes the original medium type of the source material (e.g., DVD, CD, film).
TAG / SimpleTag (TagName=SAMPLE)	provenance-chain	Nested container tag describing a sample used in the item taken from another work.
TAG / SimpleTag (TagName=COPYRIGHT)	rights	Copyright information as per the copyright holder.
TAG / SimpleTag (TagName=LICENSE)	rights	The license applied to the content (e.g., Creative Commons variants).
TAG / SimpleTag (TagName=TERMS_OF_USE)	rights	The terms of use for this item.
CRC-32 (per top-level element)	media-integrity	IEEE-CRC-32 checksum covering all element data of each top-level EBML element. Written by default by FFmpeg (write_crc32=1). Defined in RFC 8794 §11.3.1.
Attachments / AttachedFile	provenance-chain	Arbitrary file attachment; can carry sidecar metadata files including PREMIS or C2PA manifests.

Matroska uses an EBML structure defined in RFC 9559. Metadata is carried in SimpleTag elements within a Tags master element; the tag vocabulary is defined in the IETF CELLAR working group tags draft. RFC 9559 §6.2 specifies that all top-level elements SHOULD carry a CRC-32 child element (IEEE-CRC-32, covering all sibling element data within the parent); FFmpeg implements this by default. CRC-32 provides per-element corruption detection but is not a cryptographic integrity mechanism and does not prevent intentional modification. The ORIGINAL and SAMPLE nested tag containers allow structured description of source and sample provenance. Attachments can carry arbitrary sidecar files. No C2PA embedding path is defined in the current Matroska specification.

5.7.4 TCR4CAP Comments

Awareness: Matroska tag structure and CRC-32 behavior are defined in IETF RFC 9559 and RFC 8794. Tag elements are implemented in mkvtoolnix, FFmpeg, and VLC. No CAP-specific tag vocabulary is defined in the Matroska specification. The ORIGINAL, SAMPLE, and DATE_* tags provide a richer provenance vocabulary than most container formats but are not mapped to any current CAP framework.

Tamper Evidence: CRC-32 elements on each top-level EBML element detect accidental corruption of that element’s data. They do not prevent intentional modification; any writer can recompute a valid CRC-32 after altering content. EBML tag elements have no cryptographic integrity protection.

Binding: Tag elements are structurally embedded in the EBML container. CRC-32 binds element content to its checksum at write time but there is no cryptographic binding between tag values and the media data, and no external identity is involved.

AI Attribution: No AI attribution fields are defined in the Matroska tag vocabulary.

Substantiation: The ORIGINAL, ORIGINAL_MEDIA_TYPE, SAMPLE, ENCODER, ENCODER_SETTINGS, and DATE_* tags provide a structured record of encoding history and source material. LICENSE and TERMS_OF_USE fields support rights documentation. No mechanism links these values to external policies or trust anchors.

Interoperability: Matroska is defined in IETF RFC 9559. Tag vocabulary is defined in the IETF CELLAR tags draft. Widely implemented in FFmpeg, VLC, and mkvtoolnix. CRC-32 verification is supported by mkvinfo and ffprobe.

5.8 Vorbis Comment

ID: vorbis-comment

Type: embedded-metadata

5.8.1 Editability

Level	Type	Notes
File-level	embedded-writable	The Vorbis Comment block is embedded in the file and can be rewritten by tooling without re-encoding the primary content.
Content-level	overwrite	Fields are single-value and replaced on write.

5.8.2 Verifiability

Dimension	Type	Notes
Media integrity	none	Vorbis Comment carries no media-data integrity mechanism.
Metadata integrity	none	No integrity protection on the Vorbis Comment block; fields can be silently overwritten without detection.
Chain of custody	none	No provenance chain mechanism in the Vorbis Comment specification.

FDD References:

- [FDD000198](#) (full:)

5.8.3 Metadata Fields

Field	CAP Concept	Description
ARTIST	actor-identity	Name of the performing artist or creator.
DATE	creation-datetime	Date of recording or release.
ENCODER	actor-identity	Name and version of the encoding software.

5.8.4 Used by Formats

ID	Name
flac	FLAC (Free Lossless Audio Codec)

Embedded metadata blocks for human-readable key=value pairs. Fields are single-value and replaced on write. No integrity protection of the block itself. Used by FLAC, Ogg Vorbis, Ogg Opus, and Ogg FLAC.

5.8.5 TCR4CAP Comments

Awareness: Vorbis Comment fields are widely understood in audio workflows but there is no standardized mapping to CAP vocabularies.

Tamper Evidence: Vorbis Comment fields have no integrity protection and can be silently overwritten or stripped without detection.

Binding: Fields are structurally embedded in the file but there is no cryptographic binding between comment values and the audio content.

AI Attribution: No defined AI attribution fields; any AI documentation must repurpose general comment fields with no standardized vocabulary.

Substantiation: Values are readable but there is no mechanism for linking to external policies or transformation histories.

Interoperability: Vorbis Comment is an open, widely implemented specification; fields are plain text and broadly readable.

5.9 ID3 General Encapsulated Object (GEOB) — C2PA

ID: id3-geob-c2pa

Type: embedded-c2pa

5.9.1 Editability

Level	Type	Notes
File-level	embedded-writable	The GEOB frame can be written or rewritten within the host file without re-encoding the primary content.
Content-level	append-only	The C2PA manifest store within the GEOB payload is append-only; individual signed manifests are immutable once signed, and new manifests are appended on each edit.

5.9.2 Verifiability

Dimension	Type	Notes
Media integrity	signed	C2PA hard binding links the manifest cryptographically to the audio content via a content hash at signing time.
Metadata integrity	signed	Each manifest is cryptographically signed; any modification to the GEOB payload invalidates the signature.
Chain of custody	signed	The append-only manifest store preserves the full provenance chain; each manifest references prior manifests as ingredients.

5.9.3 Metadata Fields

Field	CAP Concept	Description
GEOB frame (MIME type: <code>application/c2pa</code>)	provenance-chain	ID3 GEOB frame carrying the C2PA manifest store binary payload.
<code>c2pa.claim.generator</code>	actor-identity	Software or hardware that generated the C2PA claim; carried within the manifest.
<code>c2pa.actions</code>	transformation-history	Ordered list of actions performed on the asset; carried within the manifest.
<code>c2pa.hash.data</code>	media-integrity	Cryptographic hash of the audio content; provides hard binding within the manifest.
<code>c2pa.ai.generatedWith</code>	ai-attribution	Identity of the AI model used to generate the asset; carried within the manifest.

5.9.4 Used by Formats

ID	Name
<code>flac</code>	FLAC (Free Lossless Audio Codec)

C2PA manifest store embedded via an ID3 tag using the General Encapsulated Object frame (GEOB). Each edit by a C2PA-capable tool appends a new signed manifest to the store; individual manifests are immutable once signed. The GEOB frame is not covered by FLAC’s included

checksums (MD5/CRC), so integrity of the manifest depends entirely on the C2PA cryptographic signing chain. Applicable to any format supporting ID3 tags (FLAC, MP3, AIFF).

5.9.5 TCR4CAP Comments

Awareness: C2PA is a defined, published standard; the GEOB embedding path is specified in the C2PA Technical Specification, though not yet widely implemented in FLAC-specific tooling.

Tamper Evidence: Individual manifests are cryptographically signed and immutable; the append-only store preserves the full provenance chain. Tampering with the GEOB payload invalidates the signature.

Binding: C2PA hard binding links the manifest cryptographically to the audio content; the GEOB frame provides a standardized embedding location within the ID3 structure.

AI Attribution: C2PA supports structured AI attribution assertions (c2pa.ai.generatedWith, training data, model identity) within the manifest.

Substantiation: C2PA manifests can reference external trust anchors and credential issuers; transformation history is preserved in the manifest store across the asset lifecycle.

Interoperability: C2PA is an open, published specification; GEOB is a standard ID3 frame; implementation breadth in FLAC tooling is currently limited.

5.10 BWF BEXT Chunk

ID: bwf-bext

Type: embedded-metadata

URL: guidelines: <https://www.digitizationguidelines.gov/guidelines/digitize-embedding.html>

5.10.1 Editability

Level	Type	Notes
File-level	embedded-writable	The BEXT chunk is embedded in the WAVE file and can be rewritten by tooling without re-encoding the audio data. CodingHistory is conventionally appended on each processing step; other BEXT fields are overwritten on write.
Content-level	append	

5.10.2 Verifiability

Dimension	Type	Notes
Media integrity	none	BEXT carries no media-data integrity mechanism.
Metadata integrity	none	No integrity protection on the BEXT chunk; fields can be silently overwritten without detection.
Chain of custody	none	CodingHistory is a conventional append-only log with no enforcement; no signing or provenance chain mechanism.

FDD References:

- [FDD000356](#) (full:)
- [FDD000357](#) (full:)

5.10.3 Metadata Fields

Field	CAP Concept	Description
Originator	actor-identity	Name of the originating organization; 32 bytes.
OriginatorReference	asset-identity	Unique identifier assigned by the originator; 32 bytes.
OriginationDate	creation-datetime	Date of origination in YYYY-MM-DD format.
OriginationTime	creation-datetime	Time of origination in HH-MM-SS format.
UMID	asset-identity	SMPTE Unique Material Identifier; 64 bytes.
CodingHistory	transformation-history	Free-text log of processing steps; conventionally appended on each encode or transfer.

The Broadcast Extension chunk defined in EBU Tech 3285. Carries originator identity, originator reference, origination date and time, time reference (sample offset), UMID, loudness values, and a CodingHistory field. BEXT is the primary provenance carrier in BWF files. Fields can be silently overwritten; changes are not detectable without an external fixity record or audio-data MD5 comparison. Validated against FADGI, EBU, and Microsoft rules by BWF MetaEdit.

5.10.4 TCR4CAP Comments

Awareness: BEXT is defined in EBU Tech 3285. Originator, OriginatorReference, Origination-Date/Time, and CodingHistory are recognized provenance fields in broadcast and archival audio workflows. FADGI and EBU publish guidelines for consistent field use.

Tamper Evidence: BEXT fields have no integrity protection. Any field can be overwritten without detection. CodingHistory is appended by convention; no mechanism enforces append-only behavior or detects deletion of history entries.

Binding: BEXT is embedded in the RIFF container alongside the audio data. No cryptographic mechanism links BEXT field values to the audio bitstream.

AI Attribution: No fields in the BEXT specification address AI generation, training, or processing provenance. CodingHistory is a free-text field that could carry AI processing notes by convention.

Substantiation: CodingHistory documents processing steps in a structured free-text format. UMID provides a globally unique material identifier. No mechanism links BEXT fields to external policies or trust anchors.

Interoperability: Defined in EBU Tech 3285. Implemented across broadcast and archival audio tools. FADGI and EBU publish open guidelines for field use.

5.11 BWF LIST-INFO Chunk

ID: bwf-info

Type: embedded-metadata

URL: guidelines: <https://www.digitizationguidelines.gov/guidelines/digitize-embedding.html>

5.11.1 Editability

Level	Type	Notes
File-level	embedded-writable	The LIST-INFO chunk is embedded in the WAVE file and can be rewritten by tooling without re-encoding the audio data.
Content-level	overwrite	INFO fields are replaced on write.

5.11.2 Verifiability

Dimension	Type	Notes
Media integrity	none	LIST-INFO carries no media-data integrity mechanism.
Metadata integrity	none	No integrity protection; fields can be silently overwritten without detection.
Chain of custody	none	No provenance chain mechanism in the RIFF INFO specification.

FDD References:

- [FDD000356](#) (full:)
- [FDD000357](#) (full:)

5.11.3 Metadata Fields

Field	CAP Concept	Description
ICRD	creation-datetime	Date of creation.
IART	actor-identity	Name of the artist or creator.
ISFT	actor-identity	Name of the software used to create the file.
ICOP	rights	Copyright statement.

The RIFF LIST-INFO chunk carrying standard INFO tags. A secondary metadata carrier in BWF files, complementing BEXT. Fields are plain text with no integrity protection. Less structured than BEXT; no provenance-specific fields. Validated by BWF MetaEdit against Microsoft WAVE specifications.

5.11.4 TCR4CAP Comments

Awareness: INFO tags are widely understood as descriptive metadata but are not specifically designed for provenance; no standardized mapping to CAP vocabularies.

Tamper Evidence: LIST-INFO fields have no integrity protection and can be silently overwritten or stripped without detection.

Binding: Fields are structurally embedded in the file but there is no cryptographic binding between INFO field values and the audio content.

AI Attribution: No defined AI attribution fields; general comment fields could be repurposed but with no standardized vocabulary.

Substantiation: Plain text fields with no mechanism for linking to external policies or transformation histories.

Interoperability: RIFF INFO is an open, widely implemented specification; fields are plain text and broadly readable across tools.

5.12 BWF XML Chunks (aXML / iXML)

ID: bwf-xml

Type: embedded-metadata

URL: guidelines: <https://www.digitizationguidelines.gov/guidelines/digitize-embedding.html>

5.12.1 Editability

Level	Type	Notes
File-level	embedded-writable	The aXML and iXML chunks are embedded in the WAVE file and can be rewritten by tooling without re-encoding the audio data.
Content-level	overwrite	Each XML chunk's content is replaced on write.

5.12.2 Verifiability

Dimension	Type	Notes
Media integrity	none	aXML and iXML carry no media-data integrity mechanism.
Metadata integrity	none	No integrity protection on either chunk; content can be silently overwritten without detection.
Chain of custody	none	No provenance chain mechanism in the iXML or aXML specifications.

FDD References:

- [FDD000356](#) (partial:)
- [FDD000357](#) (partial:)

5.12.3 Metadata Fields

Field	CAP Concept	Description
aXML / EBU-ISRC	asset-identity	EBU International Standard Recording Code and additional production metadata.
iXML / RECORDER	actor-identity	Name and model of the recording device.
iXML / TRACK_LIST	technical-provenance	List of recorded tracks with channel assignments.

Two distinct XML-carrying chunks supported in BWF/WAVE files. The aXML chunk (EBU Tech 3285 supplement) carries additional XML data including EBU ISRC and production metadata. The iXML chunk carries production sound metadata (scene, take, track names, recorder metadata) per the iXML specification. BWF MetaEdit supports reading and writing both chunk types. Neither chunk has integrity protection of its own.

5.12.4 TCR4CAP Comments

Awareness: iXML and aXML/EBU-ISRC are well-established in broadcast production workflows but are not yet addressed in FADGI preservation guidelines.

Tamper Evidence: aXML and iXML fields have no integrity protection and can be silently overwritten or stripped without detection.

Binding: Chunks are structurally embedded in the file but there is no cryptographic binding between chunk content and the audio bitstream.

AI Attribution: No defined AI attribution fields in iXML or aXML vocabularies.

Substantiation: iXML provides structured production context (scene, take, recorder); aXML/EBU-ISRC supports rights and identifier linkage. No mechanism for linking to external trust anchors.

Interoperability: iXML is an open specification; aXML/EBU-ISRC is an EBU standard. Both are broadly supported in professional audio and DAW tooling.

5.13 BWF Audio-Data MD5 Checksum

ID: bwf-md5

Type: integrity

URL: guidelines: <https://www.digitizationguidelines.gov/guidelines/digitize-embedding.html>

5.13.1 Editability

Level	Type	Notes
File-level	embedded-writable	The MD5 value is stored within the WAVE file and can be written, updated, or verified by BWF MetaEdit without re-encoding the audio data.
Content-level	fixed	The MD5 is computed over the audio data chunk only and is fixed at write time; it must be recomputed if the audio data changes.

5.13.2 Verifiability

Dimension	Type	Notes
Media integrity	checksum	MD5 computed over the WAVE audio data chunk payload. Detects changes to the audio bitstream. Recomputable by BWF MetaEdit and any MD5-capable tool given the raw data chunk.
Metadata integrity	none	The MD5 covers the audio data chunk only; metadata chunks (BEXT, INFO, XML) are not covered and can be altered without invalidating the checksum.
Chain of custody	none	No signing or provenance chain mechanism; the MD5 value itself is not signed and could be replaced along with modified audio data.

FDD References:

- [FDD000356](#) (partial:)
- [FDD000357](#) (partial:)

5.13.3 Metadata Fields

Field	CAP Concept	Description
MD5 digest (audio data chunk)	media-integrity	MD5 hash of the WAVE data chunk payload, excluding chunk ID, size declaration, and optional padding byte.

An MD5 checksum computed over the WAVE file’s audio data chunk, embedded within the file by BWF MetaEdit. Detects changes to the audio bitstream. Does not cover metadata chunks (BEXT, INFO, XML). Analogous to FLAC’s defined STREAMINFO MD5, but applied at the tool level rather than defined in the format specification. MD5 is not collision-resistant for adversarial contexts.

5.13.4 TCR4CAP Comments

Awareness: Audio-data MD5 is a well-understood fixity practice in broadcast and preservation workflows; BWF MetaEdit’s implementation follows FADGI guidance.

Tamper Evidence: Detects accidental or unintentional changes to the audio bitstream. Does not cover metadata chunks. MD5 is not collision-resistant for adversarial tampering; the checksum value itself is not signed.

Binding: The MD5 is computed over and embedded alongside the specific audio data chunk, providing a direct integrity link between the checksum value and the audio content within the same file.

AI Attribution: No relevance to AI attribution; the checksum documents data integrity only.

Substantiation: Provides a verifiable, reproducible integrity record for the audio bitstream. Supported by FADGI guidance. Exportable for external audit. Limited to the data chunk.

Interoperability: MD5 is a universal algorithm; the checksum can be verified by any MD5 tool. The embedding convention is specific to BWF MetaEdit / FADGI practice and not universally standardized across all WAVE-capable tools.

5.14 BWF C2PA Chunk

ID: bwf-c2pa

Type: embedded-c2pa

5.14.1 Editability

Level	Type	Notes
File-level	embedded-writable	The C2PA chunk is embedded in the WAVE/RIFF file and can be written or replaced by C2PA-capable tooling without re-encoding the audio data.
Content-level	append-only	The C2PA Manifest Store is cryptographically signed; individual manifests are immutable once signed. New manifests are appended; prior manifests are preserved as ingredient references.

5.14.2 Verifiability

Dimension	Type	Notes
Media integrity	signed	C2PA hard binding: the active manifest contains a cryptographic hash of the audio content at signing time. Any modification to the audio data invalidates the binding.
Metadata integrity	signed	Each manifest is cryptographically signed using CMS/COSE. Any modification to a manifest's assertions or claim invalidates the signature.
Chain of custody	signed	The manifest store is append-only; prior manifests are preserved as ingredient references. The full provenance chain is verifiable against the signing certificate chain and optional timestamp authority.

5.14.3 Metadata Fields

Field	CAP Concept	Description
C2PA RIFF chunk (fourCC: C2PA)	provenance-chain	RIFF chunk carrying the C2PA Manifest Store binary payload.
c2pa.claim.generator	actor-identity	Software or hardware that generated the C2PA claim.

Field	CAP Concept	Description
<code>c2pa.actions</code>	transformation-history	Ordered list of actions performed on the asset.
<code>c2pa.hash.data</code>	media-integrity	Cryptographic hash of the audio content; provides hard binding.
<code>c2pa.ai.generatedWith</code>	ai-attribution	Identity of the AI model used to generate the asset.
<code>c2pa.training-mining</code>	ai-attribution	Assertion about whether the asset may be used for AI training or data mining.
<code>c2pa.claim.signature</code>	provenance-chain	CMS/COSE signature over the claim; includes signing certificate and optional timestamp.

A RIFF chunk with four-character code C2PA carrying a C2PA Manifest Store, as defined in the C2PA Technical Specification (Appendix A). Applies to WAV, BWF, and AVI files. The manifest store contains one or more C2PA Manifests, each comprising a Claim, Claim Signature (COSE/X.509), and Assertions. The chunk is cryptographically signed; any modification to the audio data or metadata invalidates the signature. BWF MetaEdit can detect the chunk's presence but does not author or validate C2PA manifests; authoring is performed by c2patool, c2pa-rs, and compatible capture devices.

5.14.4 TCR4CAP Comments

Awareness: C2PA Technical Specification is published by the Coalition for Content Provenance and Authenticity. The C2PA RIFF chunk is defined in the specification Appendix A. No current version of BWF MetaEdit writes this chunk.

Tamper Evidence: The manifest is cryptographically signed; any modification to the signed content invalidates the signature. Hard binding means any modification to the audio data is detectable.

Binding: Hard binding: the C2PA Manifest Store is embedded in the file and the Claim covers a hash of the audio data chunk, cryptographically linking the manifest to the specific audio bitstream at signing time.

AI Attribution: C2PA defines standardized assertions for AI generative and training provenance: `c2pa.ai.generatedWith`, `c2pa.training-mining`, and related fields for model identity and generation parameters.

Substantiation: Signed assertions are verifiable against a public certificate chain and optional timestamp authority. Manifest history preserves a chain of custody across multiple signing events.

Interoperability: C2PA is an open, published specification. Tooling support for WAV/BWF is nascent; c2patool and c2pa-rs support it but broadcast and preservation tools do not yet recognize the chunk.

5.15 XMP Embedded Metadata

ID: xmp-embedded

Type: embedded-metadata

5.15.1 Editability

Level	Type	Notes
File-level	embedded-writable	XMP packets are embedded within the host file in a format-specific container (XMP chunk in WAVE, APP1 in JPEG, tag 700 in TIFF, uuid atom in MP4, iTXt in PNG, etc.) and can be rewritten by tooling without re-encoding the primary content.
Content-level	overwrite	The XMP packet is replaced on write. The xmpMM:History array supports internal modification history by convention but entries are advisory and not integrity-protected.

5.15.2 Verifiability

Dimension	Type	Notes
Media integrity	none	XMP carries no media-data integrity mechanism.
Metadata integrity	none	XMP packets have no integrity protection; the packet can be silently overwritten or stripped without detection.
Chain of custody	none	xmpMM:History provides a conventional modification log; entries are advisory and unverified. No signing mechanism in the XMP specification.

5.15.3 Metadata Fields

Field	CAP Concept	Description
xmp:CreateDate	creation-datetime	Date and time the resource was originally created.
xmp:CreatorTool	actor-identity	Name and version of the tool used to create or last modify the resource.
xmpMM:DocumentID	asset-identity	Persistent identifier for the document across all versions and renditions.
xmpMM:InstanceID	asset-identity	Identifier for the specific version or rendition of the document.
xmpMM:DerivedFrom	provenance-chain	Reference to the source asset from which this asset was derived; carries DocumentID and InstanceID of the source.
xmpMM:History	transformation-history	Array of ResourceEvent entries documenting modification history; each entry carries action, instanceID, when, and softwareAgent.
dc:rights	rights	Dublin Core rights statement.
dc:source	provenance-chain	Dublin Core source field; reference to a resource from which this resource is derived.
Iptc4xmpExt:DigitalSourceType	ai-attribution	IPTC controlled vocabulary term for the digital source type (e.g. trainedAlgorithmicMedia, compositeSynthetic).

5.15.4 Used by Formats

ID	Name
bwf	Broadcast WAVE Format (BWF)
flac	FLAC (Free Lossless Audio Codec)
jpeg	JPEG
png	PNG (Portable Network Graphics)
tiff	TIFF (Tagged Image File Format)
mp4-isobmff	MP4 / ISO Base Media File Format
mkv	Matroska (MKV)

XMP (Extensible Metadata Platform) is an RDF/XML metadata format defined by Adobe and standardized as ISO 16684. Embedded in host files via format-specific containers or stored as standalone .xmp sidecar files. Supports multiple namespaces including Dublin Core (dc:), XMP Basic (xmp:), XMP Media Management (xmpMM:), and IPTC (Iptc4xmpCore/Ext:). The xmpMM:History array provides a conventional provenance record of processing steps. XMP packets have no integrity protection of their own.

5.15.5 TCR4CAP Comments

Awareness: XMP is defined in ISO 16684. Implemented across creative, archival, and repository tools. xmpMM:History and xmpMM:DerivedFrom are recognized provenance fields in digital asset management workflows.

Tamper Evidence: XMP packets have no integrity protection and can be silently overwritten or stripped without detection. xmpMM:History entries are advisory and unverified.

Binding: XMP is structurally embedded in host files but there is no cryptographic binding between XMP content and the primary media data. The packet can be removed without affecting the media bitstream.

AI Attribution: Iptc4xmpExt:DigitalSourceType and IPTC 2025.1 AI fields (AiPrompt, AiSystemUsed, AiSystemVersion) are carried in XMP namespaces. No purpose-built AI attribution fields exist in the core XMP specification itself.

Substantiation: xmpMM:History and xmpMM:DerivedFrom provide a readable provenance record. XMP sidecar files can be published independently. No mechanism for cryptographic verification of history entries.

Interoperability: XMP is defined in ISO 16684. Implemented across all major creative, archival, and repository tools. Embedded in a wide range of format containers.

5.16 EXIF Embedded Metadata

ID: exif-embedded

Type: embedded-metadata

5.16.1 Editability

Level	Type	Notes
File-level	embedded-writable	EXIF IFD data can be written or replaced in host files without re-encoding primary content.
Content-level	overwrite	EXIF tag values are replaced on write.

5.16.2 Verifiability

Dimension	Type	Notes
Media integrity	none	EXIF carries no media-data integrity mechanism.
Metadata integrity	none	EXIF data has no integrity protection; tags can be silently overwritten or stripped.
Chain of custody	none	No provenance chain mechanism in the EXIF specification.

FDD References:

- [FDD000145](#) (full:)
- [FDD000146](#) (full:)
- [FDD000147](#) (full:)
- [FDD000618](#) (full:)

5.16.3 Metadata Fields

Field	CAP Concept	Description
Make (0x010F) / Model (0x0110)	actor-identity	Camera manufacturer and model name.
Software (0x0131)	actor-identity	Name and version of software used to process the image.
DateTimeOriginal (0x9003)	creation-datetime	Date and time the original image was captured.
DateTimeDigitized (0x9004)	creation-datetime	Date and time the image was digitized.
GPSLatitude / GPSLongitude (0x0002 / 0x0004)	creation-location	GPS coordinates of capture location.
ImageUniqueID (0xA420)	asset-identity	Unique identifier assigned to the image by the camera or software.
BodySerialNumber (0xA431)	actor-identity	Serial number of the camera body.

5.16.4 Used by Formats

ID	Name
jpeg	JPEG
png	PNG (Portable Network Graphics)

ID	Name
tiff	TIFF (Tagged Image File Format)

EXIF (Exchangeable Image File Format) is a binary metadata standard defined by CIPA (DC-008) using an IFD tag structure. Records capture device identity, camera settings, GPS coordinates, and datetime. Embedded in JPEG APP1, TIFF SubIFD, PNG eXif chunk, and HEIF. EXIF data has no integrity protection and can be silently overwritten or stripped.

5.16.5 TCR4CAP Comments

Awareness: EXIF is defined by CIPA DC-008. Implemented universally across cameras, phones, and image processing tools. Capture device identity and datetime fields are widely used as provenance indicators in photojournalism and archival workflows.

Tamper Evidence: EXIF data has no integrity protection and can be silently overwritten or stripped without detection.

Binding: EXIF is structurally embedded in host files but there is no cryptographic binding between EXIF tag values and the image content.

AI Attribution: No AI attribution fields are defined in the EXIF specification. Software (0x0131) can record processing tool identity but has no structured AI attribution vocabulary.

Substantiation: Capture datetime, GPS, and device fields provide a factual provenance record. No mechanism links EXIF fields to external policies or transformation histories.

Interoperability: EXIF is defined by CIPA DC-008. Universally implemented across cameras, phones, and image processing tools.

5.17 IPTC Photo Metadata

ID: iptc-photo

Type: embedded-metadata

5.17.1 Editability

Level	Type	Notes
File-level	embedded-writable	IPTC fields carried in XMP can be written or replaced in host files without re-encoding primary content.
Content-level	overwrite	IPTC property values are replaced on write.

5.17.2 Verifiability

Dimension	Type	Notes
Media integrity	none	IPTC carries no media-data integrity mechanism.
Metadata integrity	none	IPTC fields carried in XMP have no integrity protection; fields can be silently overwritten or stripped.
Chain of custody	none	No provenance chain mechanism in the IPTC Photo Metadata specification.

5.17.3 Metadata Fields

Field	CAP Concept	Description
<code>Iptc4xmpCore:CreatorContactInfo</code>	author-identity	Contact information for the image creator.
<code>Iptc4xmpExt:DigitalSourceType</code>	ai-attribution	Controlled vocabulary term for the digital source type (e.g. <code>trainedAlgorithmicMedia</code> , <code>compositeSynthetic</code>).
<code>Iptc4xmpExt:AiPrompt</code> (11.2)	ai-attribution	Text prompt used to generate the image with an AI system; IPTC 2025.1.
<code>Iptc4xmpExt:AiSystemUsed</code> (11.4)	ai-attribution	Name of the AI system used to generate or process the image; IPTC 2025.1.
<code>Iptc4xmpExt:AiSystemVersion</code> (11.5)	ai-attribution	Version of the AI system; IPTC 2025.1.
<code>Iptc4xmpExt:DataMiningPermissions</code> (11.11)	rights	Statement of permission for data mining or AI training use; IPTC 2025.1.
<code>Iptc4xmpExt:WebStatement</code>	rights	URL of a rights statement for the image.

5.17.4 Used by Formats

ID	Name
<code>jpeg</code>	JPEG
<code>png</code>	PNG (Portable Network Graphics)
<code>tiff</code>	TIFF (Tagged Image File Format)

The IPTC Photo Metadata Standard defines a vocabulary of fields carried in XMP namespaces (Iptc4xmpCore, Iptc4xmpExt). Version 2025.1 adds dedicated AI fields: AiPrompt (11.2), AiSystemUsed (11.4), AiSystemVersion (11.5), and DataMiningPermission (11.11). Widely used in photojournalism, stock photography, and digital asset management.

5.17.5 TCR4CAP Comments

Awareness: IPTC Photo Metadata Standard is published by IPTC. Version 2025.1 AI fields are defined in the IPTC specification. Widely implemented in photo editing, DAM, and publishing tools.

Tamper Evidence: IPTC fields carried in XMP have no integrity protection and can be silently overwritten or stripped without detection.

Binding: IPTC fields are structurally embedded via XMP in host files but there is no cryptographic binding between field values and the image content.

AI Attribution: Version 2025.1 defines AiPrompt, AiSystemUsed, AiSystemVersion, and DataMiningPermission as purpose-built, standardized AI attribution fields within the IPTC vocabulary.

Substantiation: IPTC fields provide a structured, readable provenance record. The standard is publicly documented. No mechanism links field values to external trust anchors.

Interoperability: IPTC Photo Metadata Standard is an open standard. Implemented in photo editing, DAM, and publishing tools including Adobe Photoshop, Lightroom, and Capture One.

5.18 JUMBF C2PA Manifest Store (Embedded)

ID: jumbf-c2pa

Type: embedded-c2pa

5.18.1 Editability

Level	Type	Notes
File-level	embedded-writable	The JUMBF container carrying the C2PA Manifest Store is embedded in the host file via a format-specific mechanism (APP11 in JPEG, caBX in PNG, uuid atom in MP4, C2PA chunk in RIFF/WAV) and can be written or replaced by C2PA-capable tooling without re-encoding the primary content.
Content-level	append-only	The C2PA Manifest Store is append-only at the store level; individual signed manifests are immutable once signed. New manifests are appended on each edit; prior manifests are preserved as ingredient references.

5.18.2 Verifiability

Dimension	Type	Notes
Media integrity	signed	C2PA hard binding: the active manifest contains a cryptographic hash of the asset content at signing time. Any modification to the asset content invalidates the binding.
Metadata integrity	signed	Each manifest is cryptographically signed using CMS/COSE. Any modification to a manifest's assertions or claim invalidates the signature.
Chain of custody	signed	The manifest store is append-only; prior manifests are preserved as ingredient references. The full provenance chain is verifiable against the signing certificate chain and optional timestamp authority.

FDD References:

- [FDD000653](#) (partial:)

5.18.3 Metadata Fields

Field	CAP Concept	Description
<code>c2pa.claim.generator</code>	actor-identity	Software or hardware that generated the C2PA claim.
<code>c2pa.actions</code>	transformation-history	Ordered list of actions performed on the asset (c2pa.created, c2pa.edited, c2pa.converted, c2pa.transcoded, etc.).
<code>c2pa.ingredient</code>	provenance-chain	Reference to a prior asset version or source asset, including its manifest hash.
<code>c2pa.hash.data</code>	media-integrity	Cryptographic hash of the asset content; provides hard binding between manifest and asset.
<code>c2pa.ai.generatedWith</code>	ai-attribution	Identity of the AI model used to generate the asset.
<code>c2pa.training-mining</code>	ai-attribution	Assertion about whether the asset may be used for AI training or data mining.
<code>c2pa.claim.signature</code>	provenance-chain	CMS/COSE signature over the claim; includes signing certificate and optional timestamp authority token.
<code>c2pa.soft-binding</code>	provenance-chain	Perceptual identifier (watermark or fingerprint) used to link the manifest to the asset when hard binding is not possible or has been broken.

5.18.4 Used by Formats

ID	Name
<code>bwf</code>	Broadcast WAVE Format (BWF)
<code>jpeg</code>	JPEG
<code>png</code>	PNG (Portable Network Graphics)
<code>tiff</code>	TIFF (Tagged Image File Format)
<code>mp4-isobmff</code>	MP4 / ISO Base Media File Format

A C2PA Manifest Store carried in a JUMBF (JPEG Universal Metadata Box Format) container, embedded in the host file via a format-specific mechanism. Consolidates the embedding paths for JPEG (APP11), PNG (caBX), MP4/ISOBMFF (uuid atom), and RIFF/WAV (C2PA chunk) under a single mechanism entry, as all use the same JUMBF container and C2PA manifest structure. Each manifest contains assertions, a claim, and a claim signature. Hard binding links the manifest cryptographically to the asset content via hash. The active manifest is the most recently added; prior manifests are preserved as ingredients.

5.18.5 TCR4CAP Comments

Awareness: C2PA Technical Specification is published by the Coalition for Content Provenance and Authenticity. Adopted by Adobe, Microsoft, Google, camera manufacturers, and news organizations. Reference implementation available as c2pa-rs under an open-source license.

Tamper Evidence: Each manifest is cryptographically signed; any modification invalidates the signature. The append-only store preserves the full provenance chain. Hard binding means any modification to the asset content invalidates the active manifest.

Binding: Hard binding provides cryptographic linkage between manifest and asset content via content hash. Soft binding provides a defined alternative for non-embeddable assets using perceptual identifiers.

AI Attribution: C2PA defines purpose-built assertions for AI generative and training provenance: c2pa.ai.generatedWith, c2pa.training-mining, and related fields for model identity and generation parameters.

Substantiation: Manifest store preserves full transformation history. External trust anchors (credential issuers, TSA) are supported. Public verification is possible via the C2PA trust list and open verification tools.

Interoperability: Open published specification. Reference implementation (c2patool, c2pa-rs) available under open-source license. Adopted across image, video, and audio tools and camera manufacturers. Format-specific embedding paths are defined in the C2PA Technical Specification Appendix A.

5.19 C2PA Sidecar Manifest

ID: sidecar-c2pa

Type: sidecar-c2pa

5.19.1 Editability

Level	Type	Notes
File-level	sidecar-writable	The C2PA manifest store is stored as a standalone .c2pa file alongside the asset; it can be written or replaced independently of the asset file.
Content-level	append-only	The C2PA Manifest Store within the sidecar is append-only; individual signed manifests are immutable once signed.

5.19.2 Verifiability

Dimension	Type	Notes
Media integrity	signed	C2PA hard binding: the active manifest contains a cryptographic hash of the asset content. Any modification to the asset content invalidates the binding. The sidecar file can be separated from the asset, breaking the association.
Metadata integrity	signed	Each manifest is cryptographically signed; any modification to the sidecar payload invalidates the signature.
Chain of custody	signed	The manifest store is append-only; prior manifests are preserved as ingredient references. Verifiable against the signing certificate chain and optional timestamp authority. Association with the asset depends on file management conventions.

5.19.3 Metadata Fields

Field	CAP Concept	Description
<code>c2pa.claim.generator</code>	actor-identity	Software or hardware that generated the C2PA claim.

Field	CAP Concept	Description
<code>c2pa.actions</code>	transformation-history	Ordered list of actions performed on the asset.
<code>c2pa.hash.data</code>	media-integrity	Cryptographic hash of the asset content; provides hard binding to the associated asset file.
<code>c2pa.ai.generatedWith</code>	ai-attribution	Identity of the AI model used to generate the asset.
<code>c2pa.training-mining</code>	ai-attribution	Assertion about whether the asset may be used for AI training or data mining.
<code>c2pa.claim.signature</code>	provenance-chain	CMS/COSE signature over the claim; includes signing certificate and optional timestamp authority token.

5.19.4 Used by Formats

ID	Name
<code>bwf</code>	Broadcast WAVE Format (BWF)
<code>flac</code>	FLAC (Free Lossless Audio Codec)
<code>jpeg</code>	JPEG
<code>png</code>	PNG (Portable Network Graphics)
<code>tiff</code>	TIFF (Tagged Image File Format)
<code>mp4-isobmff</code>	MP4 / ISO Base Media File Format
<code>mkv</code>	Matroska (MKV)

A C2PA Manifest Store stored as a standalone `.c2pa` file alongside the asset, used when embedding in the asset file is not possible or not desired (e.g. for TIFF, camera RAW formats, or read-only assets). The sidecar carries the same JUMBF-encoded manifest store as the embedded path. Hard binding via content hash links the sidecar to the specific asset bitstream, but the physical association between sidecar and asset depends on file management conventions; the sidecar can be separated from the asset without invalidating the manifest signatures.

5.19.5 TCR4CAP Comments

Awareness: C2PA sidecar delivery is defined in the C2PA Technical Specification. Used for formats where embedding is not defined or not practical.

Tamper Evidence: Each manifest is cryptographically signed; modification of the sidecar payload invalidates the signature. The sidecar can be deleted or separated from the asset without detection at the file level.

Binding: Hard binding via content hash links the sidecar to the specific asset bitstream. Physical

co-location of sidecar and asset depends on file management conventions and is not enforced by the format.

AI Attribution: C2PA AI attribution assertions are carried in the sidecar manifest store identically to the embedded path.

Substantiation: Manifest store preserves full transformation history. External trust anchors are supported. Sidecar files can be published or transferred independently of the asset.

Interoperability: C2PA sidecar is defined in the open C2PA Technical Specification. Supported by c2patool and c2pa-rs. Sidecar association conventions vary across repository and DAM systems.

5.20 PREMIS Sidecar / Embedded Event Record

ID: sidecar-premis

Type: sidecar-metadata

5.20.1 Editability

Level	Type	Notes
File-level	sidecar-writable	PREMIS documents are standalone XML files managed independently of the media assets they describe, or embedded within a METS wrapper.
Content-level	append	Preservation events are appended to the PREMIS document over the asset lifecycle; existing events are not modified by convention.

5.20.2 Verifiability

Dimension	Type	Notes
Media integrity	external-only	PREMIS can record fixity values (checksums) for described objects in the objectCharacteristics/fixity element, but does not compute or verify them itself. Verification depends on external tooling.

Dimension	Type	Notes
Metadata integrity	none	PREMIS XML has no integrity protection; the document can be modified without detection. Integrity depends on the repository or packaging layer.
Chain of custody	external-only	PREMIS event records provide a structured provenance chain but entries are not cryptographically signed. Integrity of the event record depends on the repository audit log or packaging layer (e.g. BagIt fixity).

5.20.3 Metadata Fields

Field	CAP Concept	Description
object/objectIdentifier	asset-identity	Persistent identifier for the described object.
object/objectCharacteristics/fixity	fixity-integrity	Fixity algorithm and digest value for the described object.
object/objectCharacteristics/format	format-provenance	Format name, version, and registry designation (e.g. PRONOM PUID) of the described object.
event/eventType	transformation-history	Type of preservation event (e.g. ingestion, normalization, fixity check, deletion).
event/eventDateTime	transformation-history	Date and time of the preservation event.
event/eventOutcomeInformation	transformation-history	Outcome of the event (success, failure) and detail notes.
agent/agentIdentifier	actor-identity	Identifier for the agent (person, organization, or software) responsible for the event.
rights/rightsStatement	rights	Rights basis and rights granted for the described object.

5.20.4 Used by Formats

ID	Name
bwf	Broadcast WAVE Format (BWF)
flac	FLAC (Free Lossless Audio Codec)

ID	Name
jpeg	JPEG
png	PNG (Portable Network Graphics)
tiff	TIFF (Tagged Image File Format)
mp4-isobmff	MP4 / ISO Base Media File Format
mkv	Matroska (MKV)

PREMIS (Preservation Metadata: Implementation Strategies) is an XML schema defined by the Library of Congress for recording preservation metadata. Models Objects, Events, Agents, and Rights. PREMIS events map conceptually to C2PA actions (c2pa.created, c2pa.converted, c2pa.edited); PREMIS agent maps to C2PA softwareAgent or person assertions. Generated by preservation systems including Archivematica, DSpace, and Fedora.

5.20.5 TCR4CAP Comments

Awareness: PREMIS Data Dictionary is published by the Library of Congress. Implemented in Archivematica, DSpace, Fedora, and other repository systems. Event types and agent types are defined vocabularies.

Tamper Evidence: PREMIS XML has no integrity protection. The document can be modified without detection. Tamper-evidence depends on the repository or packaging layer (e.g. BagIt fixity, audit logs).

Binding: PREMIS Object elements reference described files by identifier and path. Fixity values in objectCharacteristics/fixity link the PREMIS record to a specific file state, but the linkage is not cryptographically enforced within the PREMIS document itself.

AI Attribution: No purpose-built AI attribution fields in the PREMIS Data Dictionary. The agent/agentType vocabulary could accommodate AI agents; eventType could accommodate AI processing events by convention.

Substantiation: PREMIS event records provide a structured, policy-driven transformation history. The PREMIS Data Dictionary is an open standard. Event records are auditable by repository administrators.

Interoperability: PREMIS Data Dictionary is published by the Library of Congress as an open standard. Implemented across major digital preservation systems and repository platforms.

5.21 BagIt Package Fixity

ID: bagit-features

Type: package-integrity

5.21.1 Editability

Level	Type	Notes
File-level	package-writable	BagIt manifests are plain text files within the bag directory structure; they can be regenerated when payload files change.
Content-level	fixed	Manifest entries are computed at bag creation time and are fixed for the lifetime of the bag; any change to a payload file requires regenerating the manifest.

5.21.2 Verifiability

Dimension	Type	Notes
Media integrity	checksum	Manifest files (manifest-md5.txt, manifest-sha256.txt, etc.) record checksums for all payload files. Any modification to a payload file produces a checksum mismatch detectable by bag validation.
Metadata integrity	checksum	Tag manifests (tagmanifest-*.txt) record checksums for all tag files including bag-info.txt and fetch.txt. Modification of tag files is detectable by tag manifest validation.
Chain of custody	external-only	BagIt provides no signing mechanism; the integrity of the bag depends on the checksum algorithm and the security of the storage environment. Chain of custody depends on repository audit logs and transfer records.

5.21.3 Metadata Fields

Field	CAP Concept	Description
<code>manifest-*.txt</code> entries	media-integrity	Checksum algorithm and digest for each payload file; one entry per file.
<code>tagmanifest-*.txt</code> entries	metadata-integrity	Checksum algorithm and digest for each tag file.
<code>bag-info.txt</code> / Bagging-Date	creation-datetime	Date the bag was created.
<code>bag-info.txt</code> / Source-Organization	actor-identity	Name of the organization that created the bag.
<code>bag-info.txt</code> / External-Identifier	asset-identity	Identifier assigned to the bag by the source organization.
<code>bag-info.txt</code> / Payload-Oxum	media-integrity	Octet count and stream count of the payload; used for quick validation.
<code>fetch.txt</code>	provenance-chain	List of URLs and expected sizes for payload files to be fetched; used for holey bags.

BagIt (RFC 8493) is a hierarchical file packaging format for reliable digital transfer and storage. A bag contains a payload directory, manifest files recording checksums for all payload files, tag files carrying descriptive metadata, and tag manifest files recording checksums for tag files. Operates at the package level; does not interact with file-level metadata mechanisms. Supported algorithms include MD5, SHA-1, SHA-256, and SHA-512.

5.21.4 TCR4CAP Comments

Awareness: BagIt is defined in IETF RFC 8493. Widely adopted in digital preservation, library, and archival transfer workflows. Implemented in `bagit-python`, `Bagger`, and `Archivematica`.

Tamper Evidence: Manifest checksums detect modification of any payload or tag file. No signing mechanism; the manifest files themselves could be replaced along with modified payload files without detection at the BagIt level.

Binding: Manifest entries bind each checksum to a specific file path within the bag. No cryptographic binding between the bag as a whole and an external identity or signing authority.

AI Attribution: No AI attribution fields in the BagIt specification. `bag-info.txt` custom fields could carry AI processing notes by convention.

Substantiation: Manifest files provide a verifiable, reproducible integrity record for all payload and tag files. `bag-info.txt` carries descriptive provenance fields. No mechanism for linking to external trust anchors.

Interoperability: BagIt is defined in IETF RFC 8493. Implemented across digital preservation, library, and archival transfer tools and systems.

6 Featured Tools: Content Authenticity and Provenance in Practice

The following free and open source tools represent a selection of the audiovisual preservation toolkit. Many of these tools have no current or only minimal or newly developed C2PA implementations but nonetheless can be implemented in a manner to support many provenance features. For each tool, this section provides an overview of its role in provenance documentation, examples of how its outputs can be used to log and verify actions, and considerations for integrating it into a content authenticity and provenance workflow.

6.1 FFmpeg

FFmpeg is foundational to audiovisual preservation workflows. It is used for capture, format migration, rewapping, transcoding, metadata manipulation, and many other transformation or analytical tasks with audiovisual data. It is also a key dependency in larger, open-source audiovisual projects such as DVRescue, QCTools, vrecord, and others. FFmpeg is not a provenance tool by design, but its ubiquity means that it is present or involved at nearly every significant transformation in an audiovisual digital preservation workflow, which makes its logging behavior and metadata handling critical to provenance documentation.

6.1.1 FFmpeg in Digitization Workflows

Digitization tools, such as vrecord, use FFmpeg internally for video capture, controlling hardware capture cards to produce preservation-quality files. In this context FFmpeg is the encoding engine, and the parameters it is invoked with, such as encoding, sampling rate, bit depth, colorspace, and audio channel mapping, constitute the technical provenance of the resulting file. The FFmpeg command line used for capture, which is retained in vrecord's log, is a partially reproducible description of how the file was made.

6.1.2 Metadata Through Transformations

FFmpeg's default metadata handling varies by container and is not always predictable. The `-map_metadata` option provides explicit control. `-map_metadata 0` copies global metadata from the first input file (input numbering starts from zero) to the output. Without this flag, a rewrap operation may silently discard embedded metadata depending on the container pairing.

```
# Rewrap with explicit metadata copying
ffmpeg -i input.mkv -map_metadata 0 -c copy output.mkv

# Copy metadata from one stream explicitly (e.g., audio stream 0)
ffmpeg -i input.mkv -map_metadata:s:a 0:s:a -c copy output.mkv

# Set output metadata fields explicitly
ffmpeg -i input.mkv \
  -map_metadata 0 \
  -metadata title="Authentic Movie" \
```

```
-metadata comment="Rewrapped 2026-08-01, corrected digitization metadata" \
-map 0 -c copy \
output.mkv
```

Although metadata may be mapped explicitly in an ffmpeg command, a field that exists in the input container may have no equivalent in the output container, in which case FFmpeg will silently discard it. Inspecting metadata before and after any transformation, using `ffprobe` or `mediainfo` and diffing the output, is a reliable way to detect this.

```
ffprobe -v quiet -print_format json -show_format -show_streams input.mkv > input_metadata.json
ffmpeg -i input.mkv -map_metadata 0 -c copy output.mkv
ffprobe -v quiet -print_format json -show_format -show_streams output.mkv > output_metadata.json
diff input_metadata.json output_metadata.json
```

While ffmpeg can be used to add or modify metadata when creating a new output file, ffmpeg is not effective at editing metadata for an existing file in place. Other tools do support inline metadata edits, such as `mkvpropedit` or `mkvnote` for Matroska, `BWF MetaEdit` for Broadcast WAVE files, and `embarc` for DPX sequences.

6.1.3 FFmpeg and XMP

FFmpeg displays most metadata by default; however, some metadata, such as XMP, must be specifically enabled. XMP support in FFmpeg is also format-dependent and limited. The QuickTime/MOV muxer of FFmpeg can handle XMP via the `com.apple.quicktime.XMP` metadata key; however Matroska has no standardized XMP embedding path.

To display and/or map XMP metadata from a QuickTime/MOV input, the `-export_xmp` option, or the `-export_all` option must be provided to enable XMP metadata to be parsed and displayed or mapped.

6.1.4 FFREPORT

FFmpeg provides a built-in logging mechanism through the `FFREPORT` environment variable. When set, FFmpeg writes a detailed log to a timestamped file:

```
FFREPORT=file=ffmpeg_report_%t.log:level=32 ffmpeg -i input.mkv -map_metadata 0 -c copy output
```

The `%t` token generates a timestamp in the filename. Level 32 is informational; higher values produce more verbose output. The report includes:

- The full FFmpeg command line as invoked
- FFmpeg version, build configuration, and linked library versions
- Input file format detection and stream information
- Codec, bitrate, and container parameters for the output
- Any warnings or errors encountered during processing
- Processing duration and frame count

For provenance purposes, the `FFREPORT` log is a record of the initialization, processing, and outcome of the requested transformation. It documents what version of FFmpeg was used, exactly what command was run, what it found in the input, and what it produced. This log could be

retained as part of the preservation package and named to associate it unambiguously with the transformation it documents; for example, by embedding the output filename in the log path via `FFREPORT=file=${OUTPUT_BASENAME}_ffmpeg_%t.log`.

6.1.5 Mapping FFmpeg Logs to C2PA Actions

The FFREPORT log contains enough structured information to derive a C2PA actions assertion. An FFmpeg invocation that rewraps an MXF file to MOV corresponds to `c2pa.repackaged`. A transcode from ProRes to FFV1 corresponds to `c2pa.transcoded`. A post-processing script or wrapper tool could parse an FFREPORT log and construct a C2PA manifest from it, mapping the detected codec and container operations to the appropriate action labels and recording the FFmpeg version, command line, and timestamp as claim generator and action parameters.

The same approach applies to FFmpeg filterchains: a filterchain that applies scaling (`scale=1920:1080`) corresponds to `c2pa.resized`; one that applies a crop (`crop=1280:720:0:0`) corresponds to `c2pa.cropped`; one that applies color adjustments via `colorbalance` or `curves` corresponds to `c2pa.color_adjustments`. Filterchains are machine-readable and could be parsed systematically to construct action assertions.

6.1.6 Example: Deriving a C2PA Manifest from an FFmpeg Log

The following example demonstrates how a real FFmpeg log, in this case, the creation of an access derivative from a preservation file, can be analyzed and translated into a C2PA manifest. The FFREPORT log is reproduced below in abridged form, followed by an analysis of what it tells us, and the derived C2PA expression.

6.1.6.1 Source: FFmpeg Log (abridged)

```
ffmpeg started on 2026-08-11 at 13:36:03
Report written to "/ffmpeg/example/EXAMPLE123/logs/ffmpeg_20260811-133603.log"
Log level: 32
Command line:
/opt/homebrew/opt/ffmpeg-full/bin/ffmpeg -y -nostdin -v info -hide_banner -stats
-i "/ffmpeg/example/EXAMPLE123/Preservation_File.mxf"
-movflags faststart -pix_fmt yuv420p -c:v libx264 -force_key_frames chapters -f mp4
-crf 18 -maxrate 8760k -c:a aac -ac 2 -b:a 320k -ar 48000 -metadata "creation_time=now"
-filter_complex "yadif;[0:a:0][0:a:1]amerge=inputs=2,aformat=channel_layouts=stereo,loudnorm=1"
/ffmpeg/example/EXAMPLE123/Access_File.mp4

Input #0, mxf, from 'Preservation_File.mxf':
  Metadata:
    company_name      : Adobe Inc.
    product_name      : Adobe Media Encoder
    product_version   : 26.3.2
    application_platform: Mac OS X
    modification_date : 2026-08-06T15:32:27.000000Z
  Duration: 00:25:56.66, start: 0.000000, bitrate: 52650 kb/s
```



```
Stream #0:0: Video: mpeg2video (4:2:2), yuv422p(tv, unknown/unknown/bt709, top first),
1920x1080 [SAR 1:1 DAR 16:9], 50000 kb/s, 29.97 fps
Stream #0:1: Audio: pcm_s24le, 48000 Hz, mono, s32 (24 bit), 1152 kb/s
Stream #0:2: Audio: pcm_s24le, 48000 Hz, mono, s32 (24 bit), 1152 kb/s
```

Stream mapping:

```
Stream #0:0 (mpeg2video) > yadif:default
Stream #0:1 (pcm_s24le) > amerge
Stream #0:2 (pcm_s24le) > amerge
yadif:default > Stream #0:0 (libx264)
loudnorm:default > Stream #0:1 (aac)
```

```
[libx264 @ 0xa06c1d880] profile High, level 4.0, 4:2:0, 8-bit
264 - core 165 r3222 b35605a - H.264/MPEG-4 AVC codec
keyint=250 keyint_min=25 scenecut=40 rc=crf crf=18.0
```

Output #0, mp4, to 'Access_File.mp4':

```
Stream #0:0: Video: h264 (avc1), yuv420p(tv, unknown/bt709/bt709, progressive),
1920x1080 [SAR 1:1 DAR 16:9], 29.97 fps
Stream #0:1: Audio: aac (LC), 48000 Hz, stereo, fltp, 320 kb/s
```

[Parsed_loudnorm_3 @ 0xa06c15b00]

```
Input Integrated:    -22.1 LUFS
Input True Peak:     -7.0 dBTP
Input LRA:           9.0 LU
Output Integrated:   -24.1 LUFS
Output True Peak:    -6.3 dBTP
Output LRA:          6.5 LU
Normalization Type:  Dynamic
Target Offset:       +0.1 LU
```

frame=46653 fps=118 q=-1.0 Lsize=2663516KiB time=00:25:56.58 bitrate=14017.5kbits/s speed=3.95x

[libx264 @ 0xa06c1d880] frame I:415 Avg QP:15.31 size:275118

[libx264 @ 0xa06c1d880] frame P:13178 Avg QP:18.98 size:113888

[libx264 @ 0xa06c1d880] frame B:33060 Avg QP:22.11 size: 31998

[libx264 @ 0xa06c1d880] kb/s:13736.36

[aac @ 0xa06c1df80] Qavg: 50503.590

6.1.6.2 What the Log Tells Us Reading this log gives us context about the process to generate the access video file:

- **Input:** An MXF file containing MPEG-2 video at 1920×1080, 29.97fps, interlaced (top field first), 10-bit YUV 4:2:2 (yuv422p), 50 Mb/s, with two mono PCM audio tracks at 48kHz, 24-bit. Duration 25:56.66. The file was created by Adobe Media Encoder 26.3.2 on Mac OS X on 2026-08-06.
- **Video processing:** The yadif filter was applied to deinterlace the interlaced source to progressive. The pixel format was converted from yuv422p (10-bit 4:2:2) to yuv420p (8-bit

4:2:0) via the `-pix_fmt yuv420p` flag. The video was encoded to H264 (via libx264 encoder set to ‘High profile’ and level 4.0) at a constant rate factor (CRF) of 18 with a max bitrate of 8760 kb/s. Keyframes were forced at chapter boundaries (`-force_key_frames chapters`).

- **Audio processing:** Two mono PCM audio tracks (streams 0:1 and 0:2) were merged into a single stereo track via `amerge`, then loudness-normalized via `loudnorm` with measured input values producing a dynamic normalization of the audio output. The audio was encoded to AAC LC at 320 kb/s, 48kHz, stereo.
- **Container:** The output was muxed into MP4 with the `faststart` muxer option (which places the moov atom at the beginning of the output file for optimized streaming).
- **Processing context:** FFmpeg 8.0 (libavcodec 62.28.102, libavformat 62.12.102), running on Apple Silicon (ARMv8 NEON DotProd), x264 core 165. Processing took 6 minutes 34 seconds for 25 minutes 57 seconds of content (3.95× real-time). Total output size: 2,663,516 KiB (~2.5 GB).

6.1.6.3 Derived C2PA Manifest The following is a simplified JSON representation of the C2PA manifest that could be derived from this log. In an actual implementation, this would be encoded as CBOR within a JUMBF manifest store and signed with the institution’s signing credential. MP4 is a C2PA-supported format, so the manifest could be embedded in the output file. The input MXF is referenced as an ingredient.

```
{
  "claim_generator": "FFmpeg/8.0 (libavcodec 62.28.102)",
  "claim_generator_info": [
    {
      "name": "FFmpeg",
      "version": "8.0",
      "schema_version": "2.4"
    }
  ],
  "ingredients": [
    {
      "title": "Preservation_File.mxf",
      "document": {
        "format": "application/mxf",
        "creation_date": "2026-08-06T15:32:27Z"
      },
      "meta": {
        "company_name": "Adobe Inc.",
        "product_name": "Adobe Media Encoder",
        "product_version": "26.3.2",
        "application_platform": "Mac OS X"
      }
    }
  ],
  "assertions": [
    {
      "label": "c2pa.actions",
```

```

"data": {
  "actions": [
    {
      "action": "c2pa.transcoded",
      "when": "2026-08-11T13:36:03",
      "ingredientIds": [0],
      "description": "MXF preservation master transcoded to H.264/MP4 access derivative.",
      "parameters": {
        "input_container": "MXF",
        "input_video_codec": "MPEG-2 Video, 4:2:2 Profile, yuv422p, 10-bit",
        "input_audio_codec": "PCM s24le, 48kHz, 2x mono",
        "input_resolution": "1920x1080",
        "input_frame_rate": "29.97 fps",
        "input_interlaced": true,
        "input_field_order": "top first",
        "input_bitrate": "52650 kb/s",
        "input_duration": "00:25:56.66",
        "output_container": "MP4 (faststart)",
        "output_video_codec": "H.264, High profile, level 4.0, 8-bit, yuv420p",
        "output_audio_codec": "AAC LC, 48kHz, stereo, 320 kb/s",
        "output_resolution": "1920x1080",
        "output_frame_rate": "29.97 fps",
        "output_interlaced": false,
        "output_bitrate": "14017.5 kb/s",
        "output_duration": "00:25:56.58",
        "output_size": "2663516 KiB",
        "processing_time": "00:06:34.41",
        "processing_speed": "3.95x real-time",
        "ffmpeg_version": "8.0",
        "libavcodec_version": "62. 28.102",
        "libavformat_version": "62. 12.102",
        "x264_core": "165 r3222 b35605a",
        "platform": "Apple Silicon (ARMv8 NEON DotProd)",
        "command_line": "ffmpeg -y -nostdin -v info -hide_banner -stats -i [input.mxf] -r
linear=true:print_format=summary' [Access_file.mp4]"
      }
    },
    {
      "action": "c2pa.filtered",
      "when": "2026-08-11T13:36:03",
      "ingredientIds": [0],
      "description": "Deinterlace applied via yadif filter.",
      "parameters": {
        "filter": "yadif",
        "mode": "default",
        "input_scan_type": "interlaced (top field first)",
        "output_scan_type": "progressive"
      }
    }
  ]
}

```

```

},
{
  "action": "c2pa.color_adjustments",
  "when": "2026-08-11T13:36:03",
  "ingredientIds": [0],
  "description": "Pixel format converted from 10-bit yuv422p (4:2:2) to 8-bit yuv420p",
  "parameters": {
    "conversion": "pixel_format",
    "input_pixel_format": "yuv422p (10-bit, 4:2:2)",
    "output_pixel_format": "yuv420p (8-bit, 4:2:0)",
    "bit_depth_change": "10 to 8",
    "chroma_subsampling_change": "4:2:2 to 4:2:0"
  }
},
{
  "action": "c2pa.filtered",
  "when": "2026-08-11T13:36:03",
  "ingredientIds": [0],
  "description": "Audio channels merged and loudness-normalized.",
  "parameters": {
    "filters": [
      {
        "filter": "amerge",
        "inputs": 2,
        "description": "Two mono PCM tracks merged to stereo"
      },
      {
        "filter": "aformat",
        "args": "channel_layouts=stereo"
      },
      {
        "filter": "loudnorm",
        "normalization_type": "Dynamic",
        "measured_I": "-22.12 LUFS",
        "measured_LRA": "9.00 LU",
        "measured_TP": "-6.99 dBTP",
        "measured_thresh": "-32.81 LUFS",
        "offset": "-0.21 LU",
        "linear": true,
        "output_I": "-24.1 LUFS",
        "output_TP": "-6.3 dBTP",
        "output_LRA": "6.5 LU"
      }
    ]
  }
}
]
}

```

```

    },
    {
      "label": "c2pa.hash.data",
      "data": {
        "alg": "sha256",
        "hash": "a1b2c3d4e5f6...[SHA-256 of output MP4, excluding embedded manifest region]"
      }
    }
  ]
}

```

6.1.6.4 Notes on the Derivation In this example many arguments to the `ffmpeg` transcoding command play a role in provenance documentation. The derivative uses a lossy encoding and the audio is intentionally manipulated to adjust the volume. Though in an informally structured way, the `FFmpeg` log contextualizes these changes.

Action selection and ingredient requirements. For the `c2pa.transcoded` action the input MXF file is declared as an ingredient. This is a digital-to-digital transformation with an existing source file, so the ingredient chain is straightforward. The `c2pa.transcoded` action documents the container and codec conversion (MXF/MPEG-2 to MP4/H.264). Additional actions capture the specific transformations applied within the transcoding.

Filterchain decomposition. The log's `filter_complex` string, `yadif;[0:a:0][0:a:1]amerge=inputs=2,aformat=16000,aloudnorm`, contains two filterchains: one for video and one for audio. Each filter that modifies content can be mapped to a C2PA action. The `yadif` deinterlace filter maps to `c2pa.filtered` (content modification with no standard C2PA action for deinterlacing specifically). The pixel format conversion from 10-bit 4:2:2 to 8-bit 4:2:0 is a quality reduction to chroma subsampling and bit depth that maps to `c2pa.color_adjustments`. The audio filterchain (`amerge`, `aformat`, and `loudnorm`) is grouped under a single `c2pa.filtered` action with the individual filters nested as parameters. This preserves the ordering and relationship of the filterchain as a single audio processing pipeline.

loudnorm measurements as provenance data. The `loudnorm` filter's measured input and output values are preservation-relevant. They document the loudness characteristics of the source and the normalization applied and clarify that the preservation file and access file shall sound different. Recording these transformational details in the manifest could make them part of the cryptographically signed provenance record.

Command line as provenance. The full `FFmpeg` command line is recorded in the action parameters. This command describes the full process (in this case, of generating an access media file) and is machine-readable and reproducible with the transformational aspects of the process (such as loudness adjustments and deinterlacement) clarified.

What the manifest does not capture. The `ffmpeg` log records the embedded metadata of the MXF input, including the fact that it created by Adobe Media Encoder 26.3.2 on 2026-08-06, but the manifest only references this as ingredient metadata and does not establish a cryptographic chain to the input file unless the input file itself also carries a C2PA manifest. If the MXF preservation master had been signed with a C2PA manifest at the time of its creation (e.g., by the digitization tool or by a post-capture signing step), the ingredient entry would include that prior manifest, and

the chain would be cryptographically verifiable back to the digitization event. Without a prior manifest, the ingredient is declared with its metadata but the chain is not cryptographically linked. The manifest documents what FFmpeg did and what it received, but not what happened to the input before FFmpeg received it.

6.1.7 FFmpeg Forks with C2PA Features

While as of this writing no C2PA specific features are supported in FFmpeg, there are several works in progress worth acknowledging that seek to extend FFmpeg to add C2PA relevant features. Here's an example of one experimental fork on GitHub that adds C2PA capabilities to FFmpeg directly and may be worth following:

- **eqtylab/c2pa_ffmpeg** This is a mirror of the FFmpeg source tree with a `c2pa` branch that integrates C2PA signing into the encoding pipeline. A companion library, `eqtylab/c2pa_libffmpeg_eqty`, wraps the C2PA Rust SDK for use with this fork. It requires three parameters: `-c2pa_key` (certificate private key), `-c2pa_cert` (certificate), and `-c2pa_manifest` (a JSON manifest file defining the actions and assertions). It currently supports monolithic MP4 signing and signing of DASH segments. The work is experimental, not merged into FFmpeg's official codebase, and requires building from source against the C2PA Rust library; however, it demonstrates that in-process C2PA signing during FFmpeg encoding is architecturally feasible.

The fork is not production-ready or maintained at the level of mainline FFmpeg but is a proof-of-concept implementation, useful as reference material for understanding how C2PA signing can integrate with an encoding pipeline, and as a basis for any future effort to propose C2PA support for upstream FFmpeg. The existence of such forks also illustrates that C2PA adoption in the audiovisual workflows and ecosystems is still in early developmental stages and the community has interest in integrating it into core workflow tools.

6.1.8 Logging as Provenance

The FFREPORT log can serve as a meaningful provenance record for any FFmpeg transformation. This record can be modeled as a PREMIS event, an activity of type “migration,” “normalization,” or “metadata modification”, with the log file referenced as an event detail and input/output fixity values recorded as object characteristics. FFmpeg does not generate PREMIS events natively, but all the information needed to populate one is available in its outputs.

6.1.9 FFmpeg and TCR4CAP Criteria

The FADGI Audio-Visual Working Group's TCR4CAP (Tiered Community Recommendations for Content Authenticity and Provenance) effort, initiated in early 2026, is developing tiered community recommendations for CAP practices in digital audiovisual collections. Inspired by the NDSA Levels of Digital Preservation, the framework defines levels of practice ranging from basic integrity checks to more advanced implementations such as embedded provenance metadata or trust-center integrations. A draft for public comment is expected in summer 2026.

For FFmpeg, the following subsections map these emerging criteria to FFmpeg capabilities, illustrating how an institution can use FFmpeg to satisfy CAP requirements at multiple tiers of practice.

6.1.9.1 Basic Integrity Checks (Fixity) TCR4CAP context: The lowest tier of practice is verifying that file contents have not changed. This encompasses whole-file checksums, per-frame or intra-file fixity, and periodic re-verification.

FFmpeg examples:

- **Whole-file checksums (external to FFmpeg):** While FFmpeg itself does not compute whole-file checksums, it is standard practice to pair FFmpeg processing with system-level checksum generation. A potential workflow could run `md5sum` or `shasum -a 256` on the input before processing and on the output after, recording both values alongside the `FFREPORT` log.
- **Per-frame fixity via framemd5 muxer:** FFmpeg’s `framemd5` (or `framehash`) muxer generates per-frame checksums for video and per-chunk checksums for audio, producing a text file that lists the MD5 (or other algorithm) hash of each frame’s raw pixel data:

```
# produce a framemd5 report for the decoded streams of the input file
ffmpeg -i input.mkv -f framemd5 output.framemd5
```

```
# produce a framemd5 report for the encoded streams of the input file
ffmpeg -i input.mkv -f framemd5 -c copy output.framemd5
```

FFmpeg’s `framemd5` muxers can document the hashes of audiovisual streams as decoded by FFmpeg, or hash the encoded data as stored without decoding. These two procedures have different implications: generating `framemd5`s from decoded data documents the fixity of how a particular version of FFmpeg decoded particular encodings. The second example skips the decoding step by using the stream copy feature via `-c copy`; here, the `framemd5`s document the encoded data as stored.

As FFmpeg can provide intra-file fixity at the frame level, this provides a significantly stronger integrity guarantee and potential for resolution than a whole-file checksum, since it can identify exactly which frames have been altered if corruption occurs. See the upcoming `vrecord` section for an example that demonstrates generating `framemd5` concurrently during capture, binding the fixity record to the moment of creation.

6.1.9.2 Processing Documentation (Provenance of Transformations) TCR4CAP context: Beyond verifying that a file is unchanged, principles of CAP require documenting what was done to the file to detail its full the transformational history. This includes recording the tools used, the parameters applied, the time of processing, and the relationship between input and output.

FFREPORT structured logging: FFmpeg’s `-report` flag generates a structured log file containing the FFmpeg version, build configuration, command line, input and output stream mappings, codec parameters, filterchain details, and processing statistics:

```
ffmpeg -report -i input.mxf -c:v libx264 -crf 18 output.mp4
```

The resulting FFREPORT file is a complete processing record. As the worked example in this section demonstrates, every field needed to construct a C2PA `c2pa.transcoded` action (input codec, output codec, filters, encoding parameters, duration, frame count, bitrate, etc) is present in the log. The TCR4CAP framework’s processing documentation tier can be satisfied by retaining these logs, whether or not they are subsequently encoded as C2PA manifests.

Metadata embedding during processing: FFmpeg can embed descriptive and technical metadata in the output container during transcoding, creating an in-file record of the processing event:

```
ffmpeg -i input.mxf \
  -metadata "creation_time=2026-08-11T13:36:03Z" \
  -metadata "comment=Transcoded from MXF/MPEG-2 to MP4/H.264 for access. Source: TAPE_ID_1234" \
  -c:v libx264 -crf 18 output.mp4
```

While container metadata fields are limited and not cryptographically protected, they provide a human-readable provenance record embedded in the file itself, which serves as a baseline level of documentation.

ffprobe technical characterization: FFmpeg’s tool `ffprobe` can generate structured output (JSON, XML, CSV) documenting the complete technical properties of a file before and after processing:

```
ffprobe -v quiet -print_format json -show_format -show_streams input.mxf > input_probe.json
ffprobe -v quiet -print_format json -show_format -show_streams output.mp4 > output_probe.json
```

Paired `ffprobe` reports serve as before-and-after snapshots that document the transformation. These can be retained as PREMIS event documentation, stored as sidecar log files, or fed into a C2PA manifest generator as structured parameters.

6.1.9.3 Embedded Provenance Metadata (C2PA Manifests) TCR4CAP context:

There is the potential for “embedded provenance metadata” as a more advanced level C2PA manifests as cryptographically signed provenance records embedded in or attached to the file. Unlike basic integrity checks or processing logs, C2PA manifests are tamper-evident, structured according to a formal specification, and designed to chain across multiple processing steps.

FFmpeg examples:

- **FFmpeg as a source of C2PA assertion data:** (At the moment) FFmpeg does not generate C2PA manifests, but its logs and outputs can be primary data sources from which manifests can be constructed. The example earlier in this section demonstrated this derivation: the FFREPORT log provides the `c2pa.transcoded` action parameters, the `ffprobe` output provides the ingredient document metadata, and the output file’s hash provides the `c2pa.hash.data` assertion. A post-processing script (using the `c2pa` Rust SDK or `c2patool`) could read the FFmpeg log, construct the manifest JSON, sign it, and embeds it in the output file.
- **Embedding manifests in C2PA-supported output formats:** FFmpeg can write to formats that C2PA supports for embedding, including MP4 and WAV. When FFmpeg produces an MP4 access derivative, a subsequent C2PA signing step could embed the manifest directly in the file’s `moov` atom without re-encoding. The C2PA signing could be a separate operation that modifies the container structure without modifying the audio or video streams.

- **Sidecar manifests for unsupported formats:** When FFmpeg writes to formats that C2PA does not currently support for embedding (e.g., Matroska/MKV, DPX), the manifest could be managed as a sidecar file. FFmpeg’s `-f ffmetadata` muxer can export container metadata to a sidecar text file and could serve as the basis for a sidecar provenance record alongside a separately generated `.c2pa` manifest.

6.1.9.4 Format Validation and Conformance TCR4CAP context: Authenticity depends not only on documenting what was done but on verifying that the result conforms to expected specifications. A file that claims to be a preservation copy but does not conform to the relevant format standard adds a level complication and reliability issues.

FFmpeg examples:

- **FFmpeg as a conformance engine:** FFmpeg’s decoder libraries are the reference implementation for many codecs. If FFmpeg can decode a file without errors, then in some cases the file’s bitstream may be conformant to the codec specification. A simple decode test serves as a baseline conformance check:

```
ffmpeg -v error -i input.ffv1.mkv -f null -
```

The `-v error` flag suppresses all output except errors. A silent run indicates the file decodes without error. Any error messages may indicate bitstream non-conformance or corruption, a potential authenticity issue that could be documented.

- **QCTools XML generation:** FFmpeg’s metadata and filtering capabilities play a foundational role in QCTools, which generates detailed quality control reports including per-frame statistics (PSNR, spatial information, temporal information, loudness, etc.). These reports serve as a technical characterization of the file’s content at a specific point in time, providing a baseline against which future authenticity checks can be compared. If a file’s QCTools statistics change between captures, the content may be considered to have been modified.

6.1.9.5 Chain-of-Custody Documentation TCR4CAP context: The highest tier described in the FADGI project page is the integration of trust-centers where provenance claims are not just embedded in files but are verifiable against external registries or trust services. This requires that each processing step in an asset’s lifecycle produces a provenance record that references the prior step, creating a chain.

FFmpeg examples:

- **Ingredient declaration for chain construction:** When FFmpeg processes a file that already carries a C2PA manifest (e.g., a preservation copy that was signed at capture time), the post-processing C2PA signing step could declare the input as an ingredient with its prior manifest. This creates the cryptographic chain where the new manifest references the old manifest, which references the one before it, leading back to the point of origin.
- **FFREPORT as a PREMIS event record in a chain:** Even without C2PA, an institution can build a chain-of-custody record by retaining FFREPORT logs for every FFmpeg operation and by documenting them informally or via structuring them, for instance as PREMIS event records. Each log could record the input file’s checksum (if captured as a pre-processing step), the output file’s checksum, the processing tool and version, the command line, and

the timestamp. A PREMIS event record could reference the prior event for the same object, creating a documentary chain that mirrors what C2PA does cryptographically. TCR4CAP's tiered approach would recognize this as a mid-level practice, between simpler practices such as storage of a single checksum and more complex practices such as management of a signed manifest chain.

- **-metadata chaining for documentary provenance:** For workflows where C2PA is not yet implemented, FFmpeg's metadata embedding can create a lightweight provenance chain by carrying forward source identifiers and adding processing annotations at each step:

```
ffmpeg -i input.mp4 \  
-metadata "comment=Generation 2: transcoded from My_Example_Movie preservation copy. Prior c  
-c:v libx264 -crf 23 output_access.mp4
```

This is not cryptographically protected and can be manipulated or forged, but it provides a documentary chain that is readable by many media players or tools. In the context of the TCR4CAP's tiered framework this informal practice is better than no documentation, but not sufficient for authenticity guarantees in adversarial contexts.

6.2 BWF MetaEdit

BWF MetaEdit is a free and open source tool developed by the Library of Congress and FADGI (Federal Agencies Digital Guidelines Initiative) for embedding, validating, and exporting metadata in Broadcast WAVE Format audio files. It provides a metadata editor for embedded metadata features of Broadcast WAVE files, including the EBU BEXT chunk (EBU-TECH 3285) and LIST-INFO chunk specifications. BWF MetaEdit is designed closely to implement FADGI's guidelines for embedded metadata in BWF files. For audio digitization workflows, BWF MetaEdit helps ensure that the embedded metadata in a WAV file correctly documents the provenance of the recording.

6.2.1 Provenance-Relevant BEXT Fields

- **Originator:** The entity responsible for archiving the digital item.
- **OriginatorReference:** A unique identifier for the audio file.
- **OriginationDate / OriginationTime:** Date and time of the digitization event.
- **CodingHistory:** A structured field documenting the signal chain and processing history per EBU R98-1999. Each pass through the signal chain adds a row.
- **UMID:** A globally unique identifier per SMPTE ST 330.

The CodingHistory documents the transformational history of the audio recording, including its digital and analog sources, the signal chain used for digitization, and the parameters of the resulting digital file. For example:

```
A=ANALOG,M=stereo,T=VendorTapeDecoder; SN:12345; FADGI digitization project  
A=PCM,F=96000,W=24,M=stereo,T=AES/EBU interface
```

In this CodingHistory example, the first row documents the analog source; the second row documents the digital capture. If a subsequent transformation changes the file's encoding, such as a sample rate conversion, a new row would be added documenting the change.

6.2.2 The MD5 Chunk: Audio-Data Fixity

BWF MetaEdit supports the generation of an audio-data-only MD5 checksum, stored in a non-standardized chunk within the WAV file identified by the chunk ID `<MD5 >`. The checksum covers only the audio bitstream, which is the entire `<data>` chunk excluding the chunk identifier, size declaration, and any optional padding byte. This is a critical distinction from a whole-file checksum as subsequent metadata edits to the file would invalidate a whole-file checksum but should not invalidate the stored checksum of the audio chunk.

BWF MetaEdit exposes this feature through two operations:

- **Evaluate MD5 for audio data:** computes a checksum of the audio data and displays it in the `MD5Evaluated` column without storing it in the file. When an existing `<MD5>` chunk is present, BWF MetaEdit displays the stored value in the `MD5Stored` column on file open, and evaluation produces a comparison in the `MD5Evaluated` column, allowing immediate verification of audio data integrity.
- **Embed MD5 for audio data:** computes the checksum and stores it directly in the file in an `<MD5>` chunk. The declared size of this chunk is always 16 bytes (the size of an MD5 digest).

The audio-data MD5 is directly analogous to the image-data hashing strategy described in the em-ARC appendix for DPX files: it binds the preserved content (the audio bitstream) while allowing metadata to be freely edited without invalidating the fixity record. In a C2PA context, this same hash could serve as the basis for a `c2pa.hash.data` assertion with byte-range exclusions for the metadata chunks, or it could be referenced as supporting evidence in a `c2pa.actions` assertion confirming that only metadata was modified.

The FADGI guidelines describe the `<MD5>` chunk as “non-standardized” and “ad hoc”. While not part of the EBU BWF specification, it has become a de facto convention in audio preservation workflows and is widely supported by BWF MetaEdit across its CLI and GUI versions for Windows, macOS, and Linux.

6.2.3 Relationship to C2PA

WAV is a C2PA-supported format. BWF MetaEdit currently does not generate or read C2PA manifests, but it can document the digitization event in BEXT before a post-processing C2PA signing step embeds or attaches a manifest. The two operations, BEXT population and C2PA signing, could be complementary and sequenced items in a workflow.

For more information about the potential intersection of C2PA and BWF MetaEdit, please see the appendices of this report.

6.2.4 Example: A BWF MetaEdit Digitization Workflow with CAP Metadata

The following workflow describes how an institution could use BWF MetaEdit to embed provenance metadata and audio-data fixity in a batch of WAV files from an audio digitization project, and how those outputs relate to a subsequent C2PA signing step. BWF MetaEdit is available as both a GUI and a CLI tool; the CLI is used here because it is scriptable and suitable for batch processing.

6.2.4.1 Scenario An institution has digitized a collection of 1/4 inch open-reel audio tapes. The capture produced a set of WAV files at 96kHz, 24-bit, stereo. The files are stored in a project directory:

```
/audio_project/  
  Tape_001.wav  
  Tape_002.wav  
  Tape_003.wav  
  ...  
  Tape_050.wav
```

The technician needs to embed BEXT metadata documenting the digitization event, embed audio-data MD5 checksums, and prepare the files for a C2PA signing step that will generate signed manifests.

6.2.4.2 Step 1: Export Current Metadata (Baseline) Before making any changes, export the current state of all files as a baseline record:

```
bwfmetaedit --out-XML=/audio_project/metadata/before.xml /audio_project/*.wav
```

This produces an XML file containing the current BEXT, LIST-INFO, and technical metadata for every file in the directory. If the files were created by the capture software with minimal metadata, most BEXT fields will be empty. This baseline serves the same purpose as the `ffprobe` snapshot described in the FFmpeg section as it documents the state before modification so that changes can be diffed and verified.

6.2.4.3 Step 2: Prepare a Core Document with Provenance Metadata BWF MetaEdit supports importing metadata from a Core document (a CSV or XML file with one record per audio file). This is the recommended approach for batch embedding because it allows metadata to be prepared and reviewed before being committed to the files.

Create a CSV file (`/audio_project/metadata/provenance.csv`) with the following structure:

```
FileName,Originator,OriginatorReference,OriginationDate,OriginationTime,Description,CodingHistory  
Tape_001.wav,US-AudioArchive,US-AudioArchive-20260822-103000-001,2026-08-22,10:30:00,Quarter-inch  
Tape_002.wav,US-AudioArchive,US-AudioArchive-20260822-110000-002,2026-08-22,11:00:00,Quarter-inch  
Tape_003.wav,US-AudioArchive,US-AudioArchive-20260822-133000-003,2026-08-22,13:30:00,Quarter-inch  
...
```

Each row carries:

- **Originator:** The institution's FADGI-recommended identifier ([Country code]-[Entity name])
- **OriginatorReference:** A structured unique identifier encoding country, entity, date, time, and sequence number
- **OriginationDate / OriginationTime:** The date and time of the digitization event
- **Description:** A free-text description of the source item
- **CodingHistory:** The signal chain documentation, following EBU R98-1999 conventions

6.2.4.4 Step 3: Embed the Metadata Import the Core document to embed the provenance metadata in the BEXT chunks of all files:

```
bwfmetaedit --in-XML=/audio_project/metadata/provenance.xml /audio_project/*.wav
```

BWF MetaEdit reads the Core document, maps the field values to the appropriate BEXT and LIST-INFO elements, and writes the metadata into each file's header. The tool enforces FADGI validation rules during this process, rejecting values that do not conform to the guidelines (for example, an improperly formatted date or an Originator string that violates the recommended pattern).

6.2.4.5 Step 4: Embed Audio-Data MD5 Checksums After metadata is embedded, generate and store audio-data MD5 checksums:

```
bwfmetaedit --EmbedMD5 /audio_project/*.wav
```

This reads the audio data chunk of each file, computes an MD5 hash of the PCM bitstream (excluding chunk headers and metadata), and stores the 16-byte digest in an <MD5> chunk within the file. The checksum computation requires reading the entire data chunk, so for large files or large batches this step can take significant time.

6.2.4.6 Step 5: Verify the Results Export the metadata again and compare against the baseline to confirm that changes were applied correctly:

```
bwfmetaedit --out-XML=/audio_project/metadata/post_embed.xml /audio_project/*.wav  
diff /audio_project/metadata/before.xml /audio_project/metadata/post_embed.xml
```

The diff should show only the BEXT fields that were populated and the addition of the <MD5> chunk. No audio data fields should have changed.

To verify audio data integrity, bwfmetaedit can evaluate the stored MD5 against a new computation:

```
bwfmetaedit --EvaluateMD5 /audio_project/*.wav
```

BWF MetaEdit will display the stored value (MD5Stored) and the freshly computed value (MD5Evaluated) for each file. If the two match, the audio data has not been altered since the checksum was embedded and the operation provides a stable integrity check across metadata operations.

6.2.4.7 Step 6: C2PA Signing (Post-Processing) After BWF MetaEdit has embedded the BEXT provenance metadata and the audio-data MD5, a separate C2PA signing step could generate and embed a C2PA manifest in each WAV file. WAV is a C2PA-supported format, so the manifest can be embedded directly in the file rather than carried as a sidecar.

The C2PA manifest could reference the BEXT metadata as part of the digitization event documentation and include a `c2pa.hash.data` assertion binding the audio content. The audio-data MD5 already stored in the <MD5> chunk by BWF MetaEdit is not the same as a C2PA hash assertion as it is an MD5 stored in a non-standard chunk, whereas C2PA uses its own hash assertion format with specified algorithms and byte-range exclusions, however, the <MD5> chunk does serve as corroborating evidence. A validator examining the file could find both the BWF MetaEdit MD5 and the C2PA hash assertion, each independently confirming audio data integrity.

The signing step would require a separate tool (such as c2patool or the c2pa Rust SDK) and an institutional signing credential. BWF MetaEdit’s role in this example would end at Step 5; it has prepared the file with embedded provenance metadata and audio-data fixity, and the C2PA signing step could build on that foundation. The two operations would be complementary and sequenced: BWF MetaEdit populates the embedded provenance record, and C2PA signing wraps it in a cryptographically signed manifest that establishes a verifiable chain of custody from this point forward.

6.2.4.8 What This Workflow Achieves

Step	Tool	What It Documents
1. Baseline export	BWF MetaEdit	State of files before modification
2. Prepare Core document	Manual / script	Provenance metadata for each file (originator, date, signal chain)
3. Embed metadata	BWF MetaEdit	BEXT chunk populated with digitization provenance
4. Embed audio MD5	BWF MetaEdit	Audio-data-only checksum stored in <MD5> chunk
5. Verify	BWF MetaEdit	Confirm metadata applied, audio data unchanged
6. C2PA signing	c2patool / SDK	Signed manifest embedding provenance assertions and content hash

The result is a set of WAV files that carry two layers of provenance documentation: the BEXT metadata, which is readable by any BWF-compliant tool and documents the digitization event in human-readable structured fields; and the C2PA manifest, which is cryptographically signed and machine-verifiable, establishing a tamper-evident provenance chain from the moment of signing forward. The audio-data MD5 provides an additional fixity layer that bridges the two and supports the same claim that the C2PA hash assertion will attest to: that the audio content has not been modified.

6.3 vrecord

vrecord is an open-source video digitization and transfer tool developed within the audiovisual archiving community under the AMIA Open Source umbrella. It is designed for capturing analog and digital video signals through Blackmagic Design capture cards and producing preservation-quality digital files and built with archivists’ needs in mind.

6.3.1 vrecord in Digitization and Transcoding Workflows

vrecord uses FFmpeg internally for capture and encoding. Operator-configured settings for format, codec, sampling rate, audio channel mapping, and others are applied through an FFmpeg command that vrecord constructs and executes. vrecord also supports post-capture processing steps, including format conversion and quality control workflows, which involve FFmpeg transcoding and rewrapping

operations. All of the FFmpeg considerations above (`-map_metadata`, `FFREPORT`, filterchain logging) apply as well to `vrecord`'s FFmpeg invocations.

6.3.2 Provenance Through Capture Logging

For each recorded file, `vrecord` generates a structured set of log files:

- ****FFmpeg log:** (timestamped) the full FFmpeg encoding log for the session
- **Capture options log:** the full set of `vrecord` settings as configured at the time of capture
- **framemd5 file:** per-frame MD5 hashes for every frame in the captured file (when enabled)
- **QCTools XML:** comprehensive quality control data (when enabled)

This set of outputs is a more complete contemporaneous provenance record than most digitization tools produce. The capture options log documents the configuration that produced the file. The FFmpeg log documents the exact version and libraries used. Together they answer who captured the, when, with what equipment, with what settings, and with what tool version.

6.3.3 FrameMD5 as Intra-File Fixity

The `framemd5` output provides per-frame integrity verification that does not require C2PA. Given the `framemd5` file and the original video, any frame can be verified against its hash. For long-form content, multi-hour programs at 29.97fps, the `framemd5` file is large but is generated during capture at marginal additional cost.

`vrecord`'s log files should be treated as part of the preservation package and not as diagnostic output to be discarded. The capture options log in particular documents decisions that cannot be reconstructed after the fact.

6.3.4 Example: Deriving a C2PA Manifest from a `vrecord` Capture Session

The following example demonstrates how the logs from a `vrecord` capture session could be translated into a C2PA manifest. Unlike the FFmpeg transcoding example above, which modeled a digital-to-digital transformation with an existing source file as ingredient, this example models the origin of a digital asset within a videotape digitization event. The C2PA ingredients model was designed for digital-to-digital provenance chains, and this example illustrates how it encounters the analog-to-digital transition.

6.3.4.1 Source: `vrecord` Capture Options Log (abridged)

```
computer_name: Videos-Mac-mini-204.local
computer_model_id: Mac14,3
computer_memory: 16 GB
computer_cores: 8
operating_system_version: Darwin Kernel Version 23.6.0
user_name: videolabmini
vrecord version: 2026-07-07
datetime_start: 2026-07-08T16:31:32
```

```

capture_device_name: WD_BLACK SN850XE 4000GB
capture_device_location: External
FILE_PATH: /Volumes/FinalStore/Test/LC_10min_Framemd5_MP4_QCLI.mkv
video_card_name: DeckLink SDI 4K
VIDEO_INPUT_CHOICE: SDI
AUDIO_INPUT_CHOICE: SDI Embedded Audio
VIDEO_BIT_DEPTH_CHOICE: 10 bit
AUDIO_MAPPING_CHOICE: 2 Stereo Tracks (Channels 1 & 2 > 1st Track Stereo, Channels 3 & 4 > 2nd
STANDARD_CHOICE: NTSC
ASPECT_RATIO_CHOICE: 4/3
CONTAINER_CHOICE: Matroska
VIDEO_CODEC_CHOICE: FFV1 version 3
AUDIO_CODEC_CHOICE: 24-bit PCM
QCTOOLSXML_CHOICE: Yes, concurrent with recording
TECHNICIAN: MM
Record command: /opt/homebrew/bin/ffmpeg-ma -nostdin -nostats -t 600 -benchmark -benchmark_all
Playback command: /opt/homebrew/bin/ffplay-ma -v info -hide_banner -stats -autoexit -window_ti
Record exit code: 0
Playback exit code: 0
datetime_end: 2026-07-08T16:45:41
Decklink input buffer overrun: No
mediaconch_outcome: passed

```

6.3.4.2 Source: FFmpeg Log (abridged)

```

ffmpeg version 8.0 Copyright (c) 2000-2025 the FFmpeg developers
  built with Apple clang version 16.0.0 (clang-1600.0.26.4)
  libavutil      60.  8.100 / 60.  8.100
  libavcodec     62. 11.100 / 62. 11.100
  libavformat    62.  3.100 / 62.  3.100

Input #0, decklink, from 'DeckLink SDI 4K':
  Stream #0:0: Audio: pcm_s32le, 48000 Hz, 8 channels, s32, 12288 kb/s
  Stream #0:1: Video: v210, yuv422p10le(bottom first), 720x486, 223725 kb/s, 29.97 fps
  Metadata:
    timecode           : 10:33:58;09

Output #0, tee, to '[f=matroska]/Volumes/.../LC_10min_Framemd5_MP4_QCLI.mkv|...':
  Stream #0:0: Video: ffv1, yuv422p10le(tv, smpte170m/smpte170m/bt709, bottom coded first (swa
    720x486 [SAR 10:11 DAR 400:297], 29.97 fps
  Stream #0:1: Audio: pcm_s24le, 48000 Hz, 8 channels, s32 (24 bit), 9216 kb/s
  Stream #0:2: Audio: pcm_s24le, 48000 Hz, stereo, s32 (24 bit), 2304 kb/s
  Stream #0:3: Audio: pcm_s24le, 48000 Hz, stereo, s32 (24 bit), 2304 kb/s

Output #1, framemd5, to 'LC_10min_Framemd5_MP4_QCLI.framemd5':
  Stream #1:0: Video: rawvideo, yuv422p10le, 720x486, 29.97 fps

```


Output #2, mp4, to 'LC_10min_Framemd5_MP4_QCLI.mp4':

Stream #2:0: Video: h264 (avc1), yuv420p(tv, progressive), 720x480 [SAR 10:11 DAR 15:11], 35

Stream #2:1: Audio: aac (LC), 48000 Hz, stereo, fltp, 128 kb/s

Stream #2:2: Audio: aac (LC), 48000 Hz, stereo, fltp, 128 kb/s

frame=17944 fps= 21 q=-0.0 Lsize=N/A time=00:09:58.69 bitrate=N/A dup=3399 drop=0 speed=0.714x

6.3.4.3 What the Logs Tell Us These logs together provide a wealth of details about the digitization event:

- **What was captured:** An NTSC SDI video signal at 720×486, 29.97fps interlaced (bottom field first), 10-bit YUV 4:2:2 (yuv422p10le), with 8 channels of 48kHz 32-bit embedded SDI audio. The source signal carried SMPTE timecode 10:33:58;09. The properties of the video and audio aligns with the highest quality of NTSC multimedia streams that the Blackmagic capture card can provide. Prior to the Blackmagic capture card are more details such as the details of the source video tape, its playback device, any timebase corrector involved; however, this information is not documented in the current vrecord logs which start detailing the provenance chain at the capture card.
- **What produced it:** A vrecord session (version 2026-07-07) running on a Mac mini (Mac14,3, 16GB RAM, 8 cores, Darwin 23.6.0), using a DeckLink SDI 4K capture card, writing to an external SSD.
- **What FFmpeg did (preservation copy):** Captured the DeckLink SDI stream and encoded it to FFV1 version 3 (level 3, GOP=1, 24 slices with CRC) in a Matroska container, with 24-bit PCM audio mapped into two stereo pairs. The color metadata was set explicitly: SMPTE 170M primaries, BT.709 transfer, SMPTE 170M matrix, limited range. The sample aspect ratio was set to 10:11 for 4:3 NTSC.
- **What FFmpeg did (concurrent fixity):** A framemd5 file was generated concurrently with per-frame MD5 hashes for every frame in the captured video, providing intra-file fixity.
- **What FFmpeg did (concurrent QC):** A QCTools XML report was generated concurrently for quality control analysis.
- **What FFmpeg did (access derivative):** Simultaneously produced an H.264/MP4 access copy by cropping the video to 720×480 (removing 6 lines of vertical blanking: `crop=w=720:h=480:x=0:y=4`), deinterlacing with `bwdif`, converting to 8-bit YUV 4:2:0 (`format=yuv420p`), encoding to H.264 Main profile level 3.1 at 3500 kb/s, with AAC audio at 128 kb/s per stereo pair. The MP4 was written with `faststart` (moov atom moved to beginning for streaming) and color information embedded (`write_colr`).
- **Post-capture validation:** MediaConch validation passed (`mediaconch_outcome: passed`), confirming the resulting MKV file conforms to the expected Matroska/FFV1/LPCM specification.
- **Processing context:** FFmpeg 8.0 (libavcodec 62.11.100, libavformat 62.3.100), built statically with DeckLink support, running on Apple Silicon.

6.3.4.4 Derived C2PA Manifest The following is a simplified JSON representation of the C2PA manifest. In an actual implementation, this would be encoded as CBOR within a JUMBF manifest store and signed with the institution's signing credential. The manifest covers the collection of outputs produced by the capture session, the preservation MKV, the access MP4, the

framemd5 fixity file, and the QCTools XML report, with each file individually hashed. MKV is not currently a C2PA-supported format for embedding, so the manifest would be a sidecar file.

```
{
  "claim_generator": "vrecord/2026-07-07 (ffmpeg-ma/8.0)",
  "claim_generator_info": [
    {
      "name": "vrecord",
      "version": "2026-07-07",
      "schema_version": "2.4"
    }
  ],
  "ingredients": [
    {
      "title": "Analog video source (NTSC SDI signal)",
      "document": {
        "format": "video/sdi",
        "description": "Analog videotape playback via SDI connection to DeckLink SDI 4K capture"
      },
      "meta": {
        "digitalSourceType": "http://cv.iptc.org/newscodes/digitalsourcetype/analogOriginal",
        "signalStandard": "NTSC",
        "timecodeIn": "10:33:58;09"
      }
    }
  ],
  "assertions": [
    {
      "label": "c2pa.actions",
      "data": {
        "actions": [
          {
            "action": "c2pa.created",
            "when": "2026-07-08T16:31:32",
            "ingredientIds": [0],
            "description": "Video digitization capture from analog source via DeckLink SDI 4K.",
            "parameters": {
              "source_signal": "NTSC SDI",
              "capture_card": "DeckLink SDI 4K",
              "video_format": "720x486, 29.97fps, interlaced (bottom field first), yuv422p10le",
              "audio_format": "pcm_s32le, 48000 Hz, 8 channels",
              "timecode_in": "10:33:58;09",
              "technician": "MM",
              "capture_device": "WD_BLACK SN850XE 4000GB (External)",
              "capture_host": "Mac mini Mac14,3, 16GB, 8 cores, Darwin 23.6.0",
              "vrecord_version": "2026-07-07",
              "ffmpeg_version": "8.0, libavcodec 62.11.100",
              "ffmpeg_build": "Apple clang 16.0.0, static, DeckLink enabled"
            }
          }
        ]
      }
    }
  ]
}
```

```

    }
  },
  {
    "action": "c2pa.transcoded",
    "when": "2026-07-08T16:31:32",
    "ingredientIds": [0],
    "description": "DeckLink SDI v210 stream encoded to FFV1 version 3 in Matroska container",
    "parameters": {
      "input_codec": "v210 (raw 10-bit YUV 4:2:2 from DeckLink SDI)",
      "output_video_codec": "FFV1 version 3, level 3, GOP=1, 24 slices, CRC enabled",
      "output_audio_codec": "pcm_s24le, 48kHz",
      "output_container": "Matroska",
      "color_metadata": "primaries=smppte170m, transfer=bt709, matrix=smppte170m, range=full",
      "sar": "10:11 (NTSC 4:3)",
      "audio_streams": "4 streams: 1x 8-channel original, 2x stereo (ch1+2, ch3+4)",
      "concurrent_fixity": "framemd5 (per-frame MD5)",
      "concurrent_qc": "QCTools XML",
      "closed_captions": "EIA-608 extracted via readeia608 filter"
    }
  },
  {
    "action": "c2pa.transcoded",
    "when": "2026-07-08T16:31:32",
    "ingredientIds": [0],
    "description": "Access derivative: DeckLink SDI stream cropped, deinterlaced, and converted to H.264",
    "parameters": {
      "filters": [
        {
          "filter": "crop",
          "args": "w=720:h=480:x=0:y=4",
          "c2pa_action": "c2pa.cropped",
          "description": "Vertical blanking lines removed (6 lines from top: 486 to 492)"
        },
        {
          "filter": "bwdif",
          "description": "Bob-Wavering deinterlace (interlaced to progressive)"
        },
        {
          "filter": "format",
          "args": "yuv420p",
          "description": "Pixel format conversion: 10-bit yuv422p10le to 8-bit yuv420p"
        }
      ],
      "output_video_codec": "H.264, Main profile, level 3.1, 3500 kb/s",
      "output_audio_codec": "AAC LC, 48kHz, 128 kb/s per stereo pair",
      "output_container": "MP4 (faststart, write_colr)",
      "output_frame_rate": "59.94 fps (doubled from 29.97 by deinterlace)"
    }
  }
}

```

```

    }
  ]
}
},
{
  "label": "stds.device_capture",
  "data": {
    "capture_device": "Blackmagic DeckLink SDI 4K",
    "input": "SDI",
    "signal_standard": "NTSC",
    "video": {
      "width": 720,
      "height": 486,
      "frame_rate": "29.97",
      "interlaced": true,
      "field_order": "bottom first",
      "pixel_format": "yuv422p10le",
      "bit_depth": 10
    },
    "audio": {
      "sample_rate": 48000,
      "channels": 8,
      "bit_depth": 32,
      "source": "SDI Embedded"
    },
    "timecode": "10:33:58;09"
  }
},
{
  "label": "fadgi.capture_event",
  "data": {
    "technician": "MM",
    "vrecord_version": "2026-07-07",
    "ffmpeg_version": "8.0",
    "capture_start": "2026-07-08T16:31:32",
    "capture_end": "2026-07-08T16:45:41",
    "duration_captured": "00:09:58.69",
    "frames_captured": 17944,
    "frames_duplicated": 3399,
    "frames_dropped": 0,
    "processing_time": "00:13:59.03",
    "processing_speed": "0.714x real-time",
    "post_capture_validation": {
      "tool": "MediaConch",
      "outcome": "passed"
    },
    "concurrent_outputs": [
      "framemd5 (per-frame MD5 fixity)",

```

```

        "QCTools XML (quality control)",
        "EIA-608 closed captions (extracted to file)"
    ]
}
},
{
    "label": "c2pa.hash.collection.data",
    "data": {
        "alg": "sha256",
        "uris": [
            {
                "uri": "LC_10min_Framemd5_MP4_QCLI.mkv",
                "hash": "a1b2c3d4e5f6...[SHA-256 of preservation master]"
            },
            {
                "uri": "LC_10min_Framemd5_MP4_QCLI.mp4",
                "hash": "f7e8d9c0b1a2...[SHA-256 of access derivative]"
            },
            {
                "uri": "LC_10min_Framemd5_MP4_QCLI.framemd5",
                "hash": "1a2b3c4d5e6f...[SHA-256 of framemd5 fixity file]"
            },
            {
                "uri": "LC_10min_Framemd5_MP4_QCLI.qctools.xml",
                "hash": "aabbccdd1122...[SHA-256 of QCTools report]"
            }
        ]
    }
}
]
}

```

6.3.4.5 Notes on the Derivation For a walk-through of the potential for C2PA to record a vrecord digitization event, here is an overview of what is documented:

c2pa.created as the first action. The CAI SDK documentation states that “every manifest has to start with either an `c2pa.opened` or `c2pa.created` action, which has to be the first action in the manifest,” and that “each of these actions need to have an associated ingredient.” For a digitization capture from a potentially analog source, `c2pa.created` is the most fitting action with a digital asset being created with an analog source signal as its ingredient.

The analog source as ingredient. Here the C2PA ingredients model becomes an awkward fit. The specification’s ingredients model appears to be designed for digital-to-digital provenance chains, where an ingredient is a prior digital asset that may itself carry a C2PA manifest. An analog videotape signal has no prior manifest and analog material is generally incompatible with the forms of integrity management that can be applied to digital materials (such as cryptographic signatures, content hashing, and other forms of hard binding). The ingredient can be declared with its metadata (such as signal standard and timecode with the digital source type set to `analogOriginal` per

the IPTC NewsCodes vocabulary), but the `ingredientIds` array references an ingredient that carries no cryptographic manifest, only descriptive metadata. This is a genuine representation of the provenance chain: `vrecord` can document what it received and what it did, but it cannot cryptographically prove what happened to the analog source before it reached the SDI input. Within the context of a potential integration of `vrecord` and C2PA the provenance chain would begin at the capture card.

Two `c2pa.transcoded` actions. `vrecord` often generates multiple outputs from a single input, such as simultaneously creating a preservation master (FFV1/Matroska) and the access derivative (H.264/MP4). Each is a distinct transcode with different codecs, containers, processing, and intents. Rather than blending them into one action, in this example they are modeled as two separate `c2pa.transcoded` actions, both pointing to the same ingredient (the analog source) but with the details of each transcoding given its full distinction; i.e. the preservation-grade encoding using lossless FFV1 and full resolution whereas the access file encoding uses lossy H264, frame cropping, and downsampling.

Filterchain decomposition for the access derivative. The access derivative’s filterchain contains three filters, each documented as a nested parameter: `crop` (removing 6 lines of vertical blanking from 486 to 480), `bwdif` (deinterlacing from 29.97i to 59.94p), and `format=yuv420p` (converting 10-bit 4:2:2 to 8-bit 4:2:0). The `crop` filter maps to `c2pa.cropped`; the others are documented as descriptive parameters under the transcoded action.

What the manifest does not capture. The `vrecord` log records the full FFmpeg command line used for capture, including the complex filterchain syntax with `setparams`, `readeia608` (closed caption extraction), `asplit`, `pan` (channel mapping), and `setsar`. The command line is available in the `vrecord` log while the manifest itself records only the structured parameters. As with the FFmpeg transcoding example, the command line itself is pertinent to document, could be retained in the log file and bound to the manifest by hash, but is not necessarily duplicated in the manifest’s structured fields. A validator checking the manifest would confirm the log is unchanged since signing, then consult the log for the full command line detail.

Sidecar vs. embedded manifest. `vrecord` often uses Matroska as a preservation format; however Matroska is not currently a C2PA-supported format for embedding, while the access derivative (MP4) is C2PA-supported. In this example, the manifest is modeled as a sidecar covering the collection of outputs. Alternative approaches would be to embed a manifest in the MP4 (since MP4 supports embedding) and use a sidecar for the MKV or coordinate with C2PA and the IETF on establishing C2PA support for Matroska.

6.4 MediaInfo

MediaInfo is a widely used free and open-source tool for reading and reporting technical metadata from audiovisual files. It is not a metadata editing tool and does not participate in creating provenance records, but it is a critical instrument for verifying that provenance metadata exists, is correctly formed, and has survived transformations. Understanding what MediaInfo reports, and how it differs from the underlying format’s native field structure, is important for using it effectively in provenance workflows.

6.4.1 Standard MediaInfo Report

The standard MediaInfo output presents a cross-format normalized view of a file's technical metadata. It maps format-native field names and values to a common vocabulary that MediaInfo applies consistently across formats. As MediaInfo has a focus on interoperability and a cross-referenced audiovisual property vocabulary: a user can compare a DPX file and an MXF file in MediaInfo using the same field names. This normalization is useful for general analysis and assessment, but it means the reported field names are MediaInfo's own and may not correspond directly to the field names as they appear in the format specification. Values may also be interpreted, converted, or truncated to fit the common vocabulary.

The `-f` (or `--Full`) option on the command line produces a more complete output, including fields that MediaInfo populates but does not display in its default report. This surfaces additional technical and tag data that may be relevant to provenance analysis but hidden in the default view.

6.4.2 MediaTrace and `--Details=1`

MediaTrace is a separate report mode that expresses the binary architecture of a file as MediaInfo interprets it, field by field, with byte offsets, lengths, and raw values, in the context of the original format specification. It is generated using `--Details=1` on the MediaInfo command line:

```
# Standard report with all fields
mediainfo -f myfile.dpx

# MediaTrace output (binary structure, full field context)
mediainfo --Details=1 myfile.dpx

# MediaTrace in XML (recommended for scripted processing)
mediainfo --Details=1 --Output=XML myfile.dpx
```

The distinction between the standard report and MediaTrace is significant for provenance work. The standard report tells you what MediaInfo determined the value to be, normalized for cross-format comparison. MediaTrace tells you where in the file the value lives, what its raw value is, and what the format standard says the field means. For DPX, MediaTrace reports FADGI-relevant fields such as Creator (Field 12), Project name (Field 13), Creation date/time (Field 10), Source image date/time (Field 37), and the user-defined data area in their original byte positions and encoding. This is closer to a direct read of the format specification than the standard report.

6.4.3 MediaInfo Fields That Relate to CAP

MediaInfo reports several fields in its General section that have direct relevance to provenance documentation. However, it is worth noting that while MediaInfo defines these fields in its schema, implementations in the wild may use them inconsistently. A field that is defined in the format specification may be populated with values that differ from what a provenance workflow expects, or may be used for purposes other than what FADGI or the spec intends. This variability is a reason to prefer MediaTrace, which presents values in their native field context, over the standard report for provenance auditing.

Fields worth noting:

- **Encoded_Date / Tagged_Date / File_Created_Date:** MediaInfo surfaces multiple date fields, which can reflect the digitization date, the metadata tagging date, or the filesystem date depending on the format and implementation. In a format like DPX, where Field 10 (Creation date/time) and Field 37 (Source image date/time) carry distinct provenance meanings, the standard MediaInfo output may surface only one of these as **Encoded_Date** while MediaTrace shows both in their proper field context.
- **Encoded_Library / Encoded_Library_Name / Encoded_Library_Version:** Records the software used to create the file. Useful for identifying tool provenance but only as reliable as the tool that wrote it. Some encoders populate this field accurately; others use it for unrelated purposes or leave it empty.
- **OriginalSourceMedium:** Records the original source medium (film, tape, etc.). Availability and accuracy depend entirely on the originating tool and format.
- **Comment / Description:** Free-text fields that tools sometimes use to embed workflow notes, provenance information, or tool-specific data. The content is unpredictable across implementations.

For DPX specifically, MediaTrace is preferable over the standard report when the goal is auditing FADGI-compliant embedded metadata, because it surfaces the raw field values in the context of the format standard, the same context in which embARC validates them. The combination of embARC (for validation and correction) and MediaInfo MediaTrace (for inspection and documentation) provides a complete picture.

6.4.4 Scripted MediaInfo Output as Provenance Documentation

MediaInfo's `--Output=XML` and `--Output=JSON` modes produce machine-readable reports that can be ingested into PREMIS object records or other preservation metadata schemas. Running MediaInfo against files at defined workflow points, such as at ingest, after any transformation, and at periodic intervals in storage, produces a chronological record of the file's reported technical state. Differences between reports at different points can surface metadata changes, format changes, or unexpected modifications. This is a low-cost, broadly applicable provenance practice.

```
# Capture a full MediaInfo report in XML for a DPX sequence at ingest
for f in /scans/MyFilm_Reel1/*.dpx; do
    mediainfo --Output=XML "$f" >> ingest_mediainfo_report.xml
done
```

MediaInfo has recently added features to report on the presence or selected details of embedded C2PA manifests in a variety of file formats. As C2PA adoption grows in the preservation community, support for reading C2PA manifest sidecars alongside technical metadata in a single tool would be a useful development. At present, C2PA verification requires separate tooling such as `c2patool`.

6.5 embARC

embARC (Metadata Embedded for Archival Content) is a free and open source application developed by FADGI in conjunction with AVP and PortalMedia for auditing, validating, and correcting embedded metadata in DPX and MXF files. It is the primary tool for ensuring FADGI-compliant

embedded provenance at the sequence scale, and under this project’s statement of work it is the designated proof-of-concept implementation for CAP features.

6.5.1 embARC and Provenance Documentation

embARC’s core function, ensuring that DPX header fields accurately document the provenance of a scan, is itself a provenance practice. A DPX sequence in which Field 12 is correctly populated with the institution’s name, Field 37 correctly records the source material date, and Field 76 correctly encodes the digitization process history is a sequence with meaningful embedded provenance. embARC makes this achievable at the scale of a full-length film scan: correcting or populating these fields across 100,000 frames in a single batch operation, rewriting only the specific header byte ranges that need to change without touching image data.

6.5.2 Logging embARC Operations

embARC generates audit and correction reports documenting which fields were non-conformant, which corrections were applied, and across which files. These reports should be retained as provenance records as they document that a review process identified metadata problems and that a specific version of embARC applied specific corrections on a specific date. A recommended practice is to run an audit before any correction, export the report, perform corrections, and run a second audit to confirm results. The two reports together constitute a before-and-after record of the metadata state, equivalent to the `ffprobe` diff approach described for FFmpeg.

6.5.3 embARC and Content Authenticity

Because embARC does not touch image data, it can assert that any DPX sequence it has processed retains its original pixel content. The combination of an embARC audit report, pre- and post-operation fixity of the image data portion of each file, and a C2PA sidecar manifest generated by embARC would constitute a well-documented provenance chain for a corrected DPX sequence.

6.5.4 Planned C2PA Integration

As described in the companion appendix, embARC is planned to gain the ability to generate C2PA sequence-level sidecar manifests as part of its metadata operation workflow, and to display existing C2PA manifests in its audit interface. The sidecar approach uses `c2pa.hash.collection.data` with a hashing strategy that hashes only the image data region of each DPX file and excludes the header, mapping directly to embARC’s core guarantee that image data is not modified. This means the expensive I/O cost of hashing the image data of every frame in a long sequence is a one-time cost per sequence, not a recurring cost for each subsequent metadata correction. The manifest’s `c2pa.actions` assertion records which header fields were modified, with `image_data_modified: false` confirming that the image data hash should still match.

These additions will not change embARC’s existing DPX editing behavior. The initial release of C2PA features will follow community review and testing, per the process described in Section 3.2.2. Content authenticity and provenance updates to embARC will be subject to regression testing

and external user testing before release, consistent with the project’s release standards for major versions, and will not inhibit existing functionality.

7 Appendix: embARC and Content Authenticity and Provenance (CAP)

7.1 An Evaluation of embARC and CAP metadata

embARC (Metadata Embedded for Archival Content) is the FADGI-managed, open-source, cross-platform application for auditing, validating, correcting, and extending embedded metadata in DPX and MXF files. As part of this report the authors have examined embARC as an implementation of content authenticity and provenance features and have considered what opportunities exist to expand embARC’s functionality in that realm. This appendix documents the basis for those features, the options considered, the recommended approach, and implementation considerations.

Because SMPTE has a current project to map MXF to C2PA, this report is not exploring MXF workflows at this time. FADGI is unaware of similar SMPTE efforts for C2PA mapping for DPX.

embARC plays a specific role in preservation workflows surrounding the metadata embedded in DPX image sequences, generally coming from the output of a digital film scanner. DPX image sets can often be represented by hundreds of thousands of DPX image files for a feature film. As DPX image data is uncompressed each image can potentially require 50MB per image, or several terabytes per sequence, making DPX implementations often an outlier in media files in regards to per file count and total data size. The DPX header (as defined in SMPTE ST 268-1:2014 and ST 268-2:2023[extensions for High Dynamic Range and Wide Color Gamut]) carries technical and administrative metadata about the film scan and source material. The DPX metadata is stored in every file, generally containing identical data pertaining to the context of their creation. Though some fields are unique per DPX file in a sequence, such as incrementing image index numbers and embedded file names, most DPX metadata is redundantly stored across all DPX files in the sequence. FADGI has published [guidelines](#) that recommend how those fields should be populated to document digitization provenance. embARC audits DPX headers against both SMPTE conformance rules and the FADGI guidelines and corrects non-conformant fields in batch across an entire sequence.

The FADGI guidelines for DPX embedded metadata (first published 2016, last updated April 2019) extend SMPTE ST 268 in two ways. First, they specify recommended values and clarify formatting conventions for fields that SMPTE defines structurally but leaves expression styles to implementers. Second, FADGI defines the Field 75/76 user-defined data pair as a structured digitization process history, inspired by the Broadcast WAV Coding History field.

A significant, recent refactor of embARC changes how metadata changes are applied. Rather than rewriting entire files, embARC now rewrites only the specific header byte ranges that need to change, leaving the image data region of every file untouched. In past versions of embARC, the full DPX image set was rewritten with adjusted metadata, requiring a potential rewrite of terabytes of all DPX data rather than the bytes or kilobytes of data specific to the change requested. This transition adds to the provenance and authenticity approach of embARC as the rewriting of image data is avoided where possible rather than a requirement of a metadata change process. As discussed below, this aspect of embARC’s architectural design and the nature of DPX image sets relates to

how C2PA support could be implemented in embARC and why some otherwise reasonable options are impractical for both embARC and for DPX.

7.2 DPX Header Fields That Carry Provenance

embARC’s metadata corrections target a defined set of DPX header fields. Several of these fields are directly scoped to content authenticity and provenance and serve as a provenance record embedded to the file and is readable by any conformant DPX parser. The table below identifies the fields with particular CAP relevance, their FADGI tier status, and the authenticity role each plays.

Field #	Name	Byte Offset	FADGI Status	CAP Role
9	Image filename	36–135	Strongly recommended	This field is a backup of the DPX filename and associates a file to its position and order in the DPX sequence. This intentional redundancy supports auditing and checking filenames and sequence position and the discoverability of any inferred gaps in the file sequence.
10	Creation date/time	136–159	Strongly recommended	Records when the DPX file was created.
12	Creator	160–259	Strongly recommended	Identifies the institution or entity responsible for creating and preserving the DPX file.

Field #	Name	Byte Offset	FADGI Status	CAP Role
13	Project name	260–459	Strongly recommended	Carries identifiers for the work represented by the sequence and may link the file to a collection management record.
14	Right to use / copyright	460–659	Recommended	Documents rights assertions associated with the content.
37	Source image date/time	1532–1555	Recommended	Records the creation date of the source material (the original film).
38	Input device name	1556–1587	Recommended	Scanner manufacturer and model; part of the technical provenance of the scan.
39	Input device serial number	1588–1619	Recommended	Scanner serial number, enabling specific identification of the capture equipment.
43	Film manufacturing ID	1664–1665	Optional	Encodes data from film edge codes per SMPTE ST 254; as a reference to the manufacturer of the physical film source.
44	Film type	1666–1667	Optional	Film stock type from edge codes.

Field #	Name	Byte Offset	FADGI Status	CAP Role
75	User identification	2048–2079	Optional	This field has a local user-defined role, but in the context of the FADGI Guidelines this is a label for the “FADGI Process History” field (a FADGI recommended implementation of field 76).
76	FADGI Process History	2080+ (up to 1MB)	Optional	Structured multi-line record encoding the complete digitization chain. This field is inspired by the Broadcast WAVE best Coding History field and focuses on the documentation of provenance history.

This field functions as an embedded provenance record for the digitization event. A complete Field 76 example for a scanned 16mm positive:

```
O=positive, G=16mm, C=color, S=silent, F=24, A=4:3, D=warped
O=DPXv2, L=one-light, W=10-bit, R=2K, M=RGB Log, N=VendorName, T=ScannerModelX; SN:12345; in-h
O=DPXv2, W=10-bit, R=2K, M=RGB Log
```

7.3 C2PA and the DPX Format Support Problem

The C2PA specification does not define an embedding mechanism for DPX, at least as far as version 2.4. While SMPTE ST 268 does define a user-defined metadata area (Field 76), there is no standardized mechanism for embedding a JUMBF Manifest store in this user section. Additionally most DPX files generated from a scanner reserve much less space for Field 76 or possibly no space at all, thus potentially adding C2PA data into Field 76 could require an existing DPX file to be rewritten. When this data rewriting is scaled over a full DPX sequence the time and data size

of such a metadata addition can be far more burdensome than it would be for any of the other mentioned formats.

C2PA supports a more reasonable approach for DPX in the possibility of storing Manifest Stores external to the asset they describe, in the form of sidecar files.

7.3.1 Why Embedding C2PA Data in DPX Files Is Impractical

Initiated in 2026, embARC’s refactor established that corrections are applied by rewriting only the specific header byte ranges that need to change, leaving the image data region untouched. Embedding a JUMBF Manifest store in the DPX user-defined data area would require writing to a region of the file outside the specific header bytes being corrected, affecting byte offsets and potentially requiring validation of the full file structure. For a 100,000-frame sequence at roughly 50MB per frame, any approach that requires extending the metadata header can require terabytes of I/O for what would otherwise be a basic metadata operation.

There is also a challenge with C2PA Manifest sizing. A collection-level C2PA Manifest for 100,000 files could require 7-11MB of [CBOR](#)-encoded assertion data. Distributing Manifest content across 100,000 separate file headers would require a different Manifest per file, expanding every file in the sequence. A sidecar C2PA approach would achieve the same provenance goals without the burdensome costs associated with embedding such data.

7.4 4. Options Considered

Four options were considered for adding C2PA support to DPX.

Option 1: Per-file sidecar Manifests. Here a separate `.c2pa` file could be generated alongside each DPX file, with the same base filename. Each sidecar could contain a C2PA Manifest with a `c2pa.hash.data` assertion hashing the full DPX file (or the image data portion with appropriate exclusions), a `c2pa.actions` assertion recording what embARC did, and claim generator information identifying embARC. For a potentially 100,000 file DPX sequence this would produce 100,000 sidecar files, doubling the file count of the directory. Each sidecar would require an individual signing operation. Such an approach would be specification compliant but operationally burdensome at scale.

Option 2: Sequence-level sidecar Manifest using collection data hashing. Here a single `.c2pa` sidecar could be generated for the entire DPX sequence. The Manifest uses the `c2pa.hash.collection.data` assertion (see [C2PA spec §18.8](#)), which hashes each file in the sequence individually and stores the results in a single signed Manifest. Per the C2PA specification: “Each file in the collection shall be hashed individually using the specific hash algorithm defined in the `alg` field. The resultant hash value shall be stored in the `hash` field of the `uri-hashed-data-map` associated with the `uri` to the file.”

In this approach, there is one signing operation for the entire sequence, but one hash computation per file. **However, here the C2PA specification provides a significant constraint.** The `c2pa.hash.collection.data` assertion hashes the entire content of each file as the specification says “the hash shall be over all bytes (from 0 to `n`) of the content item — no exceptions.” The `uri-hashed-data-map` entries carry only the fields `uri`, `hash`, and optional `size`, `dc:format`, and `data_types`; however, there are no per-entry exclusions for the hashing. This is a significant difference from the single-file

`c2pa.hash.data` assertion, which does support an exclusions array of byte-range maps to skip over defined portions of a file such as an embedded Manifest or a format-specific header.

This means the DPX header cannot be excluded from the hash scope within a collection data hash assertion. As a result, any embARC operation that modifies DPX header fields will invalidate the hash for each frame, and those frames' entries in the Manifest must be recomputed and the Manifest re-signed after each embARC modification.

If header-exclusion is a hard requirement at the format level, the only C2PA-compliant path is `c2pa.hash.data` with per-file exclusions, which means one Manifest (or one Manifest ingredient) per DPX frame. Issues under this scenario are detailed under Option 3.

Option 3: Propose DPX as a C2PA-supported format. FADGI or other entities could consider proposing including DPX in the C2PA specification to define a mechanism for embedding C2PA data. DPX's user-defined data area could accommodate a JUMBF Manifest store. However, such an approach would be burdensome if it required extending the header to increase the user data section, so an efficient implementation of such an approach would require coordinating with the film scanners or tools that initiate the DPX file to reserve a sizable header section to hold the Manifest.

Option 4: Non-standard embedding in the user-defined data area. embARC could write a JUMBF Manifest store or a URI pointing to a Manifest into the DPX user-defined data area as an embARC/FADGI-specific convention, without waiting for formal standardization. Beyond the operational and input/output costs described above, this creates an informal implementation that may conflict with any future formal standardization. Additionally such an approach would be largely undiscoverable by other tools.

7.5 5. Recommended Approach: Sequence-Level Sidecar (Option 2)

A sequence-level sidecar approach using `c2pa.hash.collection.data` seems to be the most intuitive and efficient; however the lack of byte range exclusions for hashing within `c2pa.hash.collection.data` makes hard-binding a challenge without costly hash recomputations at each metadata edit. As the C2PA specification is currently evolving there may be some opportunity to collaborate with C2PA maintainers on clarifying a strategy that balances authenticity and efficiency. Option 2 also maps naturally to how DPX sequences are managed; as sequences rather than as individual files. The approach also produces one manageable authenticity document per sequence and is fully compliant to the related specifications and recommendations of SMPTE ST 268, the FADGI guidelines for DPX embedded metadata, and the C2PA Technical Specifications. This approach also permits per-file integrity verification without a requirement of per-file signing. Additionally this approach would not require any change to embARC's byte-range editing methodology as the sidecar C2PA file would be written alongside the sequence rather than into the DPX files.

If hard-binding is a requirement to C2PA feature development in embARC that option 2 with a coordinated resolution of the hard-binding limits would be the recommendation, else consideration could be given to the other specification compliant options.

Example sidecar placement for Option 2:

`/scans/`

```

Example_Film_Reel1/
  Example_Film_Reel1_000001.dpx
  Example_Film_Reel1_000002.dpx
  ...
  Example_Film_Reel1_100000.dpx
  Example_Film_Reel1.wav
  Example_Film_Reel1.c2pa

```

A simplified representation of the Manifest structure (actual encoding is CBOR/JUMBF):

```

{
  "claim_generator": "embARC/2.0",
  "claim_generator_info": [
    { "name": "embARC", "version": "2.0" }
  ],
  "assertions": [
    {
      "label": "c2pa.actions",
      "data": {
        "actions": [
          {
            "action": "to.be.defined.metadata_update",
            "description": "DPX header metadata updated per FADGI guidelines. Image data not m
            "parameters": {
              "fields_modified": [
                "Field 12 Creator",
                "Field 13 Project name",
                "Field 76 FADGI Process History"
              ],
              "image_data_modified": false
            }
          }
        ]
      }
    },
    {
      "label": "c2pa.hash.collection.data",
      "data": {
        "alg": "sha256",
        "uris": [
          {
            "uri": "MyFilm_Reel1/MyFilm_Reel1_000001.dpx",
            "hash": "e3b0c44298fc1c149afb..."
          },
          {
            "uri": "MyFilm_Reel1/MyFilm_Reel1_000002.dpx",
            "hash": "a665a45920422f9d417e..."
          }
        ]
      }
    }
  ]
}

```



```
    }  
  }  
]  
}
```

7.5.1 The Ingredients Problem at Scale

C2PA's ingredients model provides the methods to document custody chains. A new Manifest can reference prior Manifests from source materials as ingredients, preserving a verifiable provenance chain across multiple operations. For DPX sequences, this creates a specific problem.

A 2-hour feature film at 24fps produces approximately 172,800 individual DPX frames. If each file were treated as a C2PA ingredient in a subsequent operation, as the C2PA specification implies, then the resulting Manifest would contain 172,800 ingredient references. While the specification supports this architecturally, it can become operationally unwieldy and produces impractically-sized Manifests.

A practical alternative for consideration is to treat the prior sequence-level C2PA sidecar as a single ingredient. If a DPX sequence already has a sequence-level sidecar from a prior embARC operation, a subsequent operation could reference that sidecar as a single ingredient rather than referencing each frame individually. This method could model the semantics correctly, where the ingredient is the sequence rather than its individual frames.

Such an approach requires that the prior operation must have produced a sequence-level sidecar to reference. For existing collections with no prior C2PA history, the ingredients chain simply begins with the first embARC operation that generates a Manifest.

Advantages of the ingredient model: - Preserves a verifiable chain of custody across multiple operations on the same DPX sequence. - Allows validators to trace the provenance history of a sequence back through multiple embARC operations. - Scales the C2PA's design efficiently for multi-step workflows.

Cons at DPX scale: - If implemented as per-file ingredients, the process can produce unmanageable Manifests for long-form content. - Requires more consistent sidecar management discipline. - Nearly doubles the already substantial file counts involved in DPX sequences. - Chains that span format migrations (DPX to ProRes, for example) require C2PA support in both formats or a sidecar-to-embedded transition at the point of migration.

The sequence-as-ingredient model is the recommended approach and could be documented as a convention in embARC's C2PA implementation guidance.

7.6 C2PA Actions in Scope for embARC

Given that the design of embARC specifically avoids any edit to image content but only focuses on edits to the DPX header metadata, only a subset of the C2PA actions vocabulary is relevant.

The C2PA standard defines a set of actions including `c2pa.created`, `c2pa.opened`, `c2pa.edited`, `c2pa.resized`, `c2pa.cropped`, `c2pa.color_adjustments`, `c2pa.transcoded`, `c2pa.repackaged`, `c2pa.placed`, `c2pa.removed`, and `c2pa.redacted`. Of these, `c2pa.repackaged` is the closest standard action to a metadata-only header edit as it is defined as changing the container file format

without transcoding. However, the specification defines **repackaing** as a move from one container to another. While this defines how original versions of embARC worked, by rewriting DPX files in full upon metadata modifications; however, recent versions of embARC edit metadata in place which doesn't fit the C2PA's description on repackaging actions.

A custom FADGI-defined action, such as **to.be.defined.metadata_update**, could be more semantically precise for embARC's operations and would avoid the ingredient requirement that **c2pa.repackaged** imposes. Custom actions are permitted by the specifications and defining a FADGI action vocabulary for archival metadata operations could form the basis of practical community contributions or proposals to ongoing C2PA specification development.

Actions in scope for embARC: - Metadata update (custom **to.be.defined.metadata_update**): embARC updates DPX header fields without modifying image data. - Batch header correction: same as above, applied across a sequence as a single action. - Audit/validation: embARC auditing and other actions to assess or report on DPX sequences without modifying them.

Actions out of scope: - **c2pa.edited**: implies content modification which is out of scope to embARC. - **c2pa.transcoded**: implies format or encoding conversion which is out of scope to embARC. - **c2pa.color_adjustments**: implies pixel modifications that are out of scope to embARC. - **c2pa.created**: embARC does not create DPX files, but if the DPX creator generated a sequence-level C2PA Manifest sidecar, then embARC could document subsequent actions there. Additionally, initializing a C2PA Manifest at the point of creation would facilitate more efficient hashing.

Action parameters should always record **image_data_modified: false** to make explicit that only header data was changed.

7.7 embARC as Claim Generator vs. Manifest Consumer

embARC can participate in C2PA in two distinct roles.

As a claim generator: As embARC performs a metadata operation on a DPX sequence, it could generate a C2PA sidecar Manifest documenting the action, which fields were modified, and which files were affected, with each file individually hashed. The Manifest could also be signed with the institution's signing credential. This is the provenance-creating role and requires decisions about signing infrastructure and credential management.

As a Manifest consumer: embARC could read and display existing C2PA sidecar Manifests associated with DPX sequences, presenting the provenance history alongside its assessment results. This allows operators to see not just the current state of the header metadata but the history of documented operations. This role requires no signing credential.

The two roles could be considered and/or implemented separately. Displaying existing C2PA Manifests is certainly the simpler feature to develop and could be implemented first as a read-only feature in an initial release. Generating and signing Manifests could follow, after institutional signing infrastructure and naming conventions are established.

7.8 Implementation Notes and Release Considerations

The current development plans for embARC include at least one major release for all platforms and builds (Windows and Mac, CLI and GUI) with regression testing to demonstrate application stability and external user testing for overall functionality feedback. Minor versions follow as needed for bug fixes, security patches, and limited improvements.

Potential read-only Manifest read and display features would be a contained change that can be regression-tested against existing audit and correction behavior with minimal risk of interference. The Manifest generator feature would be an addition to the existing embARC workflows and can be controlled via a configuration option so that existing workflows that do not use C2PA are unaffected. Neither feature requires changes to embARC's byte-range editing model, which is core to its efficiency and for reducing risk to image data.

7.9 A Draft Proposal of embARC Command Line Options for C2PA

embARC's CLI currently supports batch metadata header auditing and updating. The following proposed flags extend that interface to support both C2PA claim generation and Manifest consumption, with no impact on existing option behavior.

7.9.1 Options for C2PA Manifest Consumer

- `-c2pa-read` Read and display any C2PA sidecar Manifest associated with the input DPX sequence. Presents provenance history alongside embARC's standard audit results.
- `-c2pa-read-format=text|json|xml` Output format for the Manifest report produced by `-c2pa-read`.
text Human-readable summary (default) json Machine-readable JSON representation of the Manifest xml XML representation suitable for integration with embARC's existing XML export pipeline
- `-c2pa-validate` Verify the cryptographic signature and hash integrity of an existing C2PA sidecar against the current state of the DPX sequence files. Reports which files pass or fail their hash check and whether the Manifest signature is valid. Exit code is non-zero if any file fails verification and the verification status is summarized.
- `-c2pa-sidecar=` Explicitly specify the .c2pa sidecar file to read or validate, rather than relying on the default discovery convention (sequence directory / sequence name .c2pa).

7.9.2 Options for C2PA Claim Generation

`--c2pa-generate`

Generate a sequence-level C2PA sidecar Manifest (.c2pa) after completing any metadata audit or correction operation. The Manifest is written alongside the DPX sequence directory. Requires `--c2pa-sign` or a configured signing credential in the embARC configuration file.

`--c2pa-sign=<credential_path>`

Path to the signing credential (PEM-encoded certificate and private key, or reference to a KMS/HSM endpoint) used to sign the generated Manifest.

Required when `--c2pa-generate` is specified and no default credential is configured.

note that `--c2pa-hash-scope` would only be relevant for C2PA documentation of individual DPX :

`--c2pa-hash-scope=image-data|full-file`

Controls which portion of each DPX file is hashed in the `c2pa.hash.collection.data` assertion.

`image-data` Hash only the image data region, excluding the DPX header bytes. This allows subsequent header-only edits by embARC without requiring rehashing (default).

`full-file` Hash the complete DPX file including the header. Any subsequent header edit will invalidate these hashes.

`--c2pa-alg=sha256|sha384|sha512`

Cryptographic hash algorithm to use for the collection data hash assertion. Defaults to `sha256`.

`--c2pa-action=<label>`

Override the default action label recorded in the `c2pa.actions` assertion. Defaults to `'to.be.defined.metadata.update'`.

`--c2pa-description=<text>`

Free-text description to include in the action's description field, supplementing the structured parameters recorded automatically.

`--c2pa-ingredient=<sidecar_path>`

Reference an existing C2PA sidecar from a prior embARC operation as an ingredient in the new Manifest. Enables chain-of-custody documentation across multiple operations on the same sequence. If omitted, a new chain is started.

`--c2pa-no-sign`

Generate an unsigned Manifest structure for review or testing. The resulting `.c2pa` file is not valid for verification but can be inspected before committing to a signing operation.

`--c2pa-out=<path>`

Write the generated sidecar to an explicit path rather than the default (sequence directory / sequence name `.c2pa`).

7.9.3 embARC C2PA Command Line Examples

Audit sequence, correct header fields, and generate a signed C2PA sidecar:

```
embarc --correct --fields=12,13,76 \  
  --c2pa-generate --c2pa-sign=/etc/embarc/fadgi_embarc_signing.pem \  
  /scans/Example_Film_Reel1/
```

```
# Generate a sidecar that chains from a prior operation:
embarc --correct --fields=76 \
    --c2pa-generate --c2pa-sign=/etc/embarc/fadgi_embarc_signing.pem \
    --c2pa-ingredient=/scans/Example_Film_Reel1/Example_Film_Reel1.c2pa \
    /scans/Example_Film_Reel1/

# Read and display existing provenance alongside an audit report:
embarc --audit --c2pa-read /scans/Example_Film_Reel1/

# Validate sidecar integrity after transfer:
embarc --c2pa-validate /scans/Example_Film_Reel1/
```

8 Appendix: BWF MetaEdit and Content Authenticity and Provenance (CAP)

8.1 An Evaluation of BWF MetaEdit and CAP Metadata

BWF MetaEdit is an open-source, cross-platform application for importing, editing, embedding, and exporting metadata in Broadcast WAVE Format (BWF) audio files. BWF MetaEdit supports editing the embedded metadata elements of the bext (Broadcast Extension) chunk as defined by EBU Technical Document 3285, LIST-INFO chunks, the cues chunk, aXML chunk, iXML chunk, and a selection of others. BWF MetaEdit integrates a number of metadata policies and recommendations (particularly the FADGI Guidelines for Embedded Metadata in Broadcast WAVE Files) and permits the user to select which policies to adhere to. Unlike DPX sequences, which may span hundreds of thousands of individual image files, a BWF audio file is a single self-contained file, making C2PA integration more feasible. The C2PA Technical Specifications also define how to embed C2PA Manifests into a BWF file.

8.2 BWF’s Embedded Metadata and Provenance Fields

The bext chunk carries several fields directly relevant to provenance documentation:

Field	Role
Description	Free-text description of the audio content or the digitization event.
Originator	Institution or entity responsible for the file.
OriginatorReference	Unique identifier for the file within the originator’s system.

Field	Role
OriginationDate / OriginationTime	When the file was created or captured in time.
TimeReference	A timeline reference or expression of timecode.
CodingHistory	A multi-line structured record of the signal chain and processing history.
UMID	A Unique Material Identifier, ideally providing globally unique identification.

BWF MetaEdit also supports evaluating and embedding an MD5 checksum of the audio data chunk (the data chunk payload only, not the full file), providing a built-in integrity binding to the audio bitstream independent of the surrounding RIFF/WAVE structure. This MD5 feature is directly analogous to C2PA’s hard-binding hash assertion.

8.3 C2PA and the WAV Format

WAV (and by extension BWF) is a C2PA-supported format. The C2PA Content Credentials Technical Specification defines an embedding mechanism for WAV files, allowing a JUMBF Manifest store to be embedded directly in the file.

BWF MetaEdit’s current architecture applies metadata edits by rewriting the metadata chunks in place or, moving the metadata chunk to the end of the file when more space is needed, or, in certain cases, by rewriting the full file. The last option is generally avoidable as a full file rewrite significantly increase the amount of time for a metadata operation in BWF MetaEdit, but if a user opts to follow an EBU recommendation that the bext chunk precede the data chunk (selected via a preference setting) then oftentimes a full rewrite of the file is required. An embedded C2PA Manifest is sensitive to any file rewrite, since the Manifest’s hard-binding hash covers a defined byte range of the file. Any subsequent bext edit that rewrites the file would invalidate a previously embedded Manifest unless the Manifest is regenerated as part of that operation.

8.4 Options Considered

Option 1: Embedded C2PA Manifest in the WAV JUMBF box. Since WAV is a supported C2PA format, BWF MetaEdit could embed a full JUMBF Manifest store directly in the file. This is Content Credential specification-compliant and allows any C2PA-capable validator to verify the file without requiring a sidecar. The challenge here would be that any metadata edit to the BWF file that would require a rewrite the data chunk (and subsequently adjust the byte offset for the data chunk) also would require that the exclusion ranges be recalculated even if the hash of the audio

data chunk did not change. BWF MetaEdit’s own MD5 chunk feature anticipates this pattern (metadata edits don’t affect the audio data chunk, so the MD5 stays valid), but a C2PA hash for the audio data of the file would also have to properly manage byte exclusion ranges to clarify where the audio data is. A potential version of BWF MetaEdit with features to embed and maintain C2PA Manifests could include information to the user on the consequences of editing metadata within BWF files while attempting to adhere to the EBU policy to store the bext chunk before the data chunk.

The C2PA Technical Specification requires that the C2PA chunk be placed last within the overall RIFF chunk of the BWF file. This implies that any metadata edit which requires additional space than what is stored would require BWF MetaEdit to both rewrite the full C2PA chunk and update the byte exclusion ranges.

With this option, BWF MetaEdit could also manage a preference on whether a metadata edit should be noted in an existing C2PA chunk or not. For example, for a BWF with embedded C2PA, a user could edit a bext field (if not requiring a new offset for the data chunk) and the C2PA data could still be verified as per its signature and hard binding to the audio data. However, to participate fully in the provenance chain and authenticity features of C2PA, BWF MetaEdit should add claims to the C2PA Manifest to document its action to modify metadata. It is recommended that BWF MetaEdit’s claim generation be a feature that can be disabled or enabled.

For WAV files, C2PA uses a `c2pa.hash.data` hard binding with byte-range exclusions expressed as absolute (start position and length in bytes) offsets into the file. When BWF MetaEdit edits the bext chunk and bext is positioned before the data chunk (the EBU R85 default), any change to bext’s size could shift the data chunk byte offset, invalidating the exclusion ranges in an existing Manifest even though the audio bytes themselves are unchanged. In this configuration, the hash must be recomputed with updated exclusion ranges after each bext edit.

Unfortunately even if bext is positioned after the data chunk (BWF MetaEdit’s optional ‘Place new or expanded BEXT or LIST-INFO chunks at the end of the file’ setting), any extension of the file to fit metadata edits would generate bytes not covered by prior exclusion ranges even if the data chunk’s maintains the same offset and data. For example a 2,000 byte BWF file may include an exclusion range of 0-512 to include the RIFF header, fmt chunk, and a bext chunk, and then another exclusion range of 1700-2000 to exclude an existing C2PA chunk that follows the data chunk. If a BWF MetaEdit operation needs to extend the bext chunk, the file would be rewritten by voiding the existing bext chunk (by writing ‘junk’ or ‘skip’ over the ‘bext’ chunk identifier) and then to add a new bext chunk to the end and then rewrite the C2PA chunk after that new bext chunk since C2PA requires that the C2PA chunk be last in order. With such a BWF MetaEdit operation the file size would increase beyond 2,000 bytes and any new bytes would fall outside the original C2PA’s exclusion ranges and thus fail verification. **This could be resolved if C2PA exclusion ranges could incorporate an end of file (EOF) marking instead of a specific offset number, but such relative EOF expressions are not currently supported in C2PA exclusion ranges. Another strategy to avoid recalculating or voiding exclusion ranges would be to reserve sufficient space in BWF files to accommodate extensions, but this can also lead to wasted space.**

Option 2: WAV sidecar Manifest. A sidecar with the audio data hashing keeps the original Manifest cryptographically valid across bext edits (the audio bytes it bound to are unchanged), but C2PA’s provenance model expects each editing action to be documented by a new signed update Manifest. So while the original Manifest doesn’t break, a fully C2PA-compliant workflow

still produces a new update Manifest, and recomputes the audio-data hash, for each bext edit. The re-signing operation itself is cheap (milliseconds), but the hash re-computation requires one full audio-data read per edit. For workflows with iterative bext refinement, batching edits and generating one update Manifest per session, or generating the Manifest only at workflow completion, is recommended to avoid per-edit hash overhead.

Option 3: Hybrid — sidecar for initial operations, embedded for finalization. For active working files undergoing iterative bext corrections, another option could be to maintain a sidecar Manifest that hashes the audio data chunk. Then at the point of final delivery or deposit, the full Manifest could be regenerated and embedded in the WAV file itself, covering the complete final state of the file. This provides the benefits of embedded provenance for long-term preservation while avoiding re-sign overhead during active editing.

8.5 Recommended Approach

Option 1 is recommended for BWF MetaEdit’s initial C2PA implementation. This could require the definition of an action label for BWF MetaEdit operations. Options would need to be added in order to coordinate the relationship between user preferences to follow or ignore the EBU recommendation to place the bext chunk before the data chunk and to whether or not to use C2PA to document metadata changes. Also BWF MetaEdit would set `audio_data_modified` to false.

8.6 Relationship Between BWF MetaEdit’s MD5 and C2PA Hard Binding

BWF MetaEdit can already embed a MD5 checksum to document the data chunk, providing integrity verification of the audio bitstream scoped to the same region C2PA’s audio-data-scoped hash would cover. These two mechanisms are complementary rather than redundant:

Feature	BWF MetaEdit MD5	C2PA Hard Binding
Scope	Audio data chunk only	Configurable (full file or audio data chunk only)
Algorithm	MD5 (legacy; collision vulnerabilities known)	SHA-256 or stronger
Signed	No — embedded hash only, no signature	Yes, cryptographically signed by a credentialed actor
Verifiable by third parties	Only with the file and knowledge of the convention	Yes, by any C2PA validator
Chain of custody	No	Yes, ingredients model supports multi-step provenance

BWF MetaEdit could document this relationship explicitly in its C2PA output; for example, recording the existing MD5 value as an assertion parameter alongside the C2PA hash, for cross-reference by validators who know the BWF MetaEdit convention.

8.7 Proposed Command Line Options for BWF MetaEdit C2PA Integration

8.7.1 Claim Generator Options

`--c2pa-embed`

Generate and embed a C2PA Manifest after completing the requested metadata operation. Requires a configured signing credential.

`--c2pa-sign=<credential_path>`

Path to the signing credential (PEM-encoded certificate and private key, or reference to a KMS/HSM endpoint). Required when `--c2pa-embed` is specified without a configured default.

`--c2pa-alg=sha256|sha384|sha512`

Hash algorithm for the hard-binding assertion. Defaults to sha256.

`--c2pa-action=<label>`

Override the default action label. Defaults to `'to.be.defined.metadata_update'`.

`--c2pa-description=<text>`

Free-text description for the action's `description` field.

`--c2pa-include-md5`

Include the existing BWF MetaEdit MD5 value (if present in the bext chunk) as an assertion parameter in the generated Manifest, for cross-reference by validators.

`--c2pa-ingredient=<sidecar_path>`

Reference an existing C2PA sidecar from a prior BWF operation as an ingredient. Enables chain-of-custody when using BWF across a range of applications which include C2PA sidecar generation.

`--c2pa-no-sign`

Generate an unsigned Manifest for review. Not valid for verification.

`--c2pa-out=<path>`

Write the sidecar to an explicit path.

8.7.2 Manifest Consumer Options

`--c2pa-read`

Read and display any C2PA sidecar or embedded Manifest associated with the input WAV file(s). Presented alongside BWF MetaEdit's `standard metadata report`. Read-only; no file modification. No signing credential required.

```

--c2pa-read-format=text|json|xml|csv
    Output format for the Manifest report.
    text    Human-readable summary (default)
    json    Machine-readable JSON
    xml     XML, suitable for inclusion in BWF MetaEdit's existing XML
            export pipeline
    csv     CSV row(s) appended to BWF MetaEdit's standard CORE export

--c2pa-validate
    Verify the cryptographic signature and hash integrity of an existing
    C2PA Manifest against the current state of the WAV file(s). Reports
    pass/fail per file and overall signature validity. Exit code is non-zero
    if any file fails.

--c2pa-sidecar=<path>
    Explicitly specify the .c2pa sidecar file to read or validate.

```

8.7.3 Example Commands

```

# Edit bext fields and generate a signed C2PA sidecar:
bwfmetaedit --Originator="Library of Congress" \
            --CodingHistory="A=ANALOGUE,M=mono,T=Studer A810;SN:12345" \
            --c2pa-generate --c2pa-sign=/etc/bwfmetaedit/loc_signing.pem \
            Example_Recording_001.wav

# Embed MD5 (audio data) and generate a C2PA sidecar in the same pass:
bwfmetaedit --MD5-embed --c2pa-generate --c2pa-include-md5 \
            --c2pa-sign=/etc/bwfmetaedit/loc_signing.pem \
            Example_Recording_001.wav

# Read existing provenance alongside standard metadata report:
bwfmetaedit --out-core --c2pa-read Example_Recording_001.wav

# Validate integrity after transfer:
bwfmetaedit --c2pa-validate /audio/Example_Session/*.wav

```